

Object-Oriented Programming with C++

Course Objectives

- Understand core principles of Object-Oriented Programming
- Apply OOP concepts using C++ syntax and features
- Design and implement classes, objects, inheritance hierarchies, and polymorphic behavior
- Use advanced features: templates, exception handling, file I/O
- Solve Purwanchal University exam-level problems in C++

Chapter Overview

Ch.	Title	Hrs
1	Introduction to OOP	3
2	Basic Syntax of C++	2
3	Functions in C++	3
4	Classes and Objects	5
5	Friend Function	4
6	Constructor & Destructor	4
7	Operator Overloading	5
8	Inheritance	5
9	Polymorphism	4
10	File Handling	5
11	Advanced Topics	5

Introduction to OOP

- OOP vs Procedural-Oriented Programming (POP)
- Core Features: Objects, Classes, Encapsulation, Abstraction
- Inheritance, Polymorphism, Dynamic Binding, Message Passing
- Applications and Benefits of OOP

OOP vs POP

Procedural (POP)

- Divided into functions
- Global data accessible anywhere
- Top-down design
- Less secure — open data
- Hard to maintain at scale
- Examples: C, Pascal

Object-Oriented (OOP)

- Divided into objects
- Data encapsulated in objects
- Bottom-up design
- Secure — data hidden
- Easy to maintain and extend
- Examples: C++, Java, Python

The 8 Pillars of OOP

- Objects — Real-world entities with state and behavior
- Classes — Blueprints that define objects
- Abstraction — Show essentials, hide complexity
- Encapsulation — Bundle data + methods, control access
- Inheritance — Derive new classes from existing ones
- Polymorphism — Same interface, different behavior
- Dynamic Binding — Function resolved at runtime
- Message Passing — Objects communicate via method calls

Ch.1 Analogy & PU Question

Real-World Analogy



Think of a Car Dealership: The 'Car' class is the blueprint. Each specific car sold is an object. The engine mechanism is hidden (encapsulation). A Tesla inherits from Car but adds electric-specific behavior (inheritance). Pressing the accelerator works on any car regardless of fuel type (polymorphism).



Probable PU Exam Question

Q: What is OOP? Explain any four features with examples. Differentiate OOP from POP. [PU Exam 2022]

Basic Syntax of C++

- C++ Program Structure
- new / delete operators for dynamic memory
- const, Enumeration, Reference Variables
- Scope Resolution Operator (::)
- Manipulators: setw, setprecision, setfill, endl
- Type Conversion: Implicit and Explicit

Dynamic Memory: new & delete

```
#include <iostream>
using namespace std;

int main() {
    int *p = new int;      // Heap allocation
    *p = 42;
    cout << *p;
    delete p;              // Free memory

    int *arr = new int[5]; // Array allocation
    arr[0] = 10;
    delete[] arr;          // Free array
    return 0;
}
```

💡 Always delete what you new. Memory leaks crash programs.

Reference Variables & Scope Resolution

```
int val = 100;    // Global variable

void show() {
    int val = 50;    // Local variable
    cout << val;      // Prints 50 (local)
    cout << ::val;    // Prints 100 (global)
}

// Reference variable usage
int x = 10;
int &ref = x;    // ref is alias for x
ref = 20;        // x is now 20
cout << x;      // Output: 20
```



Reference must be initialized at declaration and cannot be re-seated.

Ch.2 Analogy & PU Question



Real-World Analogy

Reference variables are like nicknames: 'Robert' and 'Bob' refer to the same person. Changing one changes the other. The scope resolution operator (::) is like a city code — you use 'Kathmandu::Ram' to distinguish from 'Pokhara::Ram'.



Probable PU Exam Question

Q: Explain reference variables and scope resolution operator with examples. Write a program using setw, setprecision, and setfill. [PU Exam 2021]

Functions in C++

- Inline Functions — eliminate call overhead
- Function Overloading — same name, different signatures
- Default Arguments — trailing parameters with default values

Functions

A function is a group of statements that together perform a task. Every C++ program has at least one function, `main()`, which is the starting point of the program. Functions can be defined by the user or provided by libraries.

Why Use Functions?

Modularity: Break down complex problems into smaller, manageable pieces.

Reusability: Write code once and use it multiple times, even in different programs.

Organization: Make the code more organized and easier to read and maintain.

Basic Syntax

A function in C++ is defined using the following syntax:

```
// Basic Syntax of Function
returnType functionName(parameterList) {
// body of the function
}
```

- **returnType:** The data type of the value the function returns. If the function does not return a value, this is specified as `void`.
- **functionName:** The name of the function.
- **parameterList:** A list of parameters (arguments) that the function takes, enclosed in parentheses. This is optional; a function may have no parameters.

Functions

Types of Functions

1. Standard Functions

Standard functions are those provided by C++ libraries, such as `std::cout` for output and `std::cin` for input.

2. User-Defined Functions

User-defined functions are written by the programmer to perform specific tasks. The example above illustrates a simple user-defined function.

```
#include <iostream>
using namespace std;

// Function declaration
int add(int, int);

int main() {
    int result = add(5, 3); // Function
    call
    cout << "The sum is " << result <<
    endl;
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

Functions

Function Components

Return Type

A function may return a value. The return type is the type of value the function sends back to the caller. If a function does not return a value, its return type is void.

Parameters

Parameters (or arguments) are the values passed to the function. They are optional; a function can have none. Parameters allow you to pass data into a function.

- Pass-by-value
- Pass-by-reference

Function Body

The function body contains the statements that define what the function does. It's enclosed in curly braces {}.

Functions

Default Arguments

Default arguments in C++ functions are a feature that allows us to specify default values for parameters. These defaults are used when a corresponding argument is omitted from a function call. This feature enhances function flexibility, simplifies function calls, and maintains code clarity by minimizing the need for overloaded functions for different cases.

```
returnType functionName(parameterType1  
parameterName1 = defaultValuel, ...);
```

Understanding Default Arguments

If the function is called without an argument for a parameter with a default value, the default value is automatically used by the function. Parameters with default values must be the rightmost parameters in the function declaration.

```
#include <iostream>
using namespace std;

// defining the default arguments
void display(char c = '*', int count = 3) {
    for(int i = 1; i <= count; ++i) {
        cout << c;
    }
    cout << endl;
}

int main() {
    int count = 5;

    cout << "No argument passed: ";
    display();

    cout << "First argument passed: ";
    display('#');

    cout << "Both argument passed: ";
    display('$', count);

    return 0;
}
```

Functions Overloading

Multiple functions with same name but different parameters.

Function overloading is a feature in C++ that allows multiple functions to have the same name but with different parameters (either in type, number, or both). This capability enhances the expressiveness and readability of the programming language, allowing functions to be used in various contexts with different types of input.

Functions Overloading

Explanation of Program:

```
double area(double radius) {
```

This version of the area function calculates the area of a circle. It takes one double parameter representing the radius of the circle and returns its area as a double.

```
double area(double length, double width) {
```

This version calculates the area of a rectangle. It is overloaded to take two double parameters representing the length and width of the rectangle, respectively, and returns the calculated area.

```
cout << "Area of circle (r=5): " << area(5.0) << endl;
```

Calls the first version of the area function with a single argument to calculate the area of a circle.

```
cout << "Area of rectangle (4x5): " << area(4.0, 5.0) << endl;
```

Calls the second version of the area function with two arguments to calculate the area of a rectangle.

```
#include <iostream>
using namespace std;

// Overloaded functions to calculate the area
double area(double radius) { // For circle
return 3.14159 * radius * radius;
}

double area(double length, double width) {
// For rectangle
return length * width;
}

int main() {
cout << "Area of circle (r=5): " <<
area(5.0) << endl;
cout << "Area of rectangle (4x5): " <<
area(4.0, 5.0) << endl;
return 0;
}
```

Functions Overloading

Use Cases for Function Overloading

Handling Different Data Types: Overloaded functions can perform analogous operations on different types of data. For example, you might have a print function that accepts both int and float types, displaying numbers in an appropriate format.

Different Implementations Based on Parameters: You can overload functions to provide different implementations based on the number or types of arguments passed. This is useful in mathematical operations, where the formula might differ based on input dimensions or types.

Improving Code Readability: Using the same function name for similar operations on different data types makes the code more intuitive and easier to understand.

Advantages

Cleaner Code: Function overloading leads to cleaner code by reducing the number of distinct function names for similar operations, making the code easier to read and maintain.

Intuitive Usage: Overloading allows functions to be called in a way that feels natural and intuitive, closely mimicking the versatility seen in human languages.

Compile-Time Polymorphism: It provides a form of polymorphism where the appropriate function is selected at compile time based on the argument types, improving performance by avoiding runtime decision-making.

Inline Function

Inline functions are a powerful feature in C++ designed to optimize the execution speed of programs, particularly for small, frequently called functions. By marking a function as inline, we give a hint to the compiler that it should attempt to embed the function's body directly into each place the function is called, instead of performing a traditional function call. This section explores what inline functions are, their origin, use cases, and the potential risks associated with their use.

Example of Inline Function

```
inline int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

This simple function returns the maximum of two integers. By declaring it as inline, we suggest to the compiler that replacing calls to max with the function's body could be more efficient than a traditional function call, especially if max is used frequently in performance-critical parts of the code.

Inline Function

```
#include <iostream>
using namespace std;

inline int square(int x) {
    return x * x;
}

int main() {
    cout << square(5) << endl; // expands to 5*5
    cout << square(6 + 1) << endl; // expands to (6+1)*(6+1)
    return 0;
}
```

```
#include <iostream>
using namespace std;

inline int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    cout << max(10, 20) << endl;
    cout << max(100, 50) << endl;
    return 0;
}
```

Inline Functions vs Function Overloading

- Inline: Compiler replaces call with function body
- Use for small, frequently called functions
- Avoids function call overhead (stack frame, jump, return)
- Hint only — compiler may ignore for complex functions
- Overloading: Same function name, different parameter list
- Resolved at COMPILE TIME (early binding)
- Cannot differ only by return type
- Enables intuitive API design

```
// Inline
inline int sq(int x) { return x*x; }

// Overloaded
int add(int a, int b)    { return a+b; }
float add(float a, float b) { return a+b; }
int add(int a,int b,int c){ return a+b+c; }

// Default args
void info(string n, int age=18,
          string city="Kathmandu") {
    cout << n<<"<<age<<"<<city;
}
```

Ch.3 Analogy & PU Question



Real-World Analogy

Function overloading is like a restaurant 'Combo Meal' that means different things at breakfast vs lunch vs dinner. The name is the same but the behavior changes based on context (arguments). Default arguments are like a standing order at your regular cafe — you say 'the usual' and specific defaults apply.



Probable PU Exam Question

Q: What are inline functions? Write a C++ program demonstrating function overloading and default arguments.
[PU Exam 2023]

Lab Work

Scenario 1: Smart Calculator

Problem Statement

Develop a calculator program that can perform addition on different types of inputs.

Requirements

Create overloaded functions named add() for:

- Two integers*
- Two floating-point numbers*
- Three integers*

Create an inline function square() that returns the square of a number.

Display the result of each operation.

Concepts needed to implement

- Function Overloading*
- Inline Function*
- Basic Arithmetic Operations*

Lab Work

Scenario 2: Employee Salary Management

Problem Statement

A company wants a salary calculation system for different categories of employees.

Requirements

Overload a function calculateSalary() for:

- *Full-time employee (basic salary)*
- *Employee with bonus*
- *Employee with bonus and overtime*

Create an inline function taxDeduction() that calculates 10% tax.

Display gross salary and net salary.

Concepts needed to implement

- *Function Overloading with different parameter lists*
- *Inline Functions*
- *Real-world business application*

Lab Work

Scenario 3: Student Result Processing

Problem Statement

Create a result-processing system for students.

Requirements

Overload a function calculateAverage() for:

Two subjects

Three subjects

Five subjects

Create an inline function isPass() that returns true if average ≥ 40 .

Display average marks and pass/fail status.

Concepts needed to implement

- *Function Overloading with different parameter lists*
- *Inline Functions*
- *Real-world business application*

Classes and Objects

- Visibility Modes: public, private, protected
- Data Members and Member Functions
- Static Members — shared across all objects
- Passing and Returning Objects
- The 'this' Pointer

Introduction

- Objects and Classes are key to understand the Object-Oriented Programming Language (OOPL)/Object-Oriented Technique. Object-Oriented Programming Languages are based on the concept of abstraction modeled by classes and objects.
- Classes and objects are the cornerstones of Object-Oriented Programming (OOP) in C++. They enable programmers to create modular, reusable, and maintainable code. In this article, we will dive deep into understanding what classes and objects are, explore their functionalities, and learn how to implement them in C++ effectively.
- Classification is one of the important concepts of Object-Oriented philosophy including identity, abstraction, encapsulation, inheritance, polymorphism, and persistence.

Object-Oriented uses classification to group objects that have attributes and behaviors in common and classify them into bigger entities. The bigger entity is called a “class”.

Thus, a class defines a group of objects with similar attributes, common operations, a common relationship to the other objects in the class, and common semantics.

Class

A class is set of objects that shares a common definition described by data and methods.

According to the Webster's New World dictionary, a class is defined as:

“A number of people or things grouped together because of certain likeness; kind; sort”. In other words, we can say that class is an object template. Every object under that class has the same data format, definition and responds in the same manner to an operation.

A **class** in C++ is a user-defined data type that encapsulates data members (variables) and member functions (methods) within a single unit. Think of a class as a blueprint or template that defines the structure and behavior of objects. Classes are the foundation of OOP and facilitate encapsulation, abstraction, and data hiding.

Class

Analogy of Classes:

Imagine a class of cars. Cars can have different names, models, and brands, yet they all share common properties like having four wheels, a speed limit, and mileage. In this analogy:

- ✓ The **Car** represents the class.
- ✓ The common properties such as wheels, speed limits, and mileage are the **data members**.
- ✓ Actions like accelerating, braking, and honking are the **member functions**.

Defining a Class in C++

A class in C++ is defined using the class keyword followed by the class name. The body of the class contains the data members and member functions, enclosed in curly braces {}.

Class

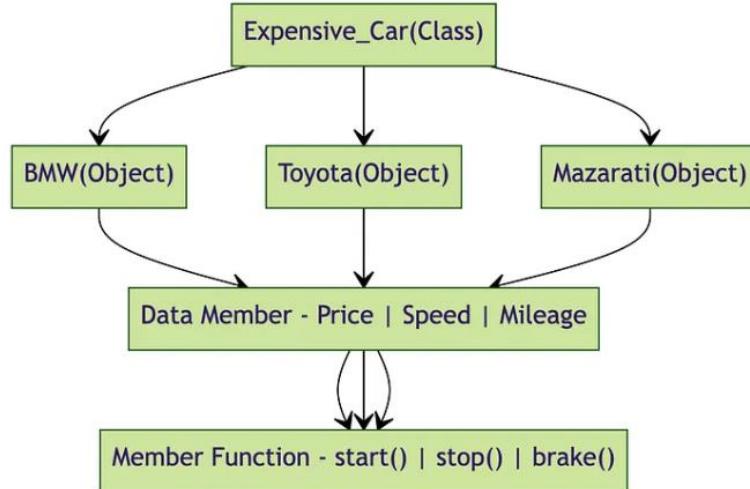
Example

Class: Expensive Cars

Objects: Mercedes, BMW, Toyota

Data Member: Price, Speed, Wheels, Mileage, Model, Color

Member Function: brakes(), speed(), start(), stop(), etc.



Class

Syntax:

```
class ClassName {  
    access_specifier:  
    // Data_members;  
    // Member_Functions()  
};
```

- The **keyword class** is used to define a class in C++, where the class identifier is given by **ClassName**.
- The **access_specifier** is the access controller which defines the visibility of class members. It can be private, public, and protected; we will elaborate on these in a later section.
- **Data_members** are the variables that hold data related to an object of the class.
- **Member_functions()** are class methods that define what operations can be performed on objects and how.

Understanding Objects in C++

*An **object** is anything in the real world that has a distinct identity, observable characteristics, and defined behaviors.*

Think of it like a **smartphone**:

State (Attributes): Brand, model, color, battery level.

Behavior (Methods): Making a call, sending a text, or opening an app.

According to the Webster's New World dictionary, an Object is defined as:

“A thing that can be seen or touched; material thing; a person or thing to which action, thought or feeling is directed”.

An **object** is an instance of a class. When an object is created, a real-world entity is created that has the properties and behaviors defined by the class. While a class only defines what an object will be, an object is an actual instance that occupies memory and can be manipulated.

Understanding Objects in C++

Creating an Object:

To use the data members and functions of a class, you need to create an object. When you create an object, memory is allocated for it, allowing you to access and manipulate the data members and functions defined in the class.

Syntax:

```
ClassName ObjectName; //Creating an object
```

- **Class_name:** It is the name of the class for which the object is created
- **Object_name:** It is the name of the object. Note that having a meaningful object name enhances code readability

Example:

```
Car myCar; // Creating an object of the Car class
```

Creating Class & Object in C++

- 1. Declare and Define a Class in C++ Program:** The first step is to create a class, as mentioned in the syntax given above. The C++ class definition must specify-
 - ✓ Data members, i.e., the attributes or member variables of objects in C++.
 - ✓ Member functions, i.e., C++ class methods that define how we can manipulate the data stored in the member variables to full required tasks.
- 2. Create an Object in C++ Program:** After a class has been defined, an object of that class can be declared by writing the class name and then the object name. The memory for the object is assigned after this declaration.
- 3.** The creation of a C++ class object is usually done in the main() part of the program.
- 4.** Once declared and initialized, the C++ class object can be used to invoke special functions in the class.

Class, Object and Member

```
#include <iostream>
using namespace std;
// Define a class called 'Car'

class Car {
    public:        // Access specifier
        string brand; // Data member
        int speed;    // Data member

        void display() {        // Member function
            cout << "Brand: " << brand << ", Speed: " << speed << " km/h" << endl;
        }
};

int main() {
    // Creating an object of the Car class
    Car myCar;

    // Setting values for the object's data members
    myCar.brand = "Toyota"; // Assign brand name
    myCar.speed = 120;       // Assign speed value

    // Displaying the car details
    myCar.display(); // Call the display function to output car information

    return 0; // Indicate successful program completion
}
```


Data Member in C++

A **data member** is a variable declared within a class or struct that holds data (state) for objects of that class type. It represents the attributes or properties of the class.

Formal Definition

A data member (also called a field, attribute, or instance variable) is a non-function member of a class that stores data. It can be of any type, including fundamental types, pointers, references, arrays, or other class types. Each data member has storage associated with it for each object instance (except static members, which are shared).

Data Member in C++

```
#include <iostream>
class ClassName {
    // Data member declaration syntax:
    [access_specifier:] [modifiers] type name [= initializer];

private:
    int age;                // simple data member
    std::string name;       // object data member
    static int count;       // static data member
    const double PI = 3.14159; // const data member
    int* ptr;               // pointer data member
    int& ref;               // reference data member (must be initialized)
    mutable int cache;      // mutable data member
    int arr[10];            // array data member
    std::vector<int> scores; // container data member
};
```

Data Member in C++

Types of Data Members

Type	Declaration	Storage	Example
Instance	type name;	Per object	int x;
Static	static type name;	Shared across all objects	static int count;
Const	const type name;	Per object, cannot modify	const int id;
Reference	type& name;	Per object, must bind	int& ref;
Mutable	mutable type name;	Can modify in const methods	mutable int cache;
Bit-field	type name : bits;	Space-efficient	int flag : 1;

Member Function in C++

A function that is defined as a class member is known as a member function in C++ programming.

What Is Member Function In C++?

- **Member functions** are functions defined within a class.
- They can access the values of the data members and can also carry out operations on the data members.
- Their definition may be included in or excluded from the class body. A member function in C++ can access both public and private class members.

Member Function in C++

Syntax for Member Function

```
// Member function declaration
returnType functionName (arguments)
{ //function body
};
```

- The **functionName** refers to the name of the function, with the **returnType** specifying the data type of its return value.
- The **arguments** refer to the parameters the member function can take.
- The **curly brackets {}** contain the function body, i.e., the operations that are performed on the data members of the class.

Member Function in C++

Additional points to note about a member function in C++ is that it can access the private data of members, whereas a non-member function is unable to do so.

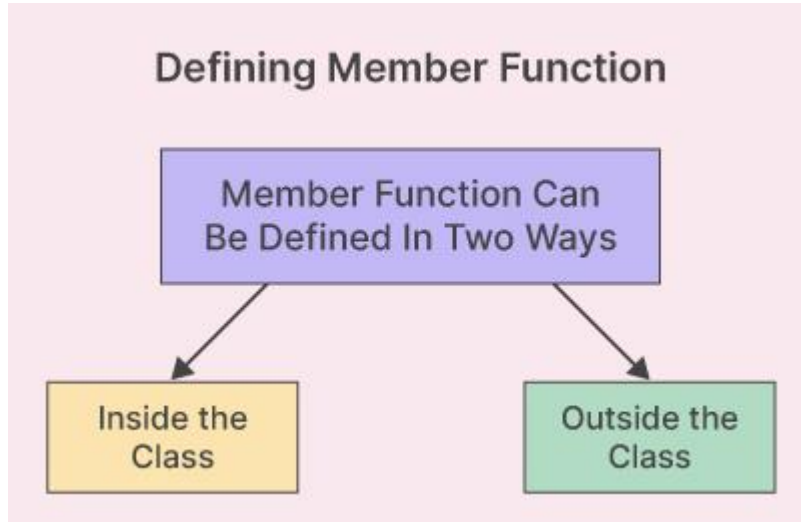
- Also, they can be called by another member function directly without using a dot operator.
- Member function makes the program modular by increasing code reusability and making it easier to understand, hence making the code maintainable.
- Member functions also provide the ability to restrict access to the data of members of the class. This can be done by making specific functions private, thus ensuring that only the class itself may access or modify sensitive data.
- Member functions are essential in C++ as they encapsulate the behavior and functionality of objects. They enable objects to interact with their data and serve as a means for other components of the program to modify the data.

Member Function in C++

Defining a Function in C++

There are mainly two ways to define a member function in C++. They are-

- Inside the class definition
- Outside the class definition.



Member Function in C++

Defining a Function in C++ Inside The Class Definition

A member function can be both declared and defined within a class itself. When a member function is defined within a class, it is referred to as an inline function. All of the constraints and limitations that apply to inline functions also apply to functions declared within a class.

Syntax:

```
class MyClass{  
    private:  
        //Data Members variables  
  
    public:  
        //Member Function  
        returnType functionName(argument){  
            //Function Definition  
        }  
};
```


Member Function in C++ Inside The Class Definition

```
#include <iostream>
using namespace std;

class Circle{
private:
    //Data Members
    float rad;

public:
    //Member Function
    void getradius(){
        cout<<"Enter Radius"<<endl;
        cin>>rad;}

    void calc_area(){
        float a;
        a = 3.14*rad*rad;
        cout<<"The Area of the Circle is = "<<a<<endl;
    };
};

int main(){
    Circle c;
    c.getradius();
    c.calc_area();
    return 0;
}
```

- *Circle class* is defined , which has a *private data member* called *rad* (of float type).
- In the class is the member function called *getradius()*, which prompts the user to enter the radius of the circle *using cout*, stores it in the rad data member, and reads the input using the *cin command*.
- Also, another void member function called *calc_area* within the Circle class. It calculates the area of the circle using the formula $a = 3.14 * rad * rad$ and displays it on the console.
- In the *main() function*, we first create an *object c* of the Circle class.
- We then *call the getradius()* member function on the c object, which prompts the user for input and stores the radius in the rad data member. The *calc_area()* member function then calculates the area of the circle and displays it on the console.

Member Function in C++

Defining a Function in C++ Outside The Class Definition

In this case, the declaration and definition of the member function are not made together. That is, while the functions are declared inside the class, their definition is given outside.

However, the definitions of member functions are very similar to those of normal functions. They should have a function header and body. Unlike a normal function, a member function includes a membership identity label in the header. The identity label is used to indicate that the function belongs to a particular class.

Member Function in C++

Syntax:

```
return_type class_name :: function-name (argument declaration){  
    //Function body  
}
```

Here,

- The **class_name** is the membership label or the name of the class to which the member function belongs.
- The **function_name** and the double colon (**::**) are the name of the function and the scope resolution operator, which tells the compiler that the function belongs to the **class_name**.
- The **arguments** and the **return_type** refer to the parameters the function takes and the data type of its return value, respectively.

Member Function in C++ Inside The Class Definition

```
#include <iostream>
using namespace std;

class Circle{
private:
float rad; //Data Members

public:
void getradius(); //Member Function
void calc_area(); //Member Function
};

void Circle :: getradius(){
cout<<"Enter Radius ";
cin>>rad;}

void Circle :: calc_area(){
float a;
a = 3.14*rad*rad;
cout<<"The Area of the Circle is = "<<a<<endl;}

int main(){
    Circle c;
    c.getradius();
    c.calc_area();
    return 0;
}
```

- *Circle class* is defined , which has a *private data member* called *rad* (of float type).
- It contains the member functions called *getradius()*, *calc_area()* which are defined outside the class using scope resolution operators (::)
- In the *main() function*, we first create an *object c* of the Circle class.
- We then *call the getradius()* member function on the c object, which prompts the user for input and stores the radius in the rad data member. The *calc_area()* member function then calculates the area of the circle and displays it on the console.

Static Data Member

Introduction

- Declared using the keyword `static` inside the class.
- Must be defined outside the class .
- Shared by all the objects of the class.
- Accessed using the class name.

A static data member in C++ refers to a member of a class that is common to all its objects. In other words, it is associated with the class as a whole. These members are created using the static keyword.

What Is Static Data Member In C++?

The static data member in C++ programming is a special type of class member that is associated with the class rather than with an individual object of a class. It means that all instances of the class share the same static data member. In other words, unlike regular (non-static members) data members, which have separate copies for each object, static data members are shared among all objects of the class.

Static Data Member

Syntax to Declare a Static Data Member In C++:

```
class ClassName {  
    access_specifier:  
    static data_type data_member_name;  
};
```

- The **ClassName** and **data_member_name** refer to the names of the class and the data member we are creating.
- An **access specifier** indicates the visibility of the class member, which can be private, public or protected.
- The **static** keyword indicates that the respective member is of a static nature and the type of data it stores is given by **data_type**.

```
Declaration:      Static datatype variableName:  
Definition :      datatype className:: variableName;  
Usage:           ClassName:: variableName;
```

By default, static data members are initialized by 0.

Static Data Member

Characteristic of Static Data Member

- ✓ **Declaration of Static Data Member in C++:** A static data member in C++ is declared using the static keyword within the class definition.
- ✓ **Definition of Static Data Member in C++:** The static data member in C++ must be defined outside the class (usually in the source file) to allocate memory for it. This is typically done by specifying the data type and the name of the static data member, along with its initial value (if any).
- ✓ **Initialization of Static Data Member in C++:** We can initialize a static data member in C++ programs using a constant expression or a compile-time constant. The initialization is done outside the class definition.
- ✓ **Accessing a Static Data Member in C++:** We must use the class name followed by the scope resolution operator(::) and the name of the respective static data member in C++ when we want to access it.

Class Work

Context: Student count using Static Data Member

Problem Statement

- 1. Needed a Program that will output the total number of students(object) recorded*
- 2. Also demonstrate the difference with normal Data Member*

Static Member Function

What Is Static Member function in C++?

Static member functions in C++ are class-level functions that are associated with the class itself rather than with individual instances (objects) of the class. They are declared using the static keyword, and we can call such a function with a class name without the need to create an object. Static member functions are commonly used for operations that are related to the class as a whole rather than to specific instances.

Static Member Function

Syntax to Declare a Static Data Member In C++:

```
class ClassName {  
    public:  
        static returnType functionName(parameters) {  
            // Function body implementation  
        }  
};
```

- The **static** keyword indicates that the function of the name- **functionName** is static in nature.
- The **parameters** and **returnType** refer to the input parameters (if any) taken by the static member function and the data type of its return value, respectively.
- As marked by the code comment, the **curly brackets** contain the body of the function.

Static Member Function

```
#include <iostream>
using namespace std;

class MathUtils {
public:
    // Static member function: belongs to the class, not instances
    // Can be called without creating an object
    static int add(int a, int b) {
        return a + b; // Simple addition operation
    }
};

int main() {
    // Call static member function using class name and scope resolution operator (::)
    int result = MathUtils::add(5, 3);

    // Output the result to console
    cout << "Result: " << result << std::endl;

    return 0; // Successful program termination
}
```

```
#include <iostream>
using namespace std;

class MyClass {
public:
    static int staticValue; // Static data member declaration

    static int getStaticValue() {
        return staticValue; // Static member function to access
        the static data member
    }
};

// Initialization of the static data member outside the
class
int MyClass::staticValue = 101;

int main() {
    // Accessing the static data member using the static
    member function
    int value = MyClass::getStaticValue();
    std::cout << "Value of Static Data Member: " << value <<
    std::endl;

    return 0;
}
```

We define a class called ***MyClass*** and declare a static data member ***staticValue*** of type int. This static member is shared across all instances of the class.

We also declare a static member function ***getStaticValue()*** within the class.

This function returns the value of the static data member ***staticValue***.

Outside the class, we initialize the static data member ***staticValue* to 101.**

This initialization is necessary because static members must be defined outside the class to allocate storage.

In the ***main()* function**, we call the static member function ***getStaticValue()*** using the class name ***MyClass*** and the ***scope resolution operator (::)***.

This function call retrieves the value of ***staticValue***, which is ***101***. We finally store the returned value in an integer variable ***value*** and print it to the console.

Static Member Function – Usages

Static member functions in C++ serve several important purposes and offer distinct advantages in various programming scenarios. Here are some key reasons why static member functions are needed and valuable in C++:

Class-Level Operations: Static member functions allow you to define operations that are relevant to the class as a whole rather than to individual instances (objects) of the class. This is especially useful when you want to perform actions that do not depend on the specific state or data of objects.

No Object Instantiation Means Efficient Code: You can call a static member function in C++ directly on the class without the need to create instances of the class. This means you can use these functions without incurring the overhead of object creation, which can lead to more efficient code execution.

Utility Functions: Static member functions are often used to encapsulate utility functions that provide common functionality related to the class. These utility functions can perform tasks such as mathematical calculations, data validation, and formatting, making your code more organized and reusable.

Static Member Function – Usages

Factory Methods: Static member functions in C++ are commonly employed as factory methods that create instances of a class with specific characteristics. Factory methods allow you to control the object creation process and provide a convenient way to initialize objects.

Singleton Pattern: Static member functions are crucial for implementing the singleton design pattern, which ensures that only one instance of a class exists throughout the application's lifetime. A static member function in C++ can create and manage a single instance, providing global access to that instance.

Global Access: Static member functions can be called from anywhere in your code by specifying the class name followed by the scope resolution operator (::). This makes them accessible globally, allowing you to perform operations that span multiple objects or modules.

Code Organization: By grouping related functions within a class and using a static member function in C++, you can improve code organization and maintainability. It becomes clear which functions are associated with a particular class, enhancing the structure of your codebase.

Property	Regular Member Function	Static Member Function
Association	Associated with instances of the class.	Associated with the class itself.
Access	Using object instances	Using the class name and scope resolution (::) operator
Can access static data members	Yes (via object or class name)	Yes (via class name and scope resolution operator)
Can access non-static data members	Yes (via object)	No (unless passed as a parameter)
Can access regular member functions	Yes (via object)	No
Can access static member functions	Yes (via object or class name)	Yes (via class name and scope resolution operator)
Access to 'this' pointer	Yes	No
Lifetime	Tied to the object's lifetime.	Tied to the class's lifetime.
Initialization	Per instance	Typically, at class declaration
Virtual Function	Can be declared virtual	Cannot be declared virtual
Polymorphism	Participates in polymorphism	Does not participate in polymorphism

Accessing Class Members

Members of the class can be accessed inside the class itself simply by using their assigned name and access them outside, the (.) dot operator is used with the object of the class.

Syntax:

```
object.dataMember;  
object.memberFunction();
```

```
#include <iostream>  
using namespace std;  
  
class Dog {  
    public:  
        string name;  
        string breed;  
  
        void display(){  
            cout << "Dog Name: " << name << endl;  
            cout << "Breed: " << breed << endl;  
        }  
};
```

```
int main() {  
    Dog tuffy;  
    tuffy.name = "Tuffy";  
    tuffy.breed = "Papillon";  
    tuffy.display();  
    return 0;  
}
```

Nesting of Members Function

We know that only the public members of a class can be accessed by the object of that class, using dot operator. However, a member function can call another member function of the same class directly without using the dot operator. This is called as *nesting of member functions*.

Characteristics of Nesting Members Function:

- Nesting of member functions occurs when one member function calls another within the same class.
- A member function can be called using the dot (.) operator with an object or directly by other member functions of the same class.
- All member functions can access and call other member functions within the class.
- Static member functions can be called by other member functions but can only directly call other static member functions.
- Member functions can be recursive, meaning they can call themselves.
- Member functions defined outside the class can also exhibit nesting, adhering to accessibility and scope rules.

Nesting of Members Function

```
#include <iostream>
using namespace std;

class c1
{
    int s = 2;
public:
    void m1() {
        cout << "m1 ";
    }
    void m2()
    {
        // Non-static member function calling another non-static member function
        m1();
        cout << "m2 ";
    }
};

int main()
{
    c1 obj;
    obj.m2();
}
```

Nesting of Members Function

```
#include <iostream>
using namespace std;
class set{
    int m,n;

    public:
        void input();
        void display ();
        int largest();
};

int set::largest(){
    if(m>n)
        return m;
    else
        return n;
}

void set::input(){
    cout<<"input values of m:";
    cin>>m;
    cout<<"input values of n:";
    cin>>n;
}

void set::display(){
    cout<<"largestvalue="<<largest()<<"\n";
}

int main(){
    set A;
    A.input();
    A.display();
    return 0;
}
```

Nesting of Members Function

```
#include <iostream>
using namespace std;

class average{
    int a,b;
public:
    void read();
    void print();
    int avg();
};

void average::read(){
    cout<<"\n enter a and b: ";
    cin>>a>>b;
}

void average::print(){
    cout<<"value of a: "<<a;
    cout<<"\nvalue of b: "<<b;
    cout<<"\naverage is : "<<avg();
}

int average::avg(){
    return (a+b)/2;
}

int main(){
    average A;
    A.read();
    A.print();
    return 0;
}
```

this pointer

The 'this' pointer in C++ is an implicit pointer available within non-static member functions of a class or structure. It points to the current object instance, letting the object access its own member variables and functions.

Syntax of 'this' Pointer

When creating or using the 'this' pointer, the 'this' keyword is used in conjunction with the arrow operator (->) along with the name of the member or method being referred to.

```
void functionName() {  
    this->memberName = value;  
}
```

- ✓ **functionName** refers to the identifier/ name you have given to the function.
- ✓ The term **this->** represents the 'this' pointer in C++, and **memberName** refers to the respective member of the class you are trying to access.

this pointer

Syntax of 'this' Pointer

```
class MyClass {  
    public:  
        void myMethod() {  
            this->myMember = 42;  
        }  
    private:  
        int myMember;  
};
```

In the above code snippet, we are trying to access the myMember variable/ attribute of the current object through the expression **this->myMember**, which is being assigned a value of 42 within the **myMethod()** function definitio

this pointer

Defining 'this' Pointer In C++

In C++ programming, each object of a class has its own set of member variables/ attributes (also known as data members) and member functions.

- When a member function is called on an object, the 'this' pointer is automatically passed as a hidden argument to the member function.
- It enables the member function to have access to the object's data members (i.e., the class member variable) and call other member functions.

```
#include <iostream>
using namespace std;

class Person {
private:
    string name;
    int age;
public:
    Person(const std::string& name, int age) {
        this->name = name;
        this->age = age;
    }
    void displayInfo() {
        std::cout << "Name: " << this->name << std::endl;
        std::cout << "Age: " << this->age << std::endl;
    }
    void updateAge(int newAge) {
        this->age = newAge;
    }
};

int main() {

    Person person("Kapil", 19);
    person.displayInfo();
    person.updateAge(20);
    cout << "After updating age:" << endl;
    person.displayInfo();
    return 0;
}
```

Analogy & PU Question



Real-World Analogy

The 'this' pointer is like saying 'myself'. When an employee says 'I will handle this myself', 'myself' refers to that specific person — just as 'this' refers to the specific object calling the method. Static members are like a shared office kitchen — all employees (objects) share one, not one per person.



Probable PU Exam Question

Q: Explain static data members and static member functions in C++. Write a program to count objects using a static variable. Explain the 'this' pointer. [PU Exam 2022]

Access Specifier / Modifier

Introduction

- Keywords that define level of accessibility of members of a class.
- Controls who can access what in a program

When designing a C++ class, we often want to control who all can access members (both the data members and member functions) of a class.

Access specifiers are a way to achieve this. Creators of a class can use access specifier to implement data hiding. Data Hiding helps in separating implementation from interface.

Type of Access Specifiers:

- Public
- Private
- Protected



Access Specifier : Public

Public Access Specifier

Members are accessible from anywhere in the program, both inside and outside the class.

Purpose: Defines the class's public interface, allowing unrestricted access to members. This is how users (other classes, functions, or main) interact with the class.

Behavior in Inheritance: Public members remain public in derived classes under public inheritance, making them accessible to both derived classes and external code.

Use Case: Public methods for operations like initialization, data retrieval, or actions (e.g., start(), stop()).

Access Specifier : Public

```
#include <iostream>
using namespace std;
// Define a class called 'Car'

class Student {
    public:        // Access specifier
        string name; // Data member
        int rollNo;  // Data member

        void display() { // Member function
            cout << "Name: " << name << endl;
            cout << "RollNo: " << rollNo << endl;
        }
};

int main() {
    // Creating an object of the Car class
    Student s1;

    s1.name = "Ankit";
    s1.rollNo = 2;
    s1.display();
    return 0; // Indicate successful program completion
}
```

Access Specifier : Private

Private Access Specifier

Accessible only within the class. Not accessible to derived classes or external code.

- **Purpose:** Ensures strict encapsulation by limiting access to the class's internal implementation. This protects data integrity and prevents misuse by external code or derived classes.
- **Behavior in Inheritance:** Private members are **not inherited** by derived classes, meaning they cannot be accessed directly or indirectly in subclasses.
- **Use Case:** Use private for sensitive data (e.g., internal counters, buffers) or helper methods that shouldn't be exposed.

Access Specifier : Private

```
#include <iostream>
using namespace std;
class BankAccount {
    private:
        double balance; // Private data member
    public:
        BankAccount(double initialBalance) {
            balance = initialBalance;
        }

        double getBalance() { // Public function to access private data
            return balance;
        }
};

int main() {
    BankAccount account(1000.0);
    // account.balance = 500; // Error: Can't access private member directly
    cout << "Balance: $" << account.getBalance() << endl;
    return 0;
}
```

Access Specifier : Protected

Protected Access Specifier

Accessible only within the class and in derived classes but not externally.

- **Purpose:** Allows derived classes to access base class members while still hiding them from external code. This is useful for inheritance hierarchies where subclasses need access to base class internals.
- **Behavior in Inheritance:** Protected members are inherited by derived classes and remain accessible unless the inheritance type restricts them further (e.g., private inheritance).
- **Use Case:** Base class members that derived classes need to manipulate (e.g., shared resources or state).

Access Specifier : Protected

```
#include <iostream>
using namespace std;
class Animal {

protected:
    string species; // Protected member
public:
    void setSpecies(string s) { species = s; }
};
class Dog : public Animal {
public:
void showSpecies() {
cout << "Species: " << species << endl; // Accessible in derived class
}
};
int main() {
    Dog myDog;
    myDog.setSpecies("XYZ");
    myDog.showSpecies();
    return 0;
}
```

Access Specifier

Common Pitfalls and Best Practices

1. Overusing Public Members:

Making too many members public can expose implementation details, making the class harder to maintain or refactor.

Solution: Use private/protected members with public getters/setters for controlled access.

2. Misunderstanding Protected:

Developers sometimes assume protected members are accessible outside the class hierarchy, but they're not.

Solution: Clearly document which members are intended for derived classes.

3. Inheritance Misuse:

Using private inheritance when composition would be better can lead to complex code.

Solution: Prefer composition (has-a relationship) over inheritance (is-a relationship) unless inheritance is clearly justified.

4. Performance Considerations:

Access specifiers don't directly impact performance, but excessive use of getters/setters for private members can add overhead in performance-critical code.

Solution: For performance-sensitive code, consider inline getters or direct access (if safe).

Array of Object

An array of objects is a collection of multiple objects of the same class type stored contiguously in memory. It allows you to manage multiple instances of a class efficiently using array indexing and iteration.

Arrays can store primitive data types like int, float, double, char, and even derived data types such as structures and pointers.

When a class is defined, it specifies the structure of objects but does not allocate memory. To use the data and functions defined in the class, you need to create objects. An array of a class type is known as an **array of objects**.

Basic Syntax

```
ClassName arrayName[arraySize];
```

Array of Object

Key Points for Beginners:

- **Declaration:** `ClassName arrayName[size];`
- **Access:** Use `arrayName[index].member` to access object members
- **Indexing:** Starts from 0, ends at size-1
- **Loop:** Use for loop to process all objects
- **Constructor:** Can initialize objects while creating the array

Most Common Simple Use Cases:

- **Student database:** Store student records
- **Product inventory:** Store product details
- **Employee list:** Store employee information
- **Game objects:** Store game characters or items
- **Book library:** Store book information

Array of Object

```
#include <iostream>
using namespace std;

class student
{
    char name[20];
    int age;
public:
    void getdata() {
        cout<<"enter the name of the student :"<<endl;
        cin>>name;
        cout<<"enter the age :"<<endl;
        cin>>age;
    }
    void putdata(){
        cout<<"the name of the student is :"<<name<<endl;
        cout<<"the age of the student is :"<<age<<endl;
    }
};
```

```
int main(){
    student s[10];
    int n;
    cout<<"Enter the number of
students :"<<endl;
    cin>>n;
    for (int i=0;i<n;i++) {
        s[i].getdata();
    }
    for (int i=0;i<n;i++) {
        s[i].putdata();
    }
    return 0;
}
```

Friend Function — Key Concepts

- NOT a member of the class but has access to private/protected data
- Declared inside class using 'friend' keyword
- Called without dot notation — object is passed as parameter
- Has NO 'this' pointer
- Does NOT inherit friendship
- Use cases:
- Operator overloading (especially <<, >>)
- Functions that need access to two different class private members
- Bridge between two unrelated classes

```
class Distance {  
    float km;  
    public:  
        Distance(float k): km(k) {}  
        // Friend declaration  
        friend Distance add(  
            Distance d1, Distance d2);  
        void show() { cout<<km; }  
};  
  
// NOT a member — no Distance::  
Distance add(Distance d1, Distance d2) {  
    return Distance(d1.km + d2.km);  
}
```


Friend Function

A **friend function** is a non-member function that has access to the private and protected members of a class. It is declared inside the class using the friend keyword but defined outside the class scope.

Key Characteristics

- **Not a member** of the class
- **Can access private/protected** members directly
- **Declared with** friend keyword inside the class
- **Defined outside** the class like a normal function
- **Can be a function** or a member function of another class

Friend Function

Basic Syntax

```
class ClassName {  
private:  
    int privateData;  
  
public:  
    // Declaration of friend function  
    friend returnType functionName(parameters);  
};  
  
// Definition of friend function (outside the class)  
returnType functionName(parameters) {  
    // Can access private members of ClassName  
}
```

Friend Function

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int width;
    int height;
public:
    Rectangle(int w, int h) : width(w), height(h) {}

    friend void displayArea(Rectangle r); // Declare friend function
};

// Friend function definition - can access private members
void displayArea(Rectangle r) {
    // Accessing private members directly
    int area = r.width * r.height;
    cout << "Area: " << area << endl;
}

int main() {
    Rectangle rect(5, 10);
    displayArea(rect); // Called like normal function
    return 0;
}
```

Analogy & PU Question



Real-World Analogy

A friend function is like a trusted accountant who is NOT an employee of your company but is allowed to see private financial records. They work outside the company but have special access granted by the company (class) itself.



Probable PU Exam Question

Q: What is a friend function? Differentiate it from a member function. Write a C++ program to add two distance objects using a friend function. [PU Exam 2021]

Friend Class

A **friend class** is a class that is granted access to the private and protected members of another class. When Class A declares Class B as a friend, all member functions of Class B can access the private and protected members of Class A.

Key Characteristics

- **Bidirectional?** No, friendship is **not mutual** (unless explicitly declared both ways)
- **Not inherited:** Friendship is not passed to derived classes
- **Not transitive:** If A is friend of B, and B is friend of C, A is NOT friend of C
- **Can be declared** in public, private, or protected sections (convention: public)
- **Forward declaration** may be needed

Friend Class

Basic Syntax

```
// Forward declaration (if needed)
class FriendClass;

class MyClass {
private:
    int privateData;

public:
    // Declare FriendClass as friend
    friend class FriendClass;
};

class FriendClass {
public:
    void accessMyClass(MyClass& obj) {
        // Can access privateData directly
        obj.privateData = 100;
    }
};
```

Friend Class

Friend classes are useful when:

- Two classes are tightly coupled (e.g., Engine and Car)
- Implementing design patterns (Factory, Builder, Iterator)
- Creating test classes
- Implementing data access layers
- Performance optimization is critical

Comparison: Friend Class vs Friend Function

Aspect	Friend Class	Friend Function
Scope	All member functions are friends	Only specific function
Granularity	Coarse (entire class)	Fine (single function)
Use case	Related classes working together	Helper/utility functions
Coupling	Creates tighter coupling	Less coupling
Complexity	Higher	Lower

Passing and Returning Objects in C++

In C++, objects can be passed to functions and returned from functions just like primitive data types. However, understanding the different ways to pass and return objects is crucial for writing efficient and bug-free code.

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int width;
    int height;

public:
    Rectangle(int w = 0, int h = 0) : width(w), height(h) {
        cout << "Constructor called" << endl;
    }

    Rectangle(const Rectangle& other) {
        width = other.width;
        height = other.height;
        cout << "Copy constructor called!" << endl;
    }

    int area() const {
        return width * height;
    }

    void setWidth(int w) {
        width = w;
    }
};
```

Passing Objects to Functions

1. Pass by Value (Copy): The object is copied when passed to the function. The function works with a copy, so the original object is not modified.

```
int main() {
    Rectangle rect(5, 10);
    cout << "Original area: " << rect.area() << endl;

    modifyRectangleByValue(rect); // Copy is created

    cout << "Original area after function: " << rect.area() << endl;
    // Original remains unchanged

    return 0;
}
```

Passing and Returning Objects in C++

Passing Objects to Functions

2. Pass by Reference:

The function receives a reference to the original object. No copy is created, and changes affect the original.

```
#include <iostream>
using namespace std;

class Counter {
private:
    int count;

public:
    Counter(int c = 0) : count(c) {}

    void increment() {
        count++;
    }

    int getCount() const {
        return count;
    }
};

// Pass by reference - no copy, modifies original
void incrementByReference(Counter& c) {
    c.increment();
    cout << "Inside function - count: " << c.getCount() << endl;
}
```

```
int main() {
    Counter counter(5);
    cout << "Original count: " << counter.getCount() << endl;

    incrementByReference(counter); // No copy made

    cout << "Original count after function: " << counter.getCount() << endl;
    // Original is modified

    return 0;
}
```

Passing and Returning Objects in C++

Passing Objects to Functions

3. Pass by Const Reference:

The function receives a read-only reference. No copy is created, but the object cannot be modified

```
#include <iostream>
using namespace std;

class Student {
private:
    string name;
    int rollNo;

public:
    Student(string n, int r) : name(n), rollNo(r) {}

    void display() const {
        cout << "Name: " << name << ", Roll: " << rollNo << endl;
    }

    string getName() const { return name; }
    int getRollNo() const { return rollNo; }
};

// Pass by const reference - read-only, efficient
void printStudent(const Student& s) {
    // s.setName("John"); // ERROR! Cannot modify const reference
    s.display(); // OK - const method
    cout << "Name from const ref: " << s.getName() << endl;
}

int main() {
    Student student("Alice", 101);
    printStudent(student); // No copy, but read-only

    return 0;
}
```

Passing and Returning Objects in C++

Returning Objects to Functions

1. Return by Value

The function returns a copy of the object

```
#include <iostream>
using namespace std;

class Point {
private:
    int x, y;

public:
    Point(int a = 0, int b = 0) : x(a), y(b) {
        cout << "Constructor called" << endl;
    }

    Point(const Point& other) {
        x = other.x;
        y = other.y;
        cout << "Copy constructor called" << endl;
    }

    void display() const {
        cout << "(" << x << ", " << y << ")" << endl;
    }

    void setX(int val) { x = val; }
};
```

```
// Return by value - returns a copy
Point createPoint(int a, int b) {
    Point p(a, b); // Constructor called
    return p;      // Copy constructor called (or move in C++11)
}

int main() {
    cout << "Creating point using function:" << endl;
    Point p1 = createPoint(5, 10); // Copy constructor called (may be optimized)

    cout << "p1: ";
    p1.display();

    // RVO (Return Value Optimization) may eliminate copy
    cout << "\nWith RVO (may not show copy):" << endl;
    Point p2 = createPoint(20, 30);
    p2.display();

    return 0;
}
```

Passing and Returning Objects in C++

Returning Objects to Functions

2. Return by Reference

Returns a reference to an existing object. Must ensure the object outlives the function.

```
#include <iostream>
#include <vector>
using namespace std;

class Team {
private:
    string name;
    vector<string> players;

public:
    Team(string n) : name(n) {}

    void addPlayer(string player) {
        players.push_back(player);
    }

    // Return reference to internal data
    vector<string>& getPlayers() {
        return players; // Return by reference
    }

    void display() const {
        cout << "Team: " << name << endl;
        for(const auto& p : players) {
            cout << "    - " << p << endl;
        }
    }
};
```

```
// Return reference to an element
string& getFirstPlayer(Team& team) {
    return team.getPlayers()[0]; // Returns reference to player
}

int main() {
    Team team("Warriors");
    team.addPlayer("Curry");
    team.addPlayer("Thompson");
    team.addPlayer("Green");

    cout << "Original team:" << endl;
    team.display();

    // Get reference to first player and modify
    string& firstPlayer = getFirstPlayer(team);
    firstPlayer = "Durant"; // Modifies the original string

    cout << "\nAfter modification:" << endl;
    team.display();

    return 0;
}
```

Passing and Returning Objects in C++

Returning Objects to Functions

3. Return by Const Reference

Returns a read-only reference

```
#include <iostream>
#include <string>
using namespace std;

class Configuration {
private:
    string setting;
    static Configuration* instance;

public:
    Configuration() : setting("default") {}

    void setSetting(string s) { setting = s; }

    // Return const reference - read-only access
    const string& getSetting() const {
        return setting;
    }
};
```

```
int main() {
    Configuration config;
    config.setSetting("High Performance");

    const string& setting = config.getSetting();
    cout << "Setting: " << setting << endl;

    // setting = "New"; // ERROR! Cannot modify const reference

    // To modify, must use setter
    config.setSetting("Low Power");
    cout << "New setting: " << setting << endl; // Reflects change

    return 0;
}
```

Constructor and Destructor

- Characteristics of Constructors
- Default Constructor
- Parameterized Constructor
- Copy Constructor — deep copy vs shallow copy
- Constructor Overloading
- Destructor — cleanup on object death

Constructor

What Is Constructor In C++?

A constructor in C++ programming is a special member function that is called automatically when an object belonging to the respective class is created. It is the responsibility for the constructors to initialize the object's data members and establish its initial state.

- They make sure that an object is created with valid and consistent values, establishing the foundation for the object's functionality and behavior.
- Constructors in C++ are particularly helpful for allocating resources, setting initial values, and setting up the object for use immediately after creation.
- They have the same name as the class name and don't have the return type, not even void.

Syntax

```
class ClassName {  
    Access_specifier:  
    ClassName() {  
    }  
};
```

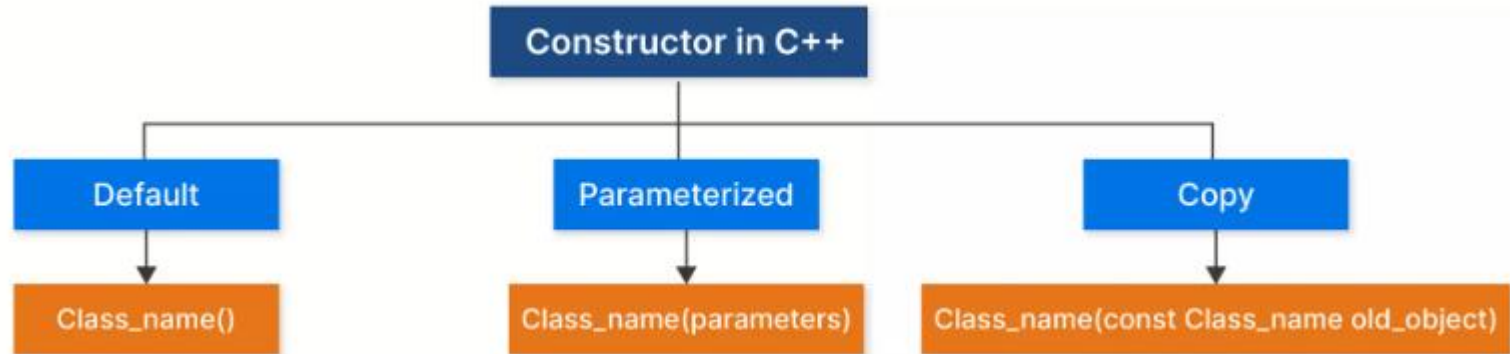
```
class Person {  
private:  
    string name;  
    int age;  
  
public:  
    // Default constructor  
    Person() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    // Parameterized constructor  
    Person(const string& name, int age) {  
        this->name = name;  
        this->age = age;  
    }  
  
    // Member function to display person's information  
    void display() const {  
        cout << "Name: " << name << ", Age: " << age << endl;  
    }  
};  
  
int main() {  
    // Creating instances using constructors  
    Person person1; // Using default constructor  
    Person person2("Unstop", 7); // Using parameterized constructor  
  
    // Displaying person information  
    cout << "Person 1: ";  
    person1.display();  
    cout << "Person 2: ";  
    person2.display();  
  
    return 0;  
}
```


Constructor

A constructor in C++ plays a critical role in classes and the creation of objects. Here are some of its most important characteristics:

- **Class Name Matching:** A constructor's name must match the class name in which it is specified. It improves the compiler's ability to recognize and associate the constructor with the class.
- **Overloading:** We can overload constructors in C++, just like standard functions. This implies that a class may have numerous constructors, each with a unique list of parameters, providing various ways to initialize objects. It provides flexibility in code.
- **Default Constructor:** In case we do not explicitly define a constructor in C++ classes, the compiler creates a default constructor. The object's attributes are initialized with default values in the default constructor, such as 0 for numbers and null for object references.
- **No Return Type:** A constructor in C++ doesn't have any return types, not even void. This is because their primary function is to initialize objects instead of returning values.
- **Automatic Invocation:** When an object is formed, the constructors are automatically called. Unlike an ordinary method, a constructor in C++ does not require an explicit call.
- **Initialization:** An object's attributes, usually referred to as instance variables, are initialized using constructors. They guarantee that an item is in a usable state as soon as it is created.
- **Access Modifiers:** Access modifiers for constructors include public, private, and protected. The constructor in C++ can only be invoked from certain places depending on the access modifier selected.
- **Inheritance:** Unlike ordinary methods, constructors cannot be overwritten or inherited by subclasses.

Constructor



Default Constructor

The default constructor in C++ is a unique constructor that is created and called automatically by the compiler when no other user-defined constructor is explicitly declared within a class.

- The concept of default constructors is used to construct objects with initial default values for its data members and is called without any arguments.
- In other words, it is called for the default initialization of class internal members.

The default constructor in C++ also known as the argument constructor is, hence, crucial when creating objects without specifying any particular values.

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int width;
    int height;

public:
    // Default constructor definition inside the class
    Rectangle() : width(0), height(0) {} // Using
    initializer list

    // Member function to display dimensions
    void displayDimensions() {
        cout << "Width: " << width << ", Height: " << height
        << endl;
    }
};

int main() {
    Rectangle rect; // Create a Rectangle object
    rect.displayDimensions(); // Display its dimensions
    return 0;
}
```

Parameterized Constructor

A **parameterized constructor** in C++ is a special member function that **accepts arguments to initialize an object with specific values at the exact moment of its creation.**

Constructors can also take parameters (just like regular functions), which can be useful for setting initial values for attributes.

- *The following class have brand, model and year attributes, and a constructor with different parameters. Inside the constructor we set the attributes equal to the constructor parameters (brand=x, etc).*

When we call the constructor (by creating an object of the class), we pass parameters to the constructor, which will set the value of the corresponding attributes to the same

```
class Car {           // The class
public:               // Access specifier
    string brand;     // Attribute
    string model;     // Attribute
    int year;         // Attribute
    Car(string x, string y, int z) { // Constructor with
parameters
        brand = x;
        model = y;
        year = z;
    }
};

int main() {
    // Create Car objects and call the constructor with
different values
    Car carObj1("BMW", "X5", 1999);
    Car carObj2("Ford", "Mustang", 1969);

    // Print values
    cout << carObj1.brand << " " << carObj1.model << " " <<
carObj1.year << "\n";
    cout << carObj2.brand << " " << carObj2.model << " " <<
carObj2.year << "\n";
    return 0;
}
```

Copy Constructor

A **copy constructor** is a special constructor in C++ that **makes a copy of an existing object while creating a new one.**

It is utilized whenever an object declaration is passed as a value to a function, returned by value from a function, or when an object is given the value of another object belonging to the same class.

The copy constructor in C++ makes sure that the new object's data members are set to the same values as those of the source object.

```
class Person {
private:
    string name;
    int age;

public:
    // Parameterized constructor
    Person(string n, int a) {
        name = n;
        age = a;}

    // Copy constructor
    Person(const Person &source) {
        name = source.name;
        age = source.age;}

    void displayInfo() {
        cout << "Name: " << name << ", Age: " << age << endl;}

};

int main() {
    Person person1("Shivani", 25);
    Person person2 = person1; // Copy constructor is called
    person2.displayInfo();

    return 0;
}
```

Copy Constructor

```
class Point {
public:
    int x, y;
    Point(int a, int b): x(a), y(b) {}

    // Copy Constructor
    Point(const Point &p) {
        x = p.x; y = p.y;
        cout << "Copy ctor called" << endl;
    }
    ~Point() { cout << "Destroyed" << endl; }
};

int main() {
    Point p1(3, 4);
    Point p2 = p1;    // Copy ctor called
    Point p3(p1);     // Also copy ctor
}
```

💡 If a class has dynamic memory, always define copy constructor to avoid shallow copy bugs.

Copy Constructor

What Happens Without a Copy Constructor?

If you don't define a copy constructor, C++ provides a **default copy constructor** that performs **shallow copy**:

For simple classes like this, the default works fine. But for classes with **dynamically allocated memory**, you **MUST** write your own copy constructor to avoid **shallow copy issues**.

```
// Default copy constructor (provided by compiler)
Rectangle(const Rectangle& other) {
    width = other.width;    // Copies value
    height = other.height;  // Copies value
    // No print message!
}
```

Copy Constructor

Shallow Copy vs Deep Copy

Concept	What Happens	Issue?
Shallow Copy	Copies pointer value, not the data it points to	Both objects point to same memory → Double delete!
Deep Copy	Creates new memory and copies the actual data	Safe - each object has its own memory

Constructor Overloading

Overloading is the technique of constructing numerous functions with the same name but various parameter lists. This is also application on constructor in C++.

Constructor overloading allows you to have multiple constructors in a class, each with a different parameter list. This enables objects to be initialized in various ways based on the provided arguments.

```
class Person {
private:
    string name;
    int age;

public:
    // Parameterized constructor
    Person(string n, int a) {
        name = n;
        age = a;}

    // Copy constructor
    Person(const Person &source) {
        name = source.name;
        age = source.age;}

    void displayInfo() {
        cout << "Name: " << name << ", Age: " << age << endl;}

};

int main() {
    Person person1("Shivani", 25);
    Person person2 = person1; // Copy constructor is called
    person2.displayInfo();

    return 0;
}
```

Constructor Overloading

```
class ClassName {  
access_specifier:  
// Default constructor  
ClassName() {}  
// Parameterized constructor with 1 parameter  
ClassName(parameter_type parameter1) {}  
// Parameterized constructor with 2 parameters  
ClassName(parameter_type parameter1, parameter_type parameter2) {}  
};
```

Here in Syntax,

- The **class** keyword is used to define the class.
- **className** is the name of the class in which the constructor is defined, as well as the name of all constructors, i.e., default and parameterized.
- The **access_specifier** specifies the visibility of the data members and member functions in the class.
- The term **parameter** refers to the parameter taken by the constructor. And **parameter_type** refers to their data type.
- **className(parameter_type parameter1)**: It is a parametrized constructor with 1 parameter.
- **className(parameter_type parameter1,parameter_type parameter2)**: It is a parametrized constructor with 2 parameters.

Constructor Overloading

```
#include <iostream>
#include <string>

using namespace std;

class Student {
public:
    // Default constructor
    Student() {
        name = "Unknown";
        age = 0;
    }

    // Parameterized constructor #1
    Student(string n) {
        name = n;
        age = 0;
    }

    // Parameterized constructor #2
    Student(string n, int a) {
        name = n;
        age = a;
    }
}
```

```
// Member function to display student information
void displayInfo() {
    cout << "Name: " << name << ", Age: " << age << endl;
}

private:
    string name;
    int age;
};

int main() {
    // Creating Student objects using different constructors
    Student student1; // Using the default constructor
    Student student2("Anya"); // Using parameterized
    constructor #1
    Student student3("Harsh", 20); // Using parameterized
    constructor #2

    // Displaying student information
    student1.displayInfo();
    student2.displayInfo();
    student3.displayInfo();

    return 0;
}
```

Destructor

A **destructor** in C++ is a special member function invoked when an object is destroyed, either when it goes out of scope or is explicitly deleted. It has the same name as the class, prefixed with a tilde (~).

- The C++ runtime automatically calls the proper destructor to release any resources retained by an object when its lifetime ends.
- This could be either because the object exits scope or if it is explicitly removed using the delete keyword (in the case of objects created dynamically).
- This ensures appropriate dynamic memory management to prevent memory leaks and resource wastage.

```
class FileHandler {
private:
    std::ofstream file;

public:
    FileHandler(const std::string& filename) {
        file.open(filename);
        if (!file.is_open()) {
            std::cout << "Error opening file.\n";
        }

        ~FileHandler() {
            if (file.is_open()) {
                file.close();
                std::cout << "The file closed successfully.\n";
            }
            // Other member functions and data members for file
            // handling operations
        };

        int main() {
            FileHandler fileHandler("example.txt");
            // Perform file operations...
            // fileHandler's destructor will automatically be called
            // at the end of the scope,
            // ensuring the file is properly closed, and resources are
            // released.
            return 0;
        }
    }
```

Constructor vs Destructor

Constructor

- Called when object is CREATED
- Can be overloaded
- Can have parameters
- Initializes object's data members
- Syntax: `ClassName()`
- Multiple constructors possible

Destructor

- Called when object is DESTROYED
- Cannot be overloaded
- No parameters
- Releases resources (memory, files)
- Syntax: `~ClassName()`
- Only ONE destructor per class

Analogy & PU Question



Real-World Analogy

A constructor is like hotel check-in: keys given, room configured. The destructor is checkout: return keys, clean room, free space for next guest. The copy constructor is like duplicating your hotel booking to give to a friend — a separate, independent reservation.



Probable PU Exam Question

Q: Explain all types of constructors in C++ with programs. What is a copy constructor and when is it invoked? [PU Exam 2023]

Operator Overloading

- What is Overloading
- Types of Overloading
- (Recap)Function Overloading
- Operator Overloading
- Rules of Operator Overloading
- Unary Operator Overloading (++ , -- , unary -)
- Binary Operator Overloading (+ , - , * , ==)
- Type Conversion: Object $\leftrightarrow$ Basic Type
- Overloading using Friend Functions

Overloading in C++

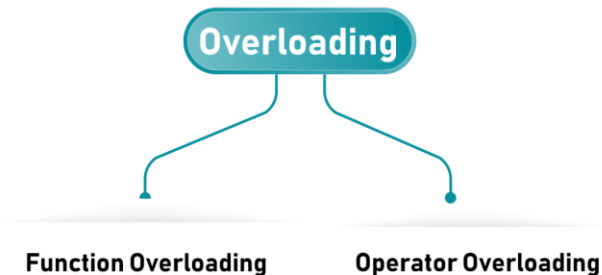
Creating two or more members that have the same name but are different in number or type of parameter is known as **overloading**.

In C++, we can overload:

- Methods
- Operators
- Constructors

We can overload these members because they have parameters *only*.

Types of overloading



Function Overloading

The process of having two or more functions with the same name, but different parameters, is known as **function overloading**.

The function is redefined by either using different types of arguments or a different number of arguments. It is only through these differences that a compiler can differentiate between functions.

The advantage of function overloading is that it increases the readability of a program because you don't need to use different names for the same action.

Different Ways of Function Overloading

A function in C++ can be overloaded in three different ways:

- By having different number of parameters.
- By having different types of parameters.
- By having both different number and types of parameters.

```
#include <iostream>
using namespace std;

class Cal {
public:
    static int add(int a,int b){
        return a + b;
    }
    static int add(int a, int b, int c) {
        return a + b + c;
    }
};

int main(void) {
    Cal C;          //class object declaration.
    cout<<C.add(10, 20)<<endl;
    cout<<C.add(12, 20, 23);
    return 0;
}
```

Function Overloading

Functions that Cannot be Overloaded

In C++, we have some specific scenarios in which the functions cannot be overloaded. As function overloading is primarily based on the function name and the types or number of its parameters. There could be cases where the C++ compiler cannot differentiate between functions and thus cannot be overloaded.

- Member function declarations with different access specifier but same name and same parameter-type-list cannot be overloaded.
- Functions with parameter declarations that differ only in a pointer * versus an array [].
- Functions with same parameter but different passing method (i.e. passed by value and passed by reference).
- Function with same parameter but different volatile type specifier.

Operator Overloading

Scenario:

Suppose we have two boxes:

Box A

5 Books

Box B

7 Books

We want to combine them.

Normally,

$$5 + 7 = 12$$

works because integers already know how to use the + operator.

But what if our Box itself is an object?

Box A + Box B

Does C++ know how to add two boxes? NO.

Because C++ only knows

int + int
float + float
double + double
char + char

It does NOT know

Box + Box
Student + Student
Time + Time
Distance + Distance
Money + Money

So, we teach C++ how to perform these operations.

This is called **Operator Overloading**

Operator Overloading

The concept of overloading a function can be applied to operators as well.

Real-Life Analogy:

Think of a universal remote control. The "Power" button works differently for your TV (turns it on/off), your sound system (switches between standby and active), and your air conditioner (turns cooling on/off). The same button (operator) performs different actions based on what device (data type) it's controlling.

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide special meaning to the user-defined data type. Operator overloading is used to overload or redefine most of the operators available in C++. It is used to perform the operation on the user-defined data type.

Technical Definition:

Operator overloading allows to define how operators (like +, -, *, etc.) work with user-defined data types (classes). It gives special meanings to existing operators when it is used with user-defined objects of the class.

Operator Overloading

Why Use Operator Overloading?

The primary reason for using operator overloading is to make code involving user-defined types as intuitive and readable as code involving built-in types.

For example,
if we have a Complex class to represent complex numbers, overloading the + operator allows us to write `c1 + c2` instead of `c1.add(c2)`.

This can make mathematical operations on complex numbers look more natural and familiar.

Operator-overloading

$(1+2i) + (2-1i)$

$(1+2i) + (2-1i)$
 $= (3+1i)$

Complex operator+()
const;



custom operator+() logic



Operator Overloading

How Operator Overloading Works

In C++, operators are essentially functions with special names. When you overload an operator, you define a function that provides the operator's new behavior for specific types. The name of this function is formed by the operator keyword followed by the symbol of the operator being overloaded.

Operators can be overloaded in two primary ways:

- **Member Function:** The operator function is defined as a member of the class.
- **Non-Member Function:** The operator function is defined outside the class, usually as a friend function.

Operator Overloading

In C++, operator overloading is precisely that: defining a function with a special name—the ***operator*** keyword followed by the ***operator symbol***—to define custom behavior for that operator when applied to user-defined types.

- **Operators are function calls in disguise.** When the compiler sees `a + b`, it looks for a function named `operator+(a, b)` (or `a.operator+(b)` for a member function).
- **The name is fixed and mandatory.** The function name *must* be `operator@`, where `@` is the actual operator (e.g., `+`, `-`, `==`, `[]`, `()`).
- **You define the behavior entirely.** The function body determines what "addition," "equality," or "function call" means for your class.
- **Restrictions apply.** You cannot invent new symbols (e.g., `operator**`), nor can you overload operators for built-in types (e.g., redefining `int::operator+`).

Operator Overloading

Syntax

```
return_type operator  
symbol(parameters)  
{  
    ...  
}
```

- return_type = What the operation returns
- operator = Keyword (mandatory)
- symbol = The operator being overloaded (+, -, *, etc.)
- parameters = Operands (at least one must be user-defined type)

Example

```
Complex operator+(Complex obj)  
{  
    ...  
}
```

```
#include <iostream>  
using namespace std;  
  
class Complex {  
private:  
    int real, imag;  
public:  
    Complex(int r = 0, int i = 0) {real = r;    imag = i;}  
  
    // This is automatically called when '+' is used with  
    // between two Complex objects  
    Complex operator + (Complex obj) {  
        Complex temp;  
        temp.real = real + obj.real;  
        temp.imag = imag + obj.imag;  
        return temp;  
    }  
    void print() { cout << real << "+" << << imag << "i" << endl;  
    }  
};  
  
int main()  
{  
    Complex c1(10, 5), c2(2, 4);  
    Complex c3 = c1 + c2; // An example call to "operator+"  
    c3.print();  
    return 0;  
}
```


Operator Overloading

```
#include <iostream>
using namespace std;
class Shape{
private:
    int length;    // Length of a box
    int width;
public:
    // Constructor definition
    Shape(int l = 2, int w = 2){
        length = l;
        width = w;
    }
    double Area(){
        return length * width;
    }
    int operator + (Shape shapeIn){
        return Area() + shapeIn.Area();
    }
};

int main(void){
    Shape sh1(4, 4);    // Declare shapel
    Shape sh2(2, 6);    // Declare shape2

    int total = sh1 + sh2;
    cout << "\nsh1.Area() = " << sh1.Area();
    cout << "\nsh2.Area() = " << sh2.Area();
    cout << "\nTotal = " << total;
    return 0;
}
```

Operator Overloading Rules

- CANNOT overload: :: ..* ?: sizeof
- At least one operand must be user-defined type
- Cannot change precedence or associativity
- Cannot create brand-new operators
- Cannot change number of operands
- Member function approach: left operand is the calling object
- Friend function approach: both operands are explicit parameters
- Use friend for commutative operations or when left operand is not your class (e.g., `cout << obj`)

```
class Complex {
public:
    float r, i;
    Complex(float a=0, float b=0):r(a),i(b){}

    // Binary + as member
    Complex operator+(Complex c) {
        return Complex(r+c.r, i+c.i);
    }

    // Unary - as member
    Complex operator-() {
        return Complex(-r, -i);
    }
};
```

Operator Overloading Rules

Rules of Operator Overloading

1. **Syntax/Declaration Rule:** While overloading a function we need to use the keyword **operator** , **preceded by class name and followed by operator symbol such as '+'**.
2. **Member and Non-member functions:** We can overload operators in two ways:- first is as a member functions , second is as a non-member functions. When overloading as a member function, the left operand represents the object on which the function is called and when overloading as a non-member function, we can specify any type for the left operand.
3. **Number of operands Required:** Most operators can be overloaded with one or two operands. Example- Unary operators like ++ require one operand, while binary operators like + require two operands.
4. **Precedence and associativity:** Operator precedence and associativity cannot be changed when overloading operators. Example- It's not allowed to change the precedence of (*) or the associativity of (/).
5. **Return type:** When we overload operators than we must define the return type of the overloaded operator function according to our need.
6. **Friend function:** If overloaded operators needs to access private member variables/functions of a class than they must be declared as a friend function.
7. **Special Syntax:** For operators like (), [], or -> we must use a special syntax when overloading. Example, in order to overload the [] operator and to make objects callable like functions, we need to define a function named operator().

Operator Overloading

How Operator Overloading Works

In C++, operators are essentially functions with special names. When an operator is overloaded, we define a function that provides the operator's new behavior for specific types. The name of this function is formed by the operator keyword followed by the symbol of the operator being overloaded.

Operators can be overloaded in two primary ways:

- **Member Function:** The operator function is defined as a member of the class.
- **Non-Member Function:** The operator function is defined outside the class, usually as a friend function.

Operator Overloading Unary and Binary

BINARY

operator+

```
MyClass operator+  
(const MyClass&  
other)  
{  
    ...  
}
```

UNARY

operator-

```
MyClass  
operator-()  
{  
    ...  
}
```

Operators can be categorized as unary or binary based on the number of operands they take. Unary operators take one operand, while binary operators take two.

- **Unary Operators:** such as ++ (increment) and -- (decrement), operate on a single operand. To overload a unary operator, you define a member function with no parameters or a non-member function with one parameter.
- **Binary Operators:** such as +, -, *, and ==, operate on two operands. To overload a binary operator, you define a member function with one parameter or a non-member function with two parameters.

Operator Overloading Unary and Binary

What are Unary Operators?

Operators that work on ONE operand.

Examples: ++, --, !, ~, - (negative)

Real-Life Analogy:

The Light Switch - One switch (operator) affects one light (operand). You press it once, and it changes the state of that single light.

What are Binary Operators?

Operators that work on **TWO** operands.

Examples: +, -, *, /, ==, !=, <, >

Real-Life Analogy:

The Calculator - You input two numbers (operands), press the '+' button (operator), and get one result.

Operator Overloading Unary and Binary

```
#include <iostream>
using namespace std;

class Counter {
private:
    int value;

public:
    Counter(int v = 0) : value(v) {}

    // Prefix increment: ++obj
    Counter operator++() {
        ++value;          // Increment first
        return *this;     // Return updated object
    }

    // Postfix increment: obj++
    Counter operator++(int) { // int parameter is dummy for
postfix
        Counter temp = *this; // Save old state
        ++value;             // Increment
        return temp;         // Return old state
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};
```

```
void display() {
    cout << "Value: " << value << endl;
}

};

int main() {
    Counter c(5);

    cout << "Original: ";
    c.display(); // Output: 5

    Counter c1 = ++c; // Prefix
    cout << "After prefix ++: ";
    c.display();      // Output: 6
    cout << "Returned: ";
    c1.display();     // Output: 6

    Counter c2 = c++; // Postfix
    cout << "After postfix ++: ";
    c.display();      // Output: 7
    cout << "Returned: ";
    c2.display();     // Output: 6

    return 0;
}
```

Operator Overloading Unary and Binary

```
#include <iostream>
using namespace std;

class Complex {
private:
    float real, imag;
public:
    Complex(float r = 0, float i = 0) : real(r), imag(i) {}

    Complex operator+(const Complex& c) {
        return Complex(real + c.real, imag + c.imag);
    }

    Complex operator-(const Complex& c) {
        return Complex(real - c.real, imag - c.imag);
    }

    bool operator==(const Complex& c) {
        return (real == c.real) && (imag == c.imag);
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};
```

```
int main() {
    Complex c1(3.0, 4.0); // 3 + 4i
    Complex c2(1.5, 2.5); // 1.5 + 2.5i

    Complex c3 = c1 + c2; // calls
    operator+(c2)
    c3.display(); // 4.5 + 6.5i

    Complex c4 = c1 - c2;
    c4.display(); // 1.5 + 1.5i
    cout << (c1 == c2) << endl; // 0 (false)
    return 0;
}
```


Operator Overloading – Type Conversion

When constants and variables of different types are mixed in an expression, C++ applies automatic type conversion to the operands as per certain rules. Similarly, an assignment operation also causes the automatic type conversion. The type of data to the right of an assignment operator is automatically converted to the type of variable on the left.

The three types of situations in the data conversion are:

1. Conversion from basic type to class type.
2. Conversion from class type to basic type.
3. Conversion from one class type to another class type

```
class Complex {  
    public:  
        float r, i;  
        Complex(float a=0, float b=0):r(a),i(b){}  
  
        // Binary + as member  
        Complex operator+(Complex c) {  
            return Complex(r+c.r, i+c.i);  
        }  
  
        // Unary - as member  
        Complex operator-() {  
            return Complex(-r, -i);  
        }  
};
```

Operator Overloading – Type Conversion

Basic Type to Class Type

To convert a primitive data type (like `int` or `double`) into a custom class object, we use a **parameterized constructor**. The constructor acts as the conversion mechanism.

```
#include <iostream>

class Distance {
private:
    int meters;
public:
    Distance() : meters(0) {}

    // Constructor handles: int -> Distance conversion
    Distance(int m) {
        meters = m;
    }

    void display() const {
        std::cout << meters << " meters\n";
    }
};

int main() {
    // Implicit conversion from basic type (int) to
    // Class type (Distance)
    Distance d = 50;
    d.display(); // Outputs: 50 meters
}
```

Operator Overloading – Type Conversion

Class Type to Basic Type

To convert a custom object back into a primitive data type, you must overload the **casting/conversion operator**.

Rules for Conversion Operators:

- It must be a **member function** of the class.
- It **cannot specify a return type** explicitly (the return type is implied by the operator's name).
- It takes **no arguments**.

```
#include <iostream>

class Fraction {
private:
    int num, den;
public:
    Fraction(int n, int d) : num(n), den(d) {}

    // Overloading the float cast operator: Class ->
    float
    operator float() const {
        return static_cast<float>(num) / den;
    }
};

int main() {
    Fraction f(3, 4);

    // Implicit conversion from Class (Fraction) to
    basic type (float)
    float val = f;
    std::cout << "Value: " << val << "\n";
    return 0;
}
```

Operator Overloading – Type Conversion

One Class Type to Another Class Type

Converting between two entirely different custom classes requires careful management to prevent code ambiguity. This type of conversion can be accomplished using two approaches

Approach A: Using a Constructor in the Destination Class

The destination class defines a constructor that accepts an instance of the source class as an argument.

```
class Inventory {
public:
    int items = 10;
};

// Destination Class
class Product {
private:
    int count;
public:
    Product() : count(0) {}

    // Constructor handles: Inventory -> Product
    conversion
    Product(const Inventory& inv) {
        count = inv.items;
    }
};
```

Operator Overloading – Type Conversion

Approach B: Using Conversion Operator in Source Class

The source class overloads a casting operator targeted to return an instance of the destination class.

```
class Product; // Forward declaration

// Source Class
class Inventory {
private:
    int items = 10;
public:
    // Overloaded conversion operator: Inventory -> Product
    operator Product() const;
};
```

Analogy & PU Question



Real-World Analogy

Operator overloading is like teaching a robot what '+' means for recipes. By default, + is not defined for Recipe objects. You define: 'Recipe + Recipe = combined ingredient list'. Now + means something meaningful for your type. The robot (compiler) knows exactly what to do.



Probable PU Exam Question

Q: What is operator overloading? Write a C++ program to overload '+' for complex numbers. Explain type conversion between objects and basic types. [PU Exam 2022]

Polymorphism

- Compile-time (Early) Binding vs Runtime (Late) Binding
- Virtual Functions — runtime dispatch
- Virtual Function Table (vtable) mechanism
- Pure Virtual Functions
- Abstract Classes

```
class Base {  
    virtual void control() = 0;  
};
```

```
class TV : public Base {  
    override void control();  
};
```

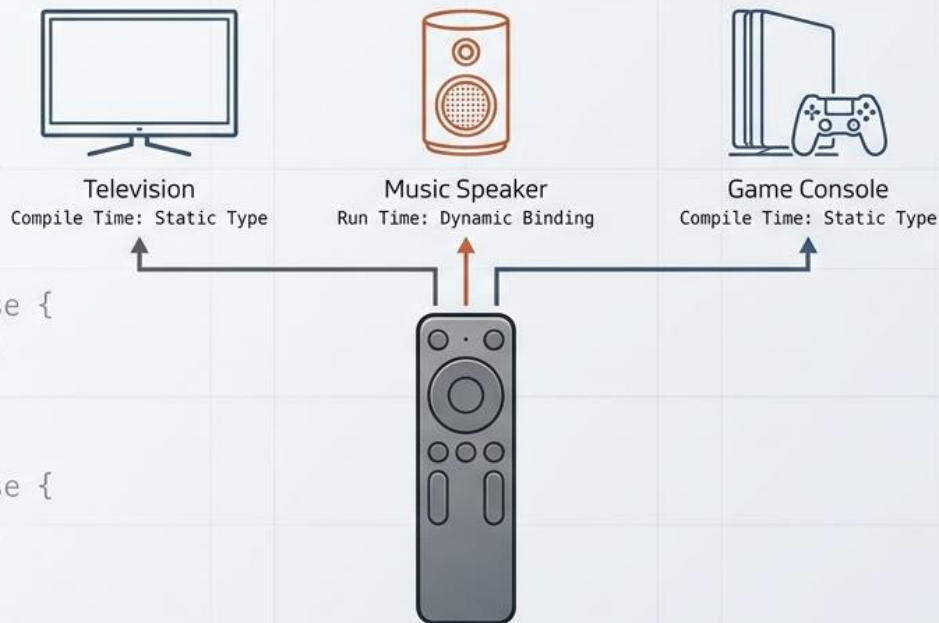
```
class TV : public Base {  
    override void control();  
};
```

```
class Speaker : public Base {  
    override void control();  
};
```

```
class Console : public Base {  
    override void control();  
};
```

The Architecture of Many Forms

A Visual Guide to C++ Polymorphism and Dynamic Binding



Polymorphism

The remote-control analogy

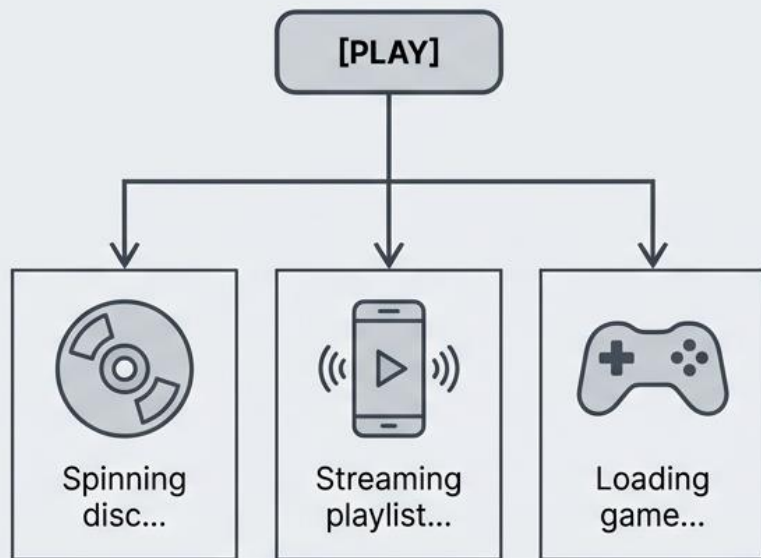
Imagine you have a universal remote with a "**Play**" button. When you press it:

- **DVD Player** → Starts spinning the disc and playing a movie
- **Music Streaming App** → Begins streaming your playlist
- **Video Game Console** → Loads and starts the game

Same action (Play), different behaviors depending on what device you're pointing at. That's polymorphism!

One Interface, Multiple Behaviors

The Real World



One Interface, Multiple Behaviors

The C++ Reality

```
Device* myDevice = getDevice();  
  
myDevice->play(); // What happens here?
```

The core philosophy of polymorphism is reducing cognitive load. You call `play()` on a generic pointer, and the system automatically routes the command to the correct behavior based on the specific device. The big question: **when does the system decide which path to take?**

Polymorphism

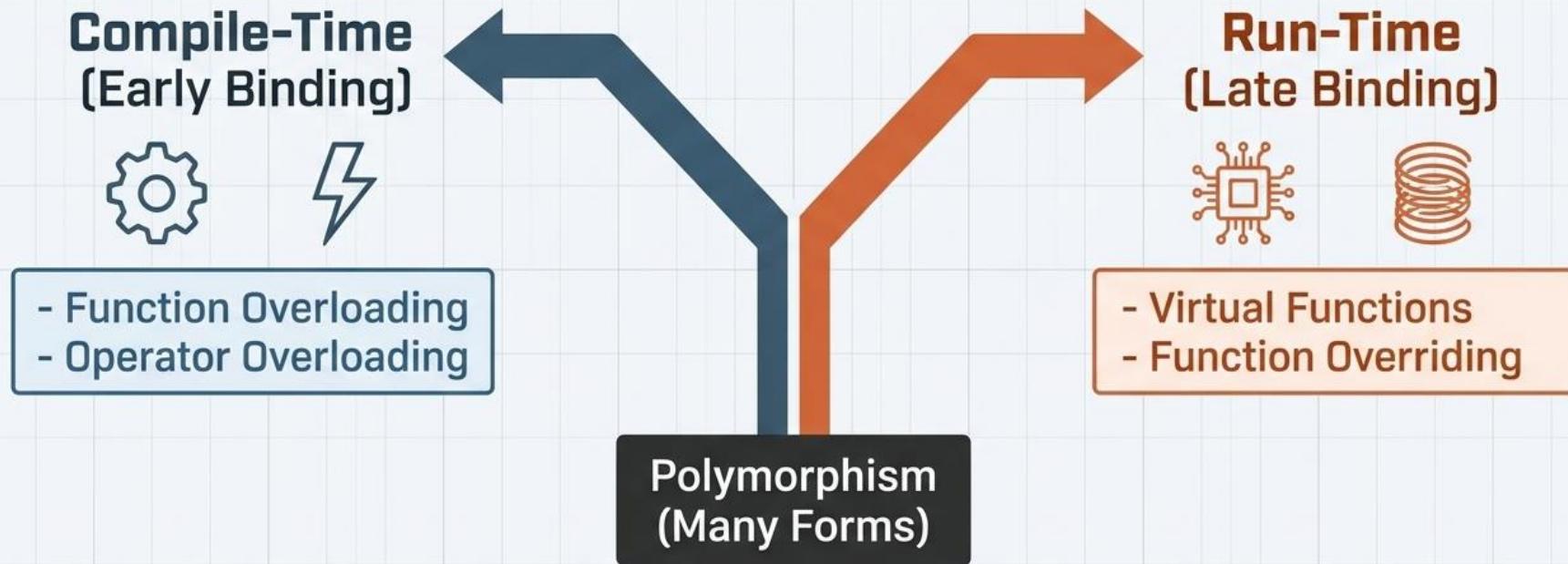
What is Polymorphism?

Polymorphism = "Poly" (many) + "Morph" (forms) = **Many forms**

In C++ OOP, polymorphism lets you treat different objects uniformly while each object performs its own specific version of an action.

Polymorphism stands as one of the four essential pillars of Object-Oriented Programming (OOP). To a Technical Architect, polymorphism is the strategic mechanism that allows for the creation of extensible and decoupled systems. It provides a single interface to a variety of underlying forms, ensuring that adding new features does not necessitate a complete rewrite of the core system logic.

The Grand Divide of C++ Execution



C++ empowers you to choose when function resolution happens: hardcoded before the program runs (Blue), or dynamically evaluated while running (Orange).

Classification of Polymorphism

Two completely different types of polymorphism

Compile-time Polymorphism (Static/Early Binding)

Compiler picks which function to call at compile time. Done through function overloading and operator overloading. Also called *early binding* or *static polymorphism*.

- function overloading
- operator overloading

Runtime Polymorphism (Dynamic/Late Binding)

Program decides which function to call while it is running. Done through virtual functions and base class pointers. Also called late binding or dynamic polymorphism.

- virtual functions
- base class pointer / Function

The key question that separates them

When does C++ decide which function to call — at compile time or at runtime?

If the answer is compile time → compile-time polymorphism. If runtime → runtime polymorphism.

Comparison Table: Binding Mechanisms

Feature	Compile-time (Static/Early Binding)	Run-time (Dynamic/Late Binding)
Resolution Timing	Occurs during the compilation phase.	Occurs during program execution.
Execution Speed	Faster; zero runtime overhead.	Slightly slower (vtable lookup overhead).
Flexibility	Fixed at compile time; less extensible.	Highly flexible; supports plugin architecture.
Implementation	Function/Operator Overloading.	Function Overriding/Virtual Functions.
Resolution Rule	Resolved via function signature.	Resolved via object type (vptr).

The "So What?" Layer

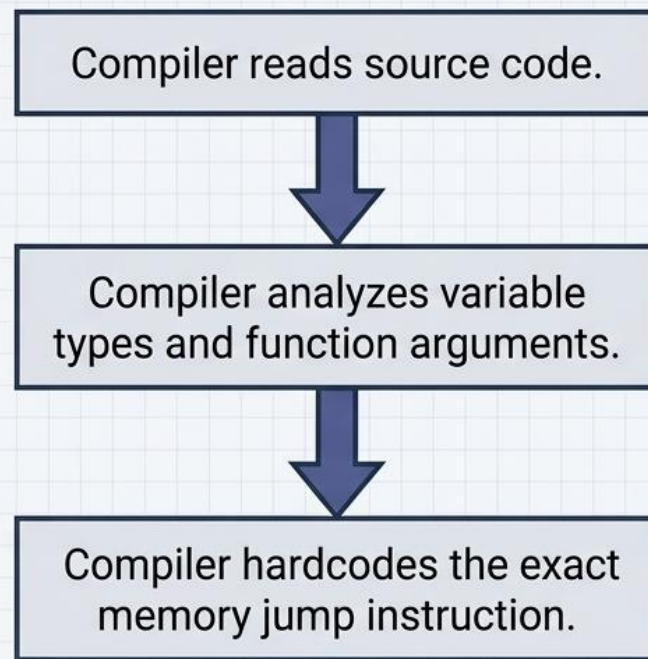
The trade-off is efficiency versus adaptability. **Compile-time polymorphism** is preferred when performance is the absolute priority and the data types are known. **Run-time polymorphism** is essential for complex systems where the specific object type—such as a user-selected tool in a graphics suite—cannot be determined until the program is actually running.

Early Binding Works Like a Fixed Menu

The Metaphor



The Mechanism



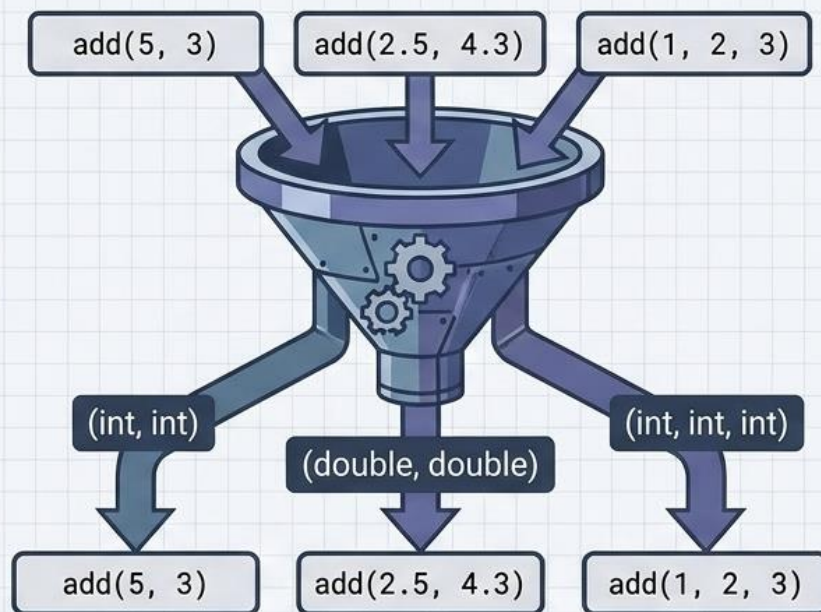
Zero Runtime Overhead. Blazingly Fast Execution. Decided before the program ever opens.

Function Overloading Sorts by Signature

The Code Definitions

```
int add(int a, int b) {  
    return a + b;  
}  
  
double add(double a, double b) {  
    return a + b;  
}  
  
int add(int a, int b, int c) {  
    return a + b + c;  
}
```

The Compile-Time Sorting Mechanism



The compiler treats these as entirely different functions based on their parameter lists. All routing is resolved purely at compile time.

Compile-time Polymorphism – Overloading (Early binding)

Overloading allows the same interface to handle different data types or parameter counts, streamlining the API for developers and ensuring intuitive syntax.

Function Overloading

Definition: Multiple functions share a name but differ in their signature (parameter type or count).

Architectural Warning:

Implicit Conversion A common pedagogical pitfall is ignoring **Implicit Conversion**.

If you define an `add(int, int)` function but pass double arguments (e.g., 2.5 and 4.3), C++ will implicitly convert them to integers, resulting in data loss (returning 6 instead of 6.8).

Overloading with a specific double version prevents this error.

```
class Calculator {  
public:  
    int add(int a, int b) { return a + b; }  
    // Overloading prevents implicit conversion/data  
    loss  
    double add(double a, double b) { return a + b; }  
    int add(int a, int b, int c) { return a + b + c; }  
};
```

Redefining Operators for Custom Types

obj1 + obj2

operator+

obj1.operator+(obj2)

```
class Complex {  
public:  
    // Redefining the '+' operator  
    Complex operator+(Complex obj) {  
        Complex res;  
        res.real = real + obj.real;  
        return res;  
    }  
};
```

SUMMARY

- **Concept:** Redefining native symbols (+, -, *, ==) for custom classes.
- **Purpose:** Makes custom objects behave intuitively, like primitive types.
- **Resolution:** The compiler translates the symbol and binds it to the custom function **at compile time**.

Compile-time Polymorphism – Overloading (Early binding)

Operator Overloading

Definition: Redefining the behavior of standard operators (e.g., +, -) for user-defined classes using the operator keyword.

Architectural Warning:

Implicit Conversion A common pedagogical pitfall is ignoring **Implicit Conversion**.

If you define an `add(int, int)` function but pass double arguments (e.g., 2.5 and 4.3), C++ will implicitly convert them to integers, resulting in data loss (returning 6 instead of 6.8).

Overloading with a specific double version prevents this error.

The "So What?" Layer

Overloading enhances code readability by allowing intuitive syntax like `s1 + s2` rather than `s1.add(s2)`. This aligns software logic with mathematical expectations, making the codebase more maintainable for large teams.

```
#include <iostream>
using namespace std;

class Score {
    int val;
public:
    Score(int v) : val(v) {}
    // Operator Overloading Syntax: Return_Type
    operator Symbol()
    void operator-() {
        val = -val;
    }
    void display() { cout << "Value: " << val << endl;
}
};

int main() {
    Score s1(50);
    -s1; // Calls operator-()
    s1.display(); // Output: Value: -50
    return 0;
}
```

Late Binding is the Chef's Special



The exact dish isn't known until you order it based on fresh ingredients.

Program is actively running.

Code requests a function via a generic pointer.

System pauses to inspect the ACTUAL object in memory.

System dynamically routes to the correct function.

Requires immense flexibility. The decision of which function to call is deferred to the exact moment of execution.

Run-time Polymorphism – Overriding (Late binding)

What is Overriding?

Definition: A derived class provides its own implementation of a virtual function from the base class.

Rules of Overriding:

Function must be virtual in base class

Same name, return type, and parameters

Run-time Polymorphism – Overriding (Late binding)

Run-time polymorphism enables dynamic behavior in class hierarchies, allowing a child class to provide a specific implementation of a parent's function.

Function Overriding Explained

Definition: A derived class provides a new definition for a base class function. To succeed, the **Same-Signature Rule** must be followed: name, parameters, and return types must match exactly.

Step-by-Step Logic:

Inheritance: A Child class inherits from a Parent class.

Definition: Both classes define void display().

Execution: When a Child object calls display(), the local version takes precedence.

Mechanism: The child's version "discards" the parent's version during execution as the compiler resolves the call based on the local scope of the object.

```
#include <iostream>
using namespace std;

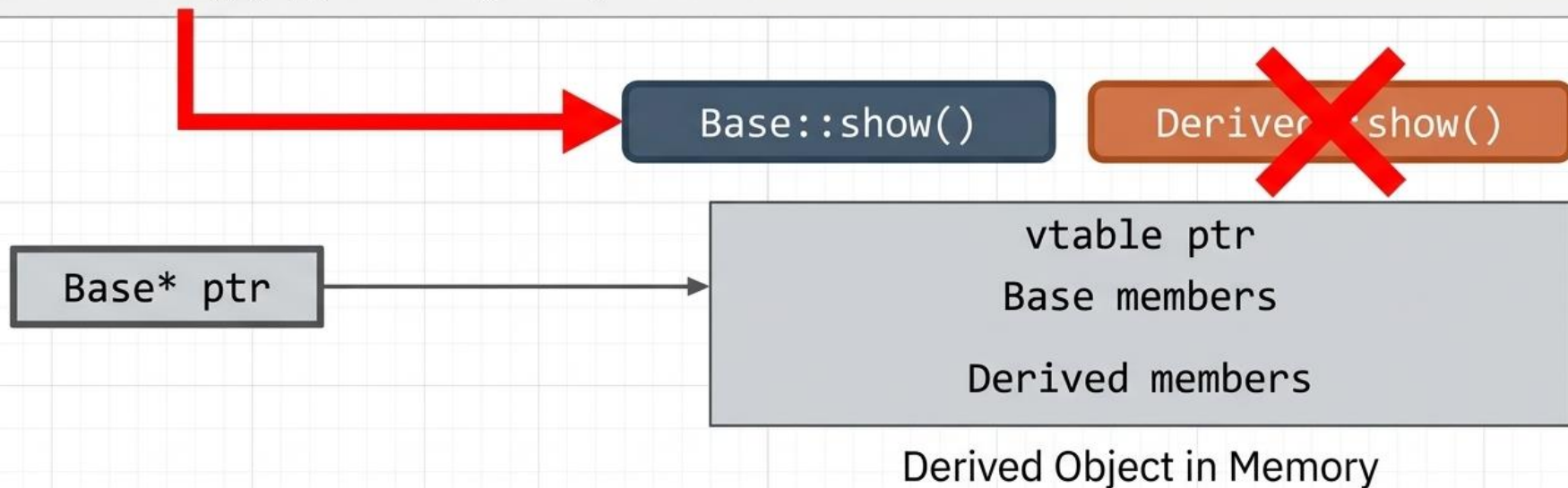
class A {
public:
void display() { cout << "I am Class A " << endl; }
};

Class B : public A {
public:
void display() { cout << "I am Class B" << endl; }
};

int main() {
    B b1;
    b1.display();
    return 0;
}
```


The Pointer Problem: When the Compiler Assumes Too Much

```
Base* ptr = new Derived();  
ptr->show(); // What gets printed?
```



Without intervention, C++ defaults to **Early Binding**. It looks at the **POINTER POINTER TYPE** (Base), not the **ACTUAL OBJECT TYPE** (Derived) it points to.

Run-time Polymorphism – Overriding (Late binding)

```
#include <iostream>
using namespace std;

class Base {
public:
void display() { cout << "I am Base Class" << endl; }
};

Class Derived : public Base {
public:
void display() { cout << "I am Derived Class" << endl; }
};

int main() {
    Base* ptr;
    Derived d;
    ptr = &d

    bptr->d;
    return 0;
}
```

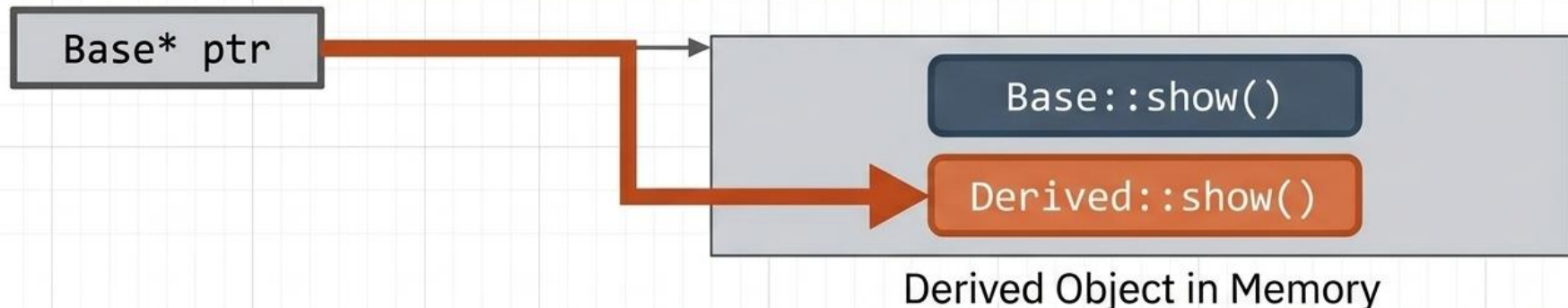

Forcing Late Binding with Virtual Functions

Base Class

```
class Base {  
public:  
    virtual  
    void show() { // Wait until runtime!  
        cout << "Base class";  
    }  
};
```

Derived Class

```
class Derived : public Base {  
public:  
    void show()  
    override { // Replaces base behavior  
        cout << "Derived class";  
    }  
};
```



> Output: Derived class

Run-time Polymorphism – Overriding (Late binding)

A **virtual function** is a member function which is declared within base class and re-defined(Overridden) by derived class

When we refer to a derived class object using pointer or a reference to the base class, we can call a virtual function for that object and execute the derived class's version of the function.

- Virtual functions ensures that the correct function is called for an object, regardless of the type of reference(or pointer) used for function call.
- They are mainly used to achieve Runtime polymorphism
- Functions are declared with virtual keyword in base class.
- The resolving of function call is done at Run-time.

Virtual Functions

```
#include <iostream>
using namespace std;

// Base class
class Animal {
public:
// Virtual function enables overriding in derived classes
    virtual void speak() {
        cout << "Animal makes a sound" << endl;
    }

// Pro Tip: Always use a virtual destructor in base
// classes to prevent memory leaks virtual ~Animal() {}
};

// Derived class
class Dog : public Animal {
public:
// 'override' keyword (C++11+) ensures the signatures match
// correctly
    void speak() override {
        cout << "Woof! Woof!" << endl;
    }
};
```

```
int main() {
// 1. Create a base class pointer
    Animal* ptr;

// 2. Create an object of the derived class
    Dog myDog;

// 3. Point the base pointer to the derived class object
    ptr = &myDog;

    /* 4. Call the virtual function through the pointer
    Because 'speak' is virtual, the program performs "late
    binding" and calls the Dog version, not the Animal
    version.*/

    ptr->speak();

    return 0;
}
```

Pure Virtual Function

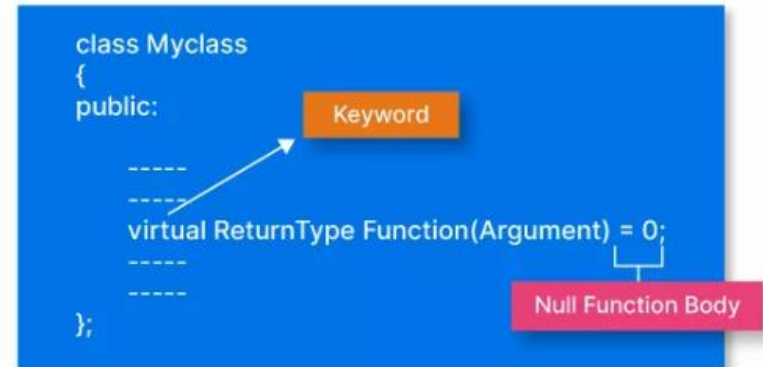
Definition:

A virtual function that has no implementation in the base class. It MUST be overridden in derived classes.

In C++, a pure virtual function (or an abstract function) is a special type of virtual function that is declared in a base class but does not have a definition (i.e., it has no implementation) in the base class. Instead, it is intended to be overridden and implemented by derived classes. A pure virtual function in C++ is used to define an interface or an abstract base class, which cannot be instantiated on its own but can be used as a blueprint for creating concrete derived classes.

Syntax: `virtual void functionName() = 0;`

- The resolving of function call is done at Run-time.



Pure Virtual Function

```
class Base {  
public:  
    virtual void pureVirtualFunction() = 0; // Pure virtual function body  
};
```

- **Base** is the name of the class which contains the function.
- The virtual keyword indicates that the function with the name **pureVirtualFunction** is virtual.
- The ***equal to zero (= 0)*** in the function definition indicates that it is a pure virtual function. This is what primarily differentiates a virtual function from a pure virtual function. This makes the class an abstract base class, and you cannot create objects of this class directly.

Abstract Class

The Abstract Class Concept

A class with at least one pure virtual function becomes an **abstract class** - you cannot create objects of it. It's like a blueprint or contract.

Real-life Analogy: The "Recipe" Contract

Think of a "**Recipe**" concept in a cookbook:

The recipe says: "You **MUST** have a way to cook this dish"

But each restaurant's specific recipe is different

You can't cook a generic "Recipe" - you must have actual dishes!

An abstract class in C++ is a class that cannot be instantiated directly and is often used as a base class for other classes. Abstract classes are designed to provide a common interface or contract for derived classes while leaving the implementation details to those subclasses. They contain pure virtual functions, which are virtual functions with no implementation in the base class. The presence of pure virtual functions in an abstract class ensures that a derived class provides an implementation for those functions.

Abstract Class

```
class AbstractClass {  
public:  
    virtual void pureVirtualFunction() = 0; // Pure virtual function  
    // Other members and methods (can be both virtual and non-virtual)  
};
```

- **AbstractClass** is the name of the abstract class.
- **pureVirtualFunction()** is a pure virtual function, which means it has no implementation in the base class.

Virtual Functions & Abstract Classes

- Virtual keyword in base class enables runtime dispatch
- Base class POINTER can call derived class methods
- override keyword (C++11) catches spelling mistakes
- Pure virtual: virtual void func() = 0;
- Makes class ABSTRACT — cannot be instantiated
- All pure virtual functions MUST be implemented in derived
- vtable: Compiler creates table of function pointers per class
- Each object has a vptr pointing to its class vtable
- Runtime lookup: ptr → vptr → vtable → correct function

```
class Shape {
public:
    virtual void draw() = 0; // Pure virtual
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Circle" << endl;
    }
};

int main() {
    Shape *p = new Circle;
    p->draw(); // Runtime: Circle
}
```


Analogy & PU Question



Real-World Analogy

Virtual functions are like a 'Speak' command to different animals. You tell any animal to 'Speak' (same command through a base pointer). A Dog barks, a Cat meows, a Duck quacks. The actual sound is determined at RUNTIME based on the real animal — not at compile time based on what you called it.



Probable PU Exam Question

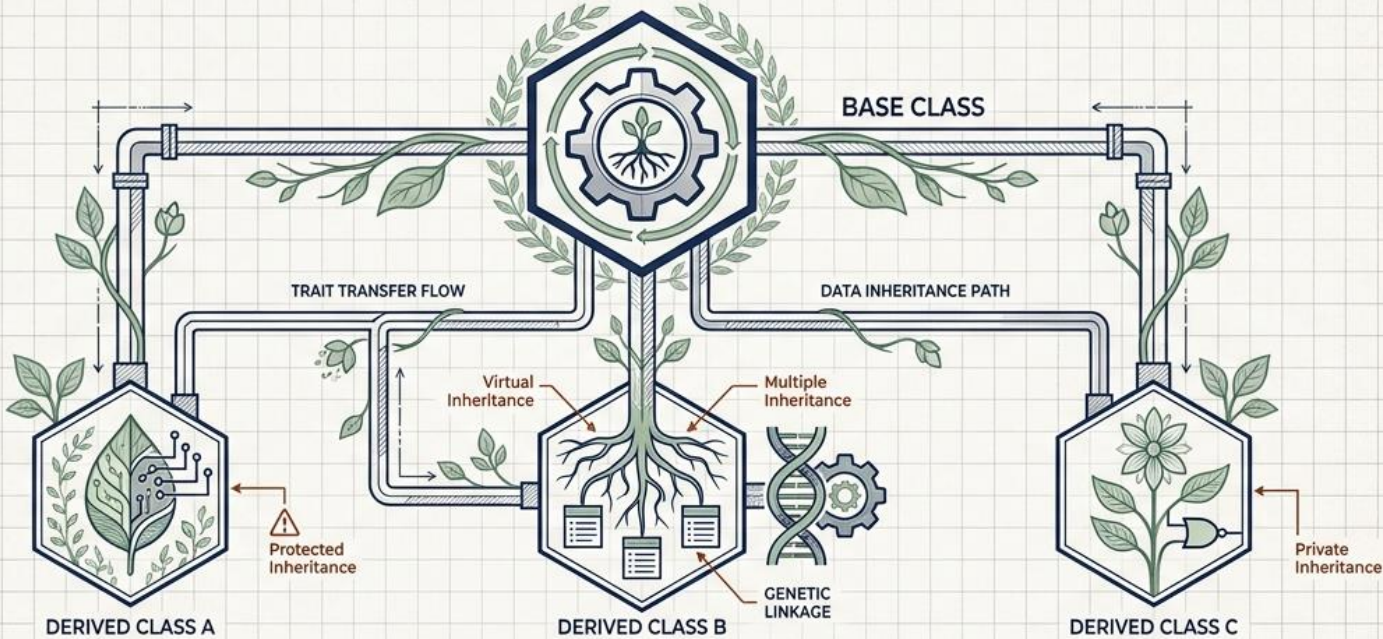
Q: What is runtime polymorphism? How is it achieved using virtual functions? Explain pure virtual functions and abstract classes. [PU Exam 2023]

Inheritance

- Base and Derived Classes
- Single, Multiple, Multilevel, Hierarchical, Hybrid
- Private, Protected, Public Inheritance
- Function Overriding
- Virtual Base Class — solving the Diamond Problem
- Constructor/Destructor in Derived Classes

The Blueprint of Code: Mastering C++ Inheritance

From Object-Oriented Foundations to Advanced Digital Genetics



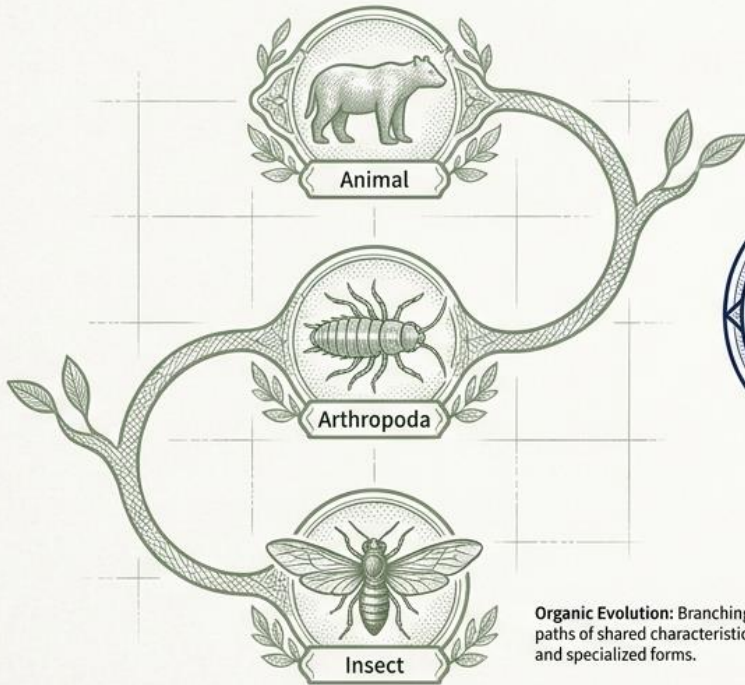
```
class Base {
public:
    int rootData;
};
class Derived : public Base {
public:
    void branchOut();
};
```

PROJECT: C++ INHERITANCE BLUEPRINT	
DRAWN BY: ARCHITECTURAL GENETICS LAB	
DATE: OCT 26, 2024	SHEET NO: 01 OF 01

The Core Concept: Digital Genetics

Inheritance represents an "Is-A" relationship. Just as an insect is an arthropod, a Derived (Child) class acts as a genetic template that absorbs properties of a Base (Parent) class to build specialized objects without rewriting DNA.

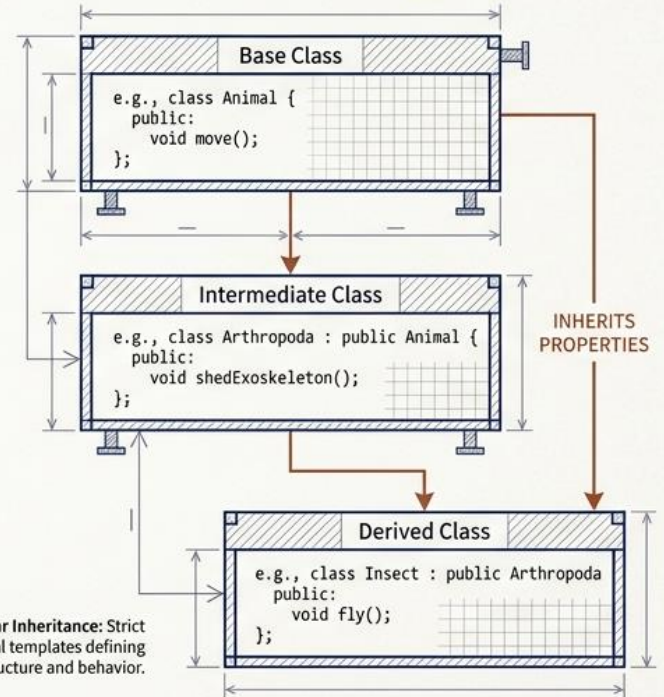
BIOLOGICAL HIERARCHY



Organic Evolution: Branching paths of shared characteristics and specialized forms.



DIGITAL BLUEPRINT (C++)



Modular Inheritance: Strict architectural templates defining object structure and behavior.

Inheritance

Foundations of Inheritance: *Concept and Real-World Modeling*

Inheritance stands as a cornerstone of Object-Oriented Programming (OOP), serving as a sophisticated mechanism that allows us to model software after the hierarchical structures found in the real world. By creating a system where new classes can be built upon existing ones, architects develop software that is naturally extensible, logically organized, and representative of complex relationships.

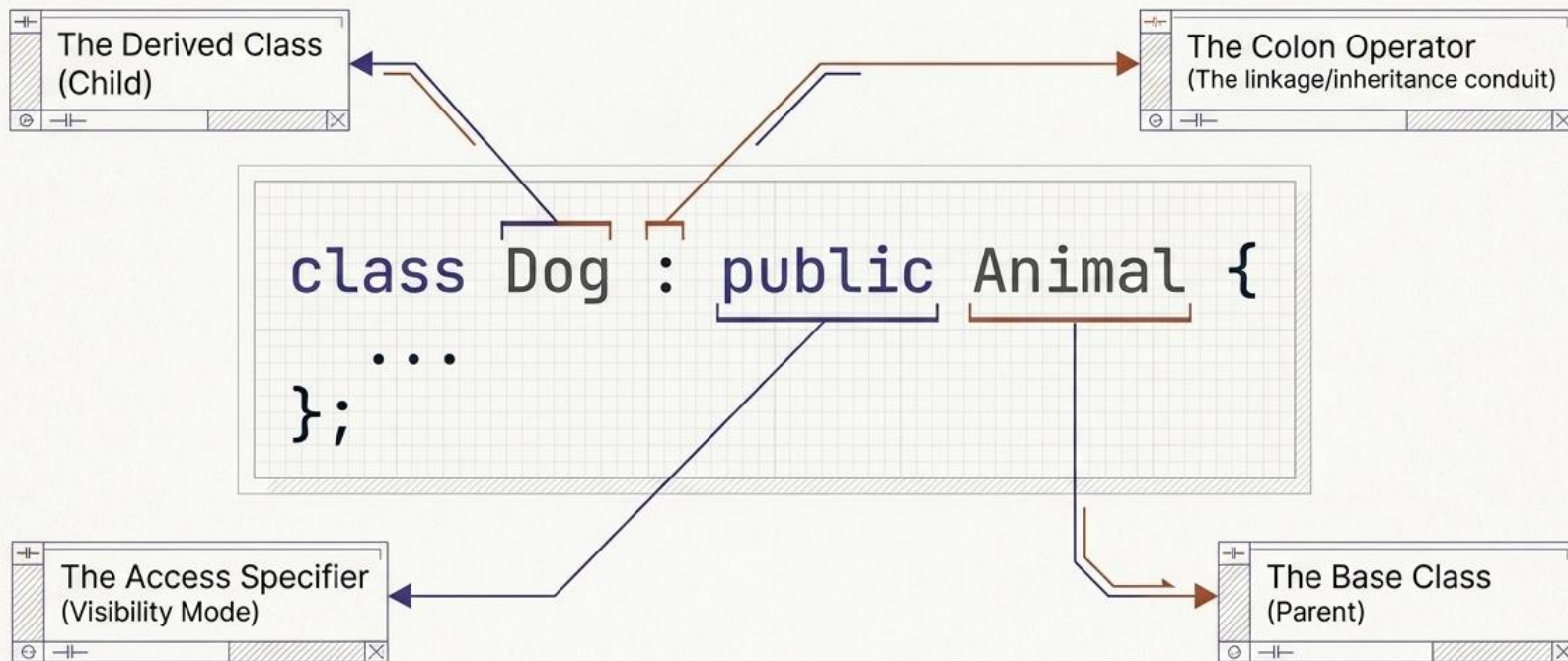
What is Inheritance?

Inheritance is the process by which a "derived" class acquires the properties, behaviors, and characteristics of an existing "base" class. This architectural choice shifts the focus from building isolated components to defining relationships between entities.

Inheritance in C++ language is a core concept of object-oriented programming (OOP) that allows you to create new classes (derived classes or child classes) based on existing classes (base classes or superclasses). It enables a subclass to inherit all the properties of the base class and add some new features to itself.

The Anatomy of Inheritance

The child absorbs the parent's attributes while opening a block `{ }` to inject its own specialized data and methods.



Inheritance

What Are Child And Parent Classes?

Inheritance introduces the concepts of the child (derived) and parent (base) classes. That is:

- **child class** (derived) - the class that inherits from another class
- **parent class** (base) - the class being inherited from

A **child class** inherits properties and behaviors from a **parent class**, allowing you to create specialized classes while reusing common attributes and functionalities. Think of it as a family tree where the characteristics are passed down from parents to children. In the previously explained example, the *Dog*, *Cat*, and *Tiger* class (child) inherits the attributes and methods from the *Animal* class (parent).

The Is-A relationship

Inheritance in C++ represents an is-a relationship between the child and parent classes. This means that an object of a child class is also an object of the parent class. For instance, if the Car class is derived from the Vehicle class, you can say that a car **is-a** vehicle, or if Cat is derived from Animal, you can also say that a cat **is-a(an)** animal.

Inheritance

Syntax And Structure Of Inheritance In C++

```
class ParentClassName {  
    // parent class definition  
};  
class ChildClassName : visibility_mode ParentClassName {  
    // child class definition  
};
```

Here,

- **class:** It is the keyword for creating a new class.
- **ParentClassName:** It is the name of the base class or parent class.
- **ChildClassName:** It is the name of the derived class or child class.
- **Visibility_mode:** It is the access specifier that determines how inherited members are visible in the derived class (public, protected, or private).

Inheritance

Implementing Inheritance In C++

```
class <derived_class_name> : <access_specifier> <base_class_name>
{
// body of the child class - class definition
}
```

Here,

- The **class** keyword indicates that we are creating a new class and the **colon symbol (:)** is the indicator of inheritance in C++.
- **base_class_name** is the name of the base class.
- **derived_class_name** is the name of the class that will inherit the properties of a base class.
- **access_specifier** defines the visibility mode through which the derived class has been created (public, private, or protected mode).

Inheritance

```
#include <iostream>
using namespace std;

// parent class
class Animal {
public:
void speak() {
cout << "Animal makes a sound" << endl;}
};

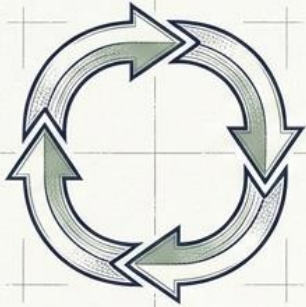
// child class
class Dog : public Animal {
public:
void speak() {
cout << "Dog barks" << endl;}
};

int main() {
    Animal animal;
    Dog dog;
    animal.speak();
    dog.speak();

    return 0;
}
```

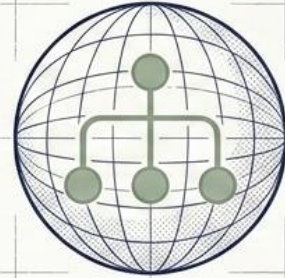
- In this simple C++ program, a class *Dog* is derived from the class *Animal*, and the *speak()* function is overridden in *Dog* to provide a specialized behaviour.
- We begin by including the *<iostream>* header file for input/ output operations and namespace *std* to use its classes without calling it.
- As mentioned in the code comments, we create a parent or base class *Animal* and define the *speak()* function, which prints a statement using the *cout* command. The use of a public access modifier makes the function publicly accessible.
- Then, we create another class, *Dog*, which inherits the *Animal* class with the public access modifier. This is the child class.
- Inside the *Dog* class, we use the public access modifier to create the *speak()* function that overrides the function from the base class.
- In the *main()* function, we create two objects, *animal* and *dog*, of the *Animal* and *Dog* classes, respectively.
- We then use the dot operator (*>*) and class objects to call the *speak()* methods from both classes.
- The first call invokes the Base class function, and the second call invokes the method from the derived class.
- The program finally terminates with a *return 0* statement indicating successful execution without any errors

The Four Pillars of Inheritance



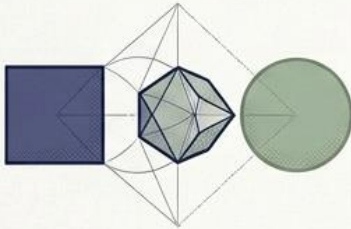
Code Reusability

Eliminates redundancy. Write a method once, inherit it endlessly.



Conceptual Modeling

Mirrors real-world hierarchies naturally (e.g., a Dog is an Animal).



Polymorphism

Enables treating specialized child objects as their generic parents.



Maintainability

Updating the base blueprint automatically propagates updates to all children.

Inheritance

Importance Of Inheritance In C++

- **Polymorphic Containers:** Inheritance in C++, combined with polymorphism, enables you to create containers (e.g., arrays or collections) that can hold objects of different derived classes but still operate on them using common base class interfaces. This is valuable for creating data structures and algorithms that work with heterogeneous data.
- **Method Overriding:** Inheritance in C++ allows derived classes to provide their own implementations of base class member functions through a mechanism called method overriding. This is particularly useful when you want to customize the behaviour of a derived class while maintaining a consistent interface defined in the base class.
- **Conceptual Modeling:** Inheritance in C++ allows you to model real-world relationships between objects. For example, in a system for modelling animals, you can have a base class, Animal, and derived classes like Dog, Cat, and Bird, which accurately represent the inheritance hierarchy in the real world.

Inheritance

Importance Of Inheritance In C++

- **Encapsulation:** Inheritance in C++ supports the principle of encapsulation, which involves bundling data and methods into a single unit (class). By inheriting from base classes, you can access and manipulate the data and methods of the base class within the derived class, following the access control rules defined in C++.
- **Behavioural Classification:** Inheritance in C++ allows you to classify objects based on their behaviours or characteristics. For example, you can have a base class Vehicle and derived classes like Car and Motorcycle, which share common characteristics and behaviours specific to vehicles.
- **Template for Design Patterns:** Many design patterns, such as the Factory, Singleton, and Strategy patterns, rely on inheritance to implement their solutions. Inheritance in C++ provides a foundation for creating and using these design patterns effectively.
- **Better Code Organization:** The concept of inheritance in C++ helps in organizing and structuring your code in a more logical and systematic manner, making it easier to understand and navigate.
- **Framework and Library Design:** Inheritance in C++ programs is crucial when designing frameworks and libraries. Frameworks often provide base classes with defined behaviours, allowing users to create derived classes tailored to their specific requirements. This approach encourages standardization and reuse in software development.

Rules of Transmission: The Visibility Matrix

Visibility only shrinks, it never expands. Protected acts as a VIP key—keeping data hidden from the outside world, but fully accessible to derived classes.

Derivation Mode:				
Base Class Member:		Public Mode	Protected Mode	Private Mode
Public Member		Public	Protected	Private
Protected Member		Protected	Protected	Private
Private Member		Not Inherited	Not Inherited	Not Inherited



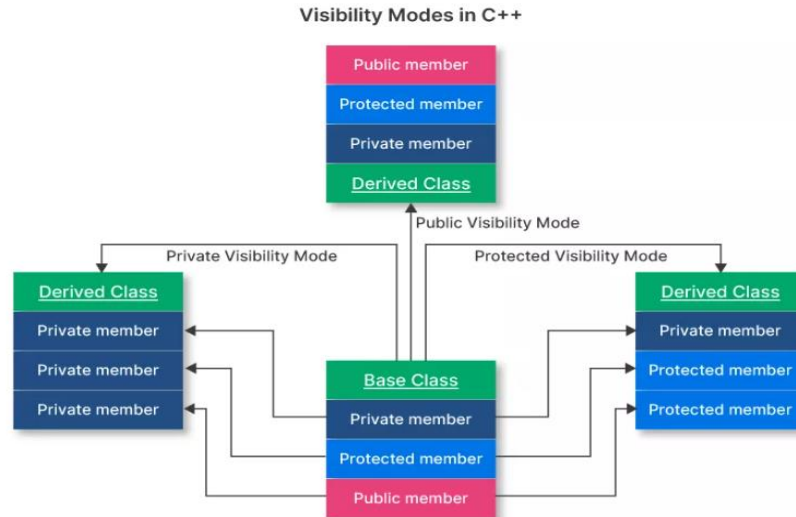
CRITICAL: Private base members are NEVER directly inheritable by a child class.

Inheritance

Visibility Modes Of Inheritance In C++

Visibility modes control the accessibility of inherited members in the derived class. In other words, the visibility mode explains how the inherited members of the base class will be accessible to the derived class.

There are three access modes, i.e., public visibility mode, protected visibility mode, and private visibility mode. In this section, we will discuss these types of visibility modes in detail.



Inheritance

Public Visibility Mode Of Inheritance In C++

Public visibility mode is when we derive a class from a parent class with the public keyword. This is referred to as public inheritance, and all the members retain their visibility in the derived class. That is, when we derive a class in public mode, the members of the base class that are public remain public in the derived class, members that are protected in the parent class remain protected in the derived class, and private members are inaccessible.

Syntax

```
class DerivedClass : public BaseClass {  
    // Body of the class  
}
```

Here,

- The class keyword is used for creating a new class.
- BaseClass and DerivedClass are the names of the base/ parent and the derived/ child class, respectively.
- The public keyword/ access specifier makes all the members inherited from the BaseClass public only if the members being inherited are public in the BaseClass.

Inheritance

```
#include <iostream>
using namespace std;

// parent class
class Animal {
public:
void speak() {
cout << "This is an animal.\n";
};

// child class
class Dog : public Animal {
};

int main() {
Dog dog;
dog.speak();
return 0;
}
```

Explanation:

In this example for public visibility mode-

- *We create a class Animal after including the necessary header files and namespace.*
- *Inside the Animal class, we define the speak() function, which prints a phrase using the cout command.*
- *When defining the function, we use the public access modifier to ensure that it is publicly accessible to all derived classes from anywhere in the program.*
- *Next, we create a class, Dog, which is derived from the Animal class, in public mode, which means objects of the Dog class can access the public members of the base class.*
- *Inside the main() function, we create an object dog of the Dog class and use the dot operator to call the speak() function on the object.*
- *This invokes the speak() function from the base class, and the output is printed to the console.*

Inheritance

Protected Visibility Mode Of Inheritance In C++

The protected mode refers to the situation where we derive a class from a base class using the protected keyword. As a result of this, the protected and public members of the base class become protected in the derived class, and private members remain inaccessible.

And these protected members can only be accessed by the derived class and its member functions. Also, any subclass can inherit these protected members, with the exception of private members.

Syntax

```
class DerivedClass : protected BaseClass {  
    // body of the class  
}
```

Here,

- The **class** keyword, **BaseClass**, and **DerivedClass** all remain the same from the syntax for private visibility mode.
- The access specifier **protected** here indicates that the derived class has protected inheritance. That is, all its members are protected only if inherited members are public or protected in BaseClass.

Inheritance

```
#include <iostream>
using namespace std;

// parent class
class Animal {
public:
void speak() {
cout << "This is an animal.\n";
};

// child class
class Dog : protected Animal {
public:
void display() {
speak();}
};

int main() {
Dog dog;
dog.display();
return 0;
}
```

Explanation:

In this example for public visibility mode-

- We create a **parent class *Animal*** with a public **member function *speak()***, which prints a phrase using the ***cout command***.
- Next, we create a **derived class, *Dog***, using the ***protected*** keyword/ access modifier. This means that only the Dog class member functions can access the member functions of the Animal class, that too from inside the derived class only.
- We then define a ***display()* function** inside Dog, which further calls on the *speak()* function from the base class.
- Inside the ***main()* function**, we create an **object *dog*** of the Dog class and **call the *display method*** using the dot operator.
- This invokes the *speak()* method inherited from the Animal class, and the result is shown in the output window.

Inheritance

Private Visibility Mode Of Inheritance In C++

The private mode of visibility also referred to as private inheritance, is when we derive a class with the use of the public keyword. In this case, all the members of the parent class (i.e., public, protected, or private) become private when inherited by the child class.

As a result, these members are only accessible inside the derived class or their access is restricted outside of the derived class. Also, only the member functions or friend functions of the derived class can access these private members.

Syntax

```
class DerivedClass : private BaseClass {  
    // body of the class  
}
```

Here,

- A major part of the syntax is the same as in the case of both the private and public visibility modes. The only difference is the access modifier and its implications.
- The **private access specifier** implies that all the inherited members from the base class turn private for the derived class.

Inheritance

```
#include <iostream>
using namespace std;

class Animal {
public:
    void speak() {
        cout << "This is an animal.\n";
    };

    class Dog : private Animal {
    public:
        void display() {
            speak();
        };

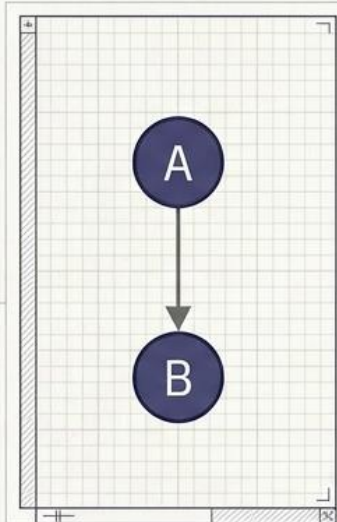
        int main() {
            Dog obj;
            obj.display();
            return 0;
        }
    };
};
```

Explanation:

In this example for public visibility mode-

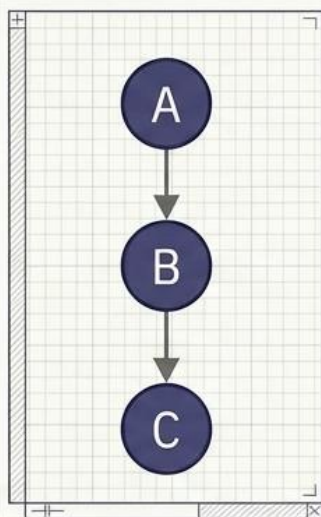
- We first create a **parent class** called **Animal**, which contains a **public member function speak()** that prints a phrase using the **cout command**. and This means that the function is publicly accessible to the derived class.
- Next, we create a **derived class, Dog**, which inherits the Animal class with the **private** access modifier. This means that the member functions of the Animal class are only accessible inside the Dog class, not by its objects from outside of the class.
- The Dog class also contains a public member function **display()**, which contains the **speak()** function in the code body.
- Inside the **main() function**, we then create an **object dog** of the Dog class and use the dot operator to **call the display() method**.
- As mentioned, we can't use the object to call the parent class member function directly since the derived class is in private mode.
- So, we use the object to call the **display()** function, which further invokes the **speak method** from the Animal class.
- The result is then printed on the output console before the program terminates with a return 0.

Topological Mapping: The 5 Archetypes



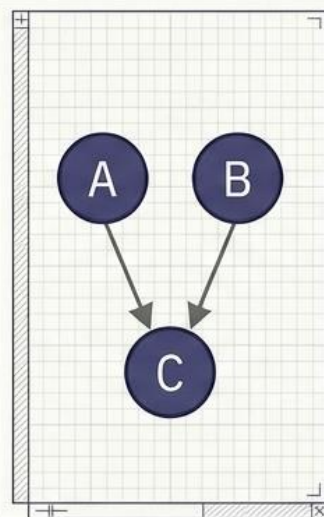
Single

One parent,
one child.



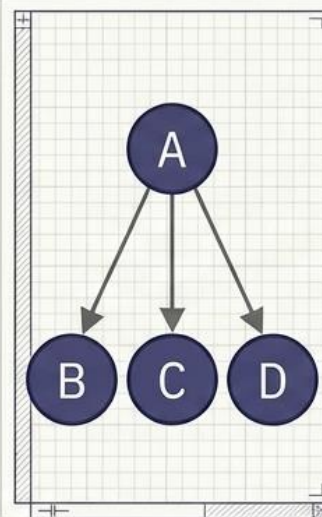
Multilevel

A linear chain of
inheritance.



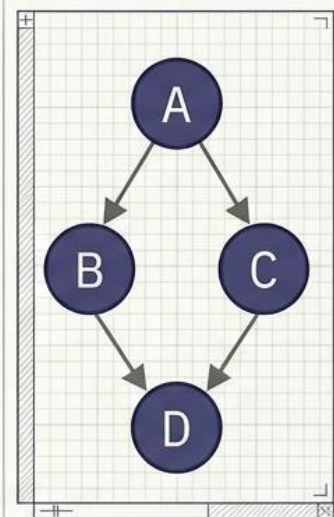
Multiple

Two parents
converging into
one child.



Hierarchical

One parent diverging
to categorical families.



Hybrid

A complex network
combining topologies.

Linear Evolution: Single & Multilevel Paths

Inheritance is transitive. If Aeroplane inherits FlyingObject, and FlyingObject inherits Vehicle, the Aeroplane possesses the genetic traits of a Vehicle by default.

Single Inheritance



```
class Vehicle { ... };  
class Car : public Vehicle { ... };
```

Multilevel Inheritance



```
class Vehicle { ... };  
class FlyingObject : public Vehicle { ... };  
class Aeroplane : public FlyingObject { ... };
```

Inheritance

Single Inheritance

- A class inherits only from one other class.
- The derived class can reuse or extend the parent's functionality.

Syntax

```
class A {  
    //Base Class  
};  
  
class B: access_specifier A {  
    //Derived Class  
};
```

Here,

- `visibility_mode`: It provides the visibility mode in the class declaration since it specifies how the members of the base class will be inherited by the derived class.
- `base_class`: This is the name of the class from which the derived class inherits its properties and characteristics.
- `derived_class`: This is the name of the derived class that can inherit the properties of the base class using the visibility mode.

Inheritance

```
#include <iostream>
using namespace std; //using standard namespace

class Account {    //Base Class
public:    //Access Modifier
    float salary = 65000;
};

    class Programmer: public Account {    //Derived Class
public:
    float bonus = 5500;
};

int main(void) {    //Main Function
    Programmer p1;
    cout<<"Salary: "<<p1.salary<<endl;
    cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}
```

Inheritance

Multilevel Inheritance

- A Chain of inheritance where a class can inherit from a derived class.

Syntax

```
class A {  
    //Base Class  
};  
  
class B: access_specifier A {  
    //Derived Class  
};  
  
class C: access_specifier B {  
    //Derived Class  
};
```

Inheritance

```
#include <iostream>

class LivingBeing {
public:
    void breathe() { cout << "Breathing...\n"; }
};

class Animal : public LivingBeing {
public:
    void move() { cout << "Moving...\n"; }
};

class Dog : public Animal {
public:
    void bark() { cout << "Barking...\n"; }
};
```

```
#include <iostream>

class Grandparent{
public:
    void car(){
        cout << "Grandparent owns a house"<< id << endl;
    }
};

class Parent : protected Grandparent{
public:
    void house(){
        cout << "Parent owns a car"<< id << endl;
    }
};

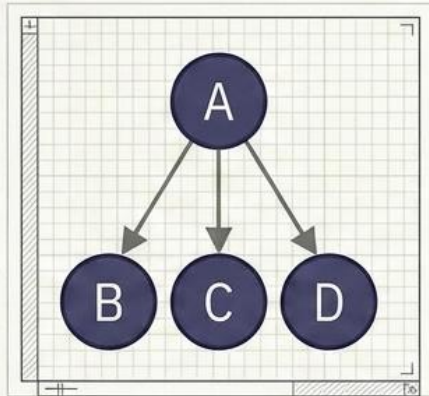
class Child : protected Parent{
public:
    void laptop(){
        cout << "Child owns a Laptop"<< id << endl;
    }
};

int main(){
    Child c;
    c.laptop();
    return 0;
}
```

Branching Out: Hierarchical & Multiple Inheritance

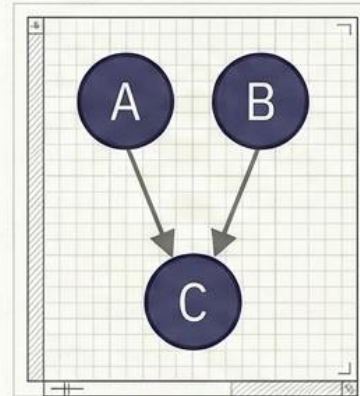
Hierarchical creates categorical families. Multiple fuses distinct functional blueprints into a single, multi-capable object.

Hierarchical Inheritance



```
class Fruit { void display(); };  
class Mango : public Fruit { ... };  
class Durian : public Fruit { ... };
```

Multiple Inheritance



```
class Animal { ... };  
class Aerial { ... };  
class Bird : public Animal, public Aerial { ... };
```

Fuses multiple
ancestral blueprints.

Inheritance

Hierarchical Inheritance

Hierarchical Inheritance is defined as the process of deriving multiple classes from a single base class. In this inheritance, we can include all features in the base class that are common in child classes.

Syntax

```
class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}
```

Inheritance

```
#include <iostream>
using namespace std;
class Shape {
public:
    int a;
    int b;
    void get_data(int n,int m){
        a= n;
        b = m;
    }
};
class Rectangle : public Shape{
public:
    int rect_area(){
        int result = a*b;
        return result;
    }
};
class Triangle : public Shape{
public:
    float triangle_area()
    {
        float result = 0.5*a*b;
        return result;
    }
};

int main() {
    Rectangle r;
    Triangle t;
    int length,breadth,base,height;
    std::cout << "Enter the length and breadth of a rectangle: " << std::endl;

    cin>>length>>breadth;
    r.get_data(length,breadth);
    int m = r.rect_area();
    std::cout << "Area of the rectangle is : " <<m<< std::endl;
    std::cout << "Enter the base and height of the triangle: " << std::endl;

    cin>>base>>height;
    t.get_data(base,height);
    float n = t.triangle_area();
    std::cout <<"Area of the triangle is : " << n<<std::endl;
    return 0;
}
```

Inheritance

Multiple Inheritance

- A class inherits from more than once base classes. Meaning a derived class have more than one parents.

Syntax

```
class A {  
    //Base Class  
};  
  
class B{  
    //Base Class  
};  
  
class C: access_specifier B, access_specifier B {  
    //Derived from A and B  
};
```

Inheritance

```
#include <iostream>

class Printer {
public:
    void print() { cout << "Printing...\n"; }
};

class Scanner {
public:
    void scan() { cout << "Scanning...\n"; }
};

class AllInOne : public Printer, public Scanner
{
public:
    void doEverything() {
        print();    // From Printer
        scan();     // From Scanner
    }
};
```


Inheritance

Hybrid Inheritance

In C++, Hybrid Inheritance is a combination of more than one type of inheritance. In this inheritance, we can contain elements of single inheritance, multiple inheritance, multilevel, and hierarchical inheritance in the class.

Syntax

```
class A {  
class Base  
{  
    // base class members and statements  
};  
class Intermediate1 : public Base  
{  
    // intermediate class 1 members and statements  
};  
class Intermediate2 : public Base  
{  
    // Body of intermediate class 2 members  
};  
class Derived : public Intermediate1, public Intermediate2  
{  
    // derived class members and statements  
};
```

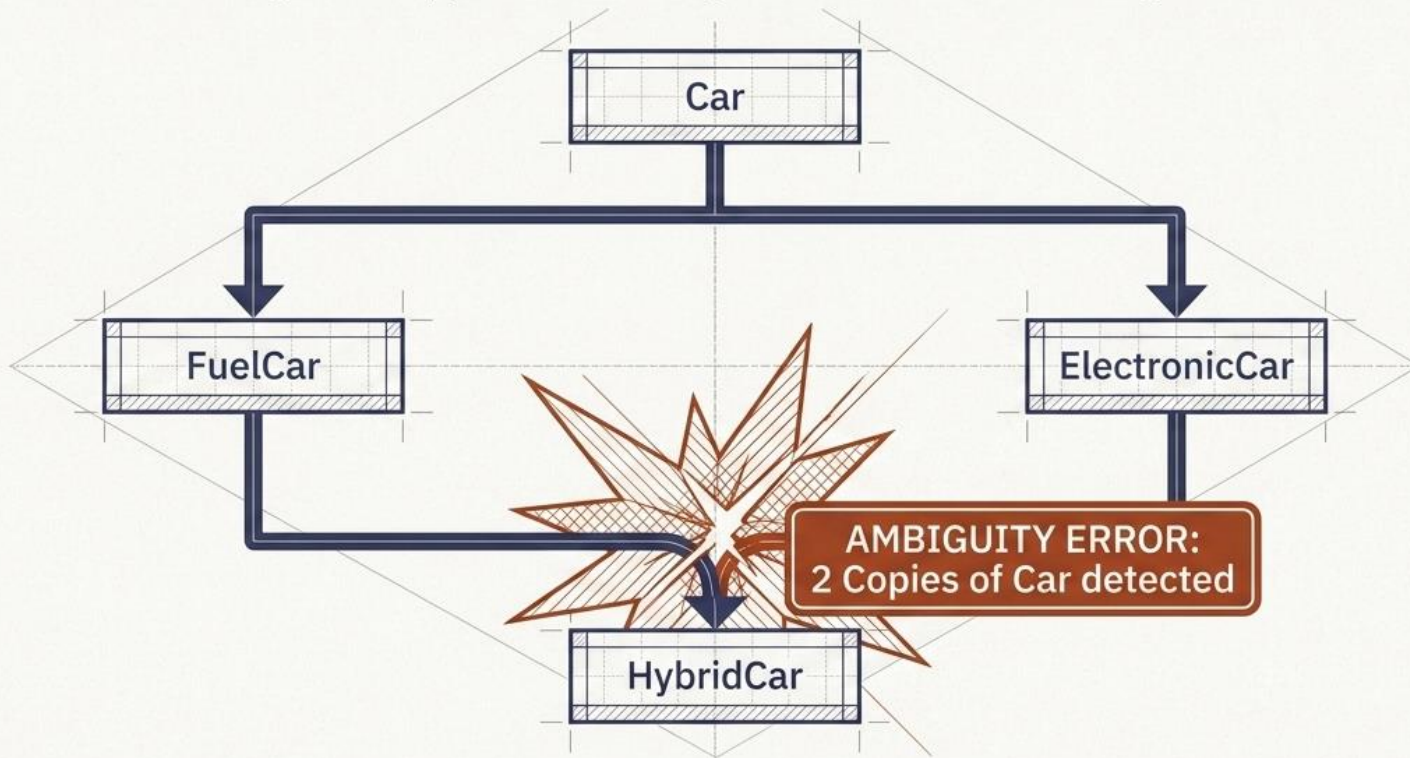
Inheritance

```
#include <iostream>
using namespace std; //using standard namespace
class A { //Base class
protected:
    int a;
public:
    void get_a()
    {
        cout << "Enter the value of 'a' : " << endl;
        cin>>a;}
};
class B : public A{ //class B derived from Class A
protected:
    int b;
public:
    void get_b(){
        cout << "Enter the value of 'b' : " << endl;
        cin>>b;}
};
class C{
protected:
    int c;
public:
    void get_c(){
        cout << "Enter the value of 'c' : " << endl;
        cin>>c;}
};
```

```
//class D derived from both class B and Class C
class D : public B, public C{
protected:
    int d;
public:
    void mul(){
        get_a();
        get_b();
        get_c();
        cout << "Multiplication of a, b, c is : " <<a*b
*c<< endl;
    }
};
int main(){
    D d;
    d.mul();
    return 0;
}
```

The Structural Crisis: Multipath Ambiguity

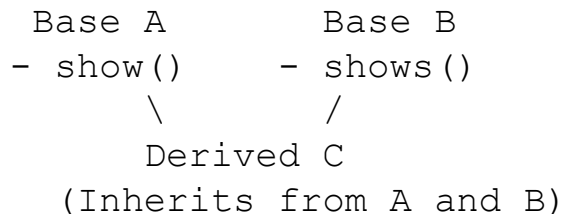
The 'Diamond Problem.' If HybridCar tries to access a base property from Car, the compiler freezes:
Should it use the genetic copy inherited through **FuelCar**, or the one through **ElectronicCar**?



Inheritance – Ambiguity

Multipath Inheritance

Multipath inheritance occurs when a derived class inherits from multiple base classes, and those base classes share a common ancestor. This creates multiple paths to the same base class, leading to ambiguity.



Inheritance – Multipath Ambiguity

Ambiguity Resolution in Multiple Inheritance

To resolve this ambiguity due to multiple inheritance in C++ classes, we must explicitly specify which base class's member function or data member we want to use in the derived class. There are two common ways to handle this:

- Using the scope resolution operator (::)
- Overriding the member function in the derived class.

Scope Resolution Operator (::) To Resolve Ambiguity Of Multiple Inheritance In C++

In this method, we can explicitly specify the base class from which we want to access the member by using the scope resolution operator along with the base class's name.

Syntax

```
class A {base_class::member_name
```

Here,

- **base_class**: The name of the base class from which you want to access a member.
- **member_name**: The name of the member (function or data) that could lead to ambiguity.

Inheritance – Multipath Ambiguity

```
#include <iostream>
using namespace std;

class Teacher {
public:
    void teach() {
        cout << "Teaching general subject \n";}
};

class Musician {
public:
    void teach() {
        cout << "Teaching music\n";}
};

class MusicTeacher : public Teacher, public
Musician {
public:
    // No specific teach() function defined in
    MusicTeacher
};

int main() {
    MusicTeacher myTeacher;
    myteacher.teach();
    return 0;
}
```

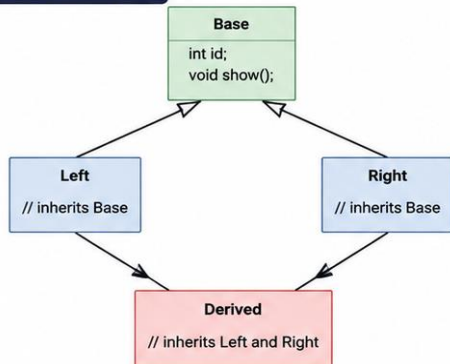
Explanation:

- We create a **base class**, **Teacher**, with a public **teach()** member function, which prints a message using `std::cout` when called.
- Next, we create another **base class**, **Musician**, containing a public **teach()** method that prints a different message when called.
- Then, we define a **derived class** **MusicTeacher**, which publicly inherits from both **Teacher** and **Musician**.
- This means that the public members of both base classes will be accessible in the derived class. This leads to **ambiguity due to inherited member functions**.
- Since **MusicTeacher** class does not have its own **teach()** function, it inherits two functions with the same name. One from the **Teacher** class, which teaches general subjects, and another from the **Musician** class, which teaches music.
- Now, inside the **main()** function, we create an instance **myTeacher**, of the **MusicTeacher** class.
- We then use this **object to call teach()** function. This leads to an **ambiguity error**, because the **compiler cannot decide** which **teach()** method to call—the one from the **Teacher** class or the one from the **Musician** class.
- Since there is no clear instruction on which method to use, the compiler **throws an error**.

The Diamond Problem in Multiple Inheritance (C++)

Scenario: When a class inherits from two classes that both inherit from the same base class, the derived class ends up with **two copies** of the base class members.

Class Hierarchy



```
class Base {
public:
    int id;
    void show() { cout << "Base::show()" << endl; }
};

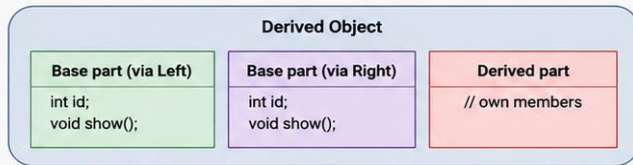
class Left : public Base { };
class Right : public Base { };

class Derived : public Left, public Right {
    // Inherits from both Left and Right
};

int main() {
    Derived d;
    // d.id;           // Error: Ambiguous!
    // d.show();       // Error: Ambiguous!
    return 0;
}
```

What Happens?

Derived object memory layout



Data Duplication

Two copies of Base members (id, show)



Ambiguity

Which 'id' or 'show()' to access?



Conflicting Behavior

Which version to use if behavior differs?

Why is it a Problem?

Problem	Explanation	Example in Code
Data Duplication	Two copies of base class members in Derived object.	d will have two 'id' and two 'show()'.
Ambiguity	Compiler doesn't know which base class member to use.	d.id; // Error d.show(); // Error
Conflicting Behavior	If base classes override methods differently.	Which 'show()' should be called?

The Core Problem:

When a class inherits from two classes that both inherit from the same base class, the derived class ends up with **two copies** of the base class members. This creates:

1.Data duplication (multiple copies of base class data)

2.Ambiguity (which copy to access?)

3.Conflicting behavior (which version to use?)



Solution: Use Virtual Inheritance to share a single copy of the base class.

```
class Left : virtual public Base { };    class Right : virtual public Base { };
```



Now, Derived has only one copy of Base!



Inheritance – Diamond Problem

The **diamond problem** occurs with multiple inheritance in C++ classes when a class inherits from two or more classes that share a common base class. The ambiguity arises because the derived class ends up with *multiple instances of the common base class*, leading to conflicts when accessing base class members.

Real-world example of the diamond problem with multiple inheritance in C++:

- **Super Base Class:** Say you have a *base class, Person*, representing a Person being with various attributes.
- **First Inheritance/ Base Class:** We derive *two classes, Teacher* (representing individuals) and *Researcher* (representing expert), both *inherited from Person*.
- **Second Inheritance/ Derived Class:** Then, we create a *Professor class* that *inherits from both Teacher and Researcher*.

Inheritance – Diamond Problem

Scenario: University System with Conflicting Roles

```
Person (Base Class)
  /      \
Teacher  Researcher
  \      /
Professor (Derived Class)
```

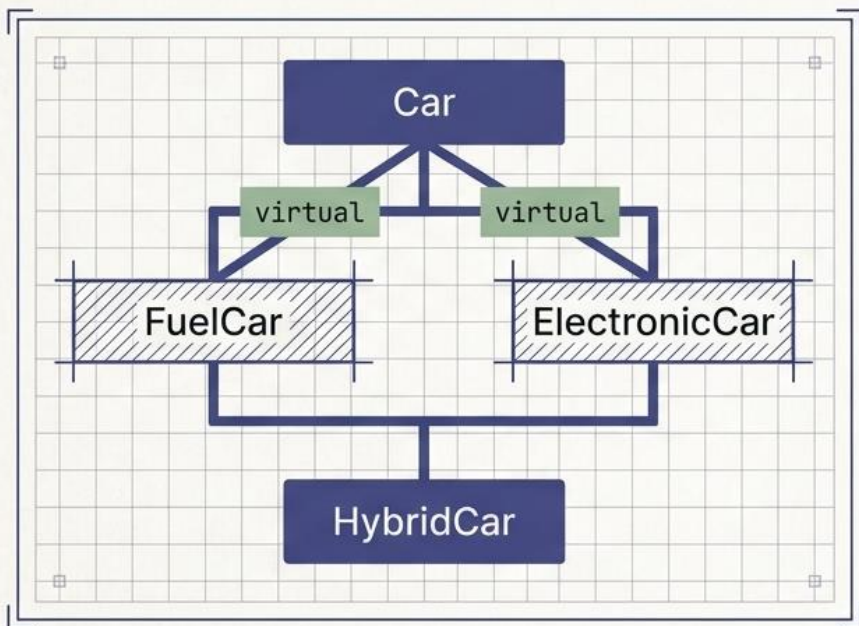
Context:

- A Person has a unique ID and can introduce themselves
- A Teacher teaches courses and has a staff ID
- A Researcher conducts research and has a project list
- A Professor inherits from both Teacher and Researcher

***Note:** The diamond problem differs from the general ambiguity problem with multiple inheritance. The diamond problem involves both **multiple** and **multi-level inheritance**, whereas ambiguity can arise in simple multiple inheritance in C++ classes.*

The Fix: Virtual Base Classes & Scope Resolution

Virtual Inheritance guarantees the compiler passes only ONE shared copy of the ancestor class. Scope Resolution manually forces a specific path.



Solution 1: Virtual Inheritance

```
class FuelCar : virtual public Car
class ElectronicCar : virtual public Car
```

Solution 2: Scope Resolution

```
obj.FuelCar::x
```

Manually resolves ambiguity
by specifying the path.

Inheritance – Virtual inheritance / Virtual Base Class

Virtual inheritance is a feature in C++ that ensures that only one instance of a common base class is shared among multiple derived classes in a diamond-shaped inheritance hierarchy. It eliminates the possibility of repetitive instances.

What is Virtual Base Class?

Virtual Class is defined by writing a keyword “virtual” in the derived classes, allowing only one copy of data to be copied to Class B and Class C. It prevents multiple instances of a class appearing as a parent class in the inheritance hierarchy when multiple inheritances are used.

Genetic Mutation: Method Overriding

Derived classes can provide their own specific implementations of base class functions. The base sets the rule, the child customizes the action.

Base Blueprint: Class Animal

```
class Animal {  
    virtual void speak() {  
        cout << 'Animal speaks';  
    }  
};
```

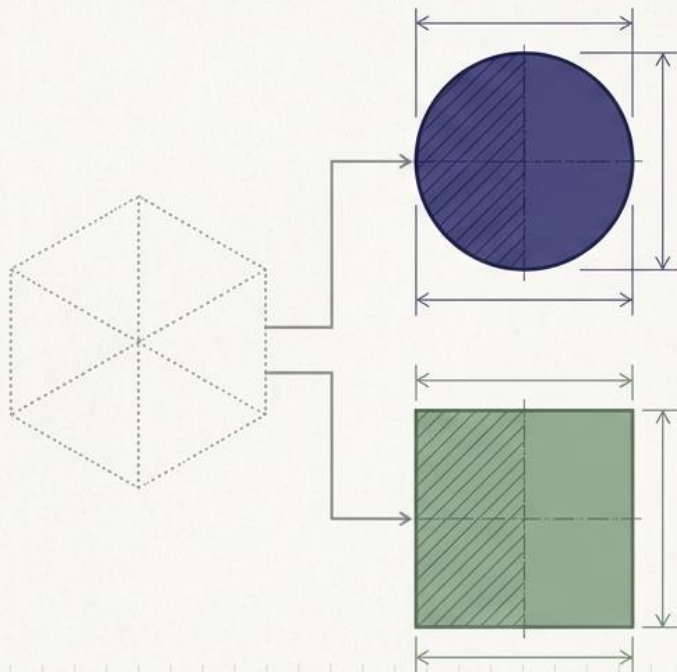


Derived Blueprint: Class Dog : Public Animal

```
class Dog : public Animal {  
    void speak() override {  
        cout << 'Dog barks';  
    }  
};
```

The Ultimate Template: Abstract Base Classes

An interface that enforces a strict architectural contract. It exists only to force derived classes to provide the implementation and cannot be instantiated.



```
class Shape {  
    virtual void draw() = 0;  
};
```

Pure Virtual Function:
The exact syntax that
makes a class
Abstract.

Abstract Base Class

An **Abstract Base Class (ABC)** is a class that **cannot be instantiated (no objects can be created from it)** because it contains **at least one pure virtual function**.

- It acts as a **blueprint** for other classes by defining what functions derived classes **must implement**.
- A class becomes abstract when it contains a **pure virtual function**, declared using `= 0`.

Syntax

```
virtual return_type function_name(parameters) = 0;
```

Real-Life Analogy

Imagine a company has an employee policy.

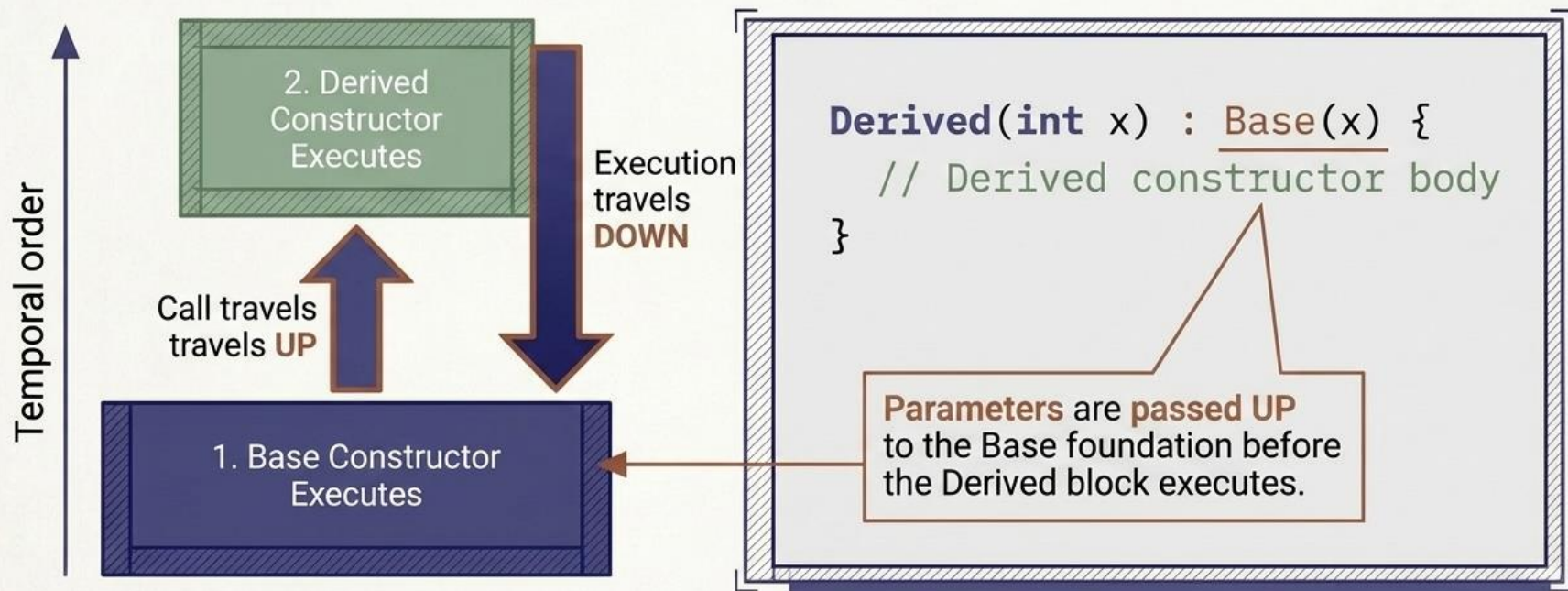
There are different types of employees:

- Manager
- Developer
- Accountant

Every employee **must work**, but the way each employee works is different.

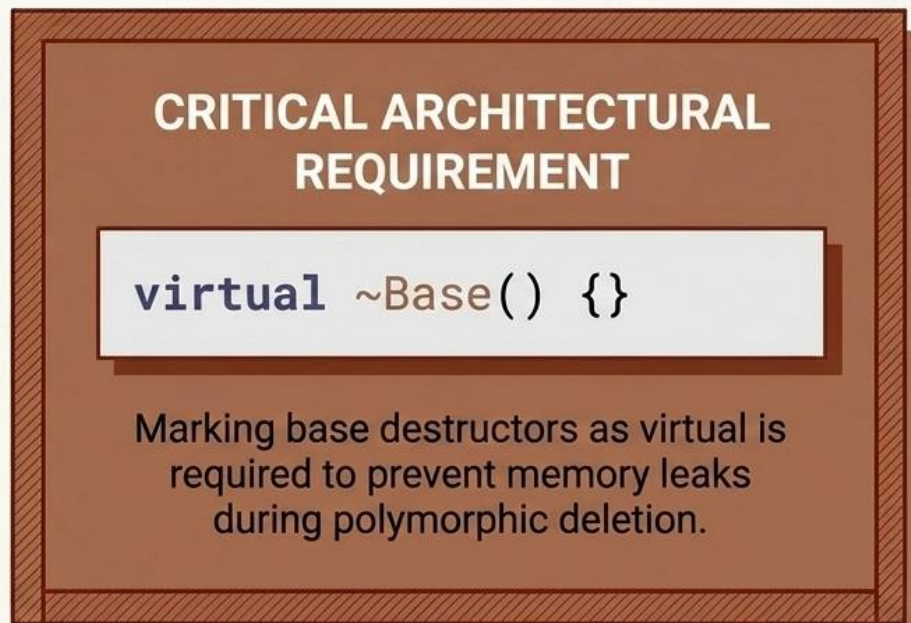
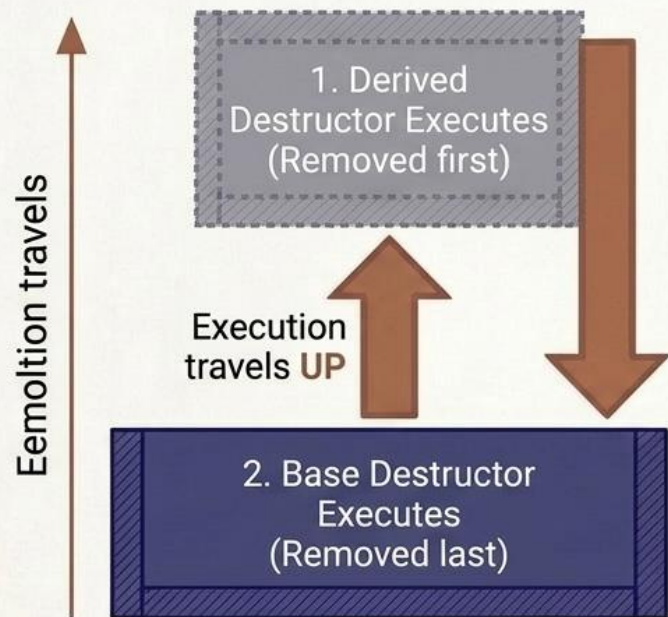
The Lifecycle Part 1: Constructors Build Up

Object creation travels UP the inheritance tree and executes DOWN the tree.
The foundation (Base) must be laid before the roof (Derived) can be built.



The Lifecycle Part 2: Destructors Tear Down

Strict reverse-order execution. The roof is dismantled before the foundation is destroyed to ensure memory safety.



Constructor and Destructor in Inheritance

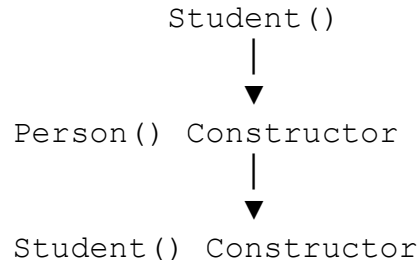
Constructor Execution Order

When an object of a derived class is created:

- 1. Base class constructor is called first.**
- 2. Derived class constructor is called next.**

Reason: The derived class depends on the base class, so the base part must be initialized before the derived part.

Create Student Object



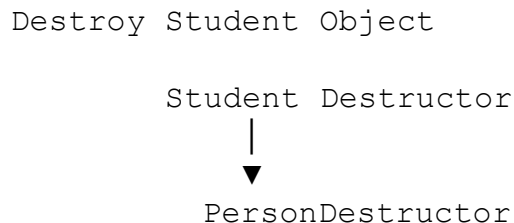
Constructor and Destructor in Inheritance

Destructor Execution Order

When an object destroyed:

- 1. Derived class destructor executes first.**
- 2. Base class destructor executes last.**

Reason: The derived class should clean up its own resources before the base class is destroyed.



Synthesis: The Architectural Decision Matrix

Mastering inheritance is about choosing the correct blueprint and visibility rules to mirror logic while ensuring code safety.

Architectural Goal	Topology to Use	Visibility Mode	Key Implication
Enforce a shared interface	Abstract Base Class	Public	⚠ Cannot be instantiated; children must override pure virtual functions.
Deep categorical specialization	Multilevel	Public	Traits are highly transitive across multiple generations.
Share data internally, hide externally	Single	Protected	Subclasses have access, but the outside world is locked out.
Fuse discrete distinct tools	Multiple	Public or Private	⚠ Risk of Diamond Problem; utilize 'virtual' inheritance if paths overlap.

5 Forms of Inheritance

- Single: One derived $\leftarrow$ One base (Dog : Animal)
- Multiple: One derived $\leftarrow$ Many bases (Duck : Flying, Swimming)
- Multilevel: Chain — C : B : A (GrandChild : Child : Parent)
- Hierarchical: Many derived $\leftarrow$ One base (Cat, Dog, Bird : Animal)
- Hybrid: Combination of two or more forms
- Diamond Problem:
 - When B and C both inherit A, and D inherits B and C $\rightarrow$ two copies of A
 - Solution: virtual public inheritance in B and C

```
// Single
class Dog : public Animal {};

// Multiple
class Duck: public Flying,
            public Swimming {};

// Virtual base (Diamond fix)
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {
    // Only ONE copy of A
};
```

Analogy & PU Question



Real-World Analogy

Inheritance is like a family tree. A child inherits traits from parents — eye color, height. But the child can also have unique traits or override inherited ones (choosing a different hairstyle). The diamond problem is like when two parents share the same grandparent — you'd only want ONE set of grandparent traits in the grandchild, not two duplicates.



Probable PU Exam Question

Q: What is the diamond problem in C++ inheritance? How is it solved using virtual base classes? Explain all 5 forms of inheritance with diagrams. [PU Exam 2021]

Advanced C++ Topics

- Function Templates and Class Templates
- Standard Template Library (STL) Introduction
- Namespaces — using directive vs using declaration
- Exception Handling: try, throw, catch
- Creating and Using Header Files

Agenda

01

Why Templates?

The copy-paste problem templates solve

02

Template Definition & Syntax

typename / class, instantiation

03

The Analogy

A tailor's paper pattern

04

Function Templates

2 full programs, traced output

05

Function Template Overloading

Rules + 1 full program

06

Class Templates

2 full programs: Box<T>, Pair<T1,T2>

07

Templates vs. Inheritance

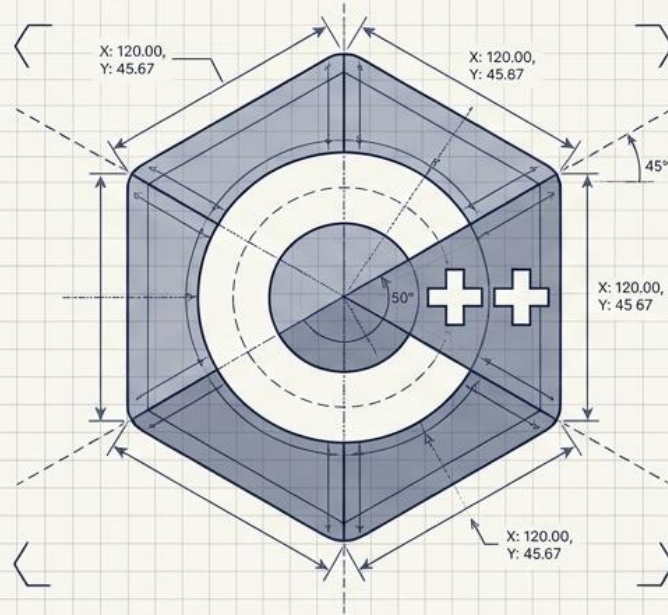
Two different reuse strategies

08

Standard Template Library

Containers, iterators, algorithms + 2 programs

Blueprinting Logic: The Architecture of C++ Templates



Mastering Compile-Time Code Generation and Generic Programming.

The Redundancy Problem in Typed Languages

```
void print(int value) {  
    cout << value;  
}
```

```
void print(float value) {  
    cout << value;  
}
```

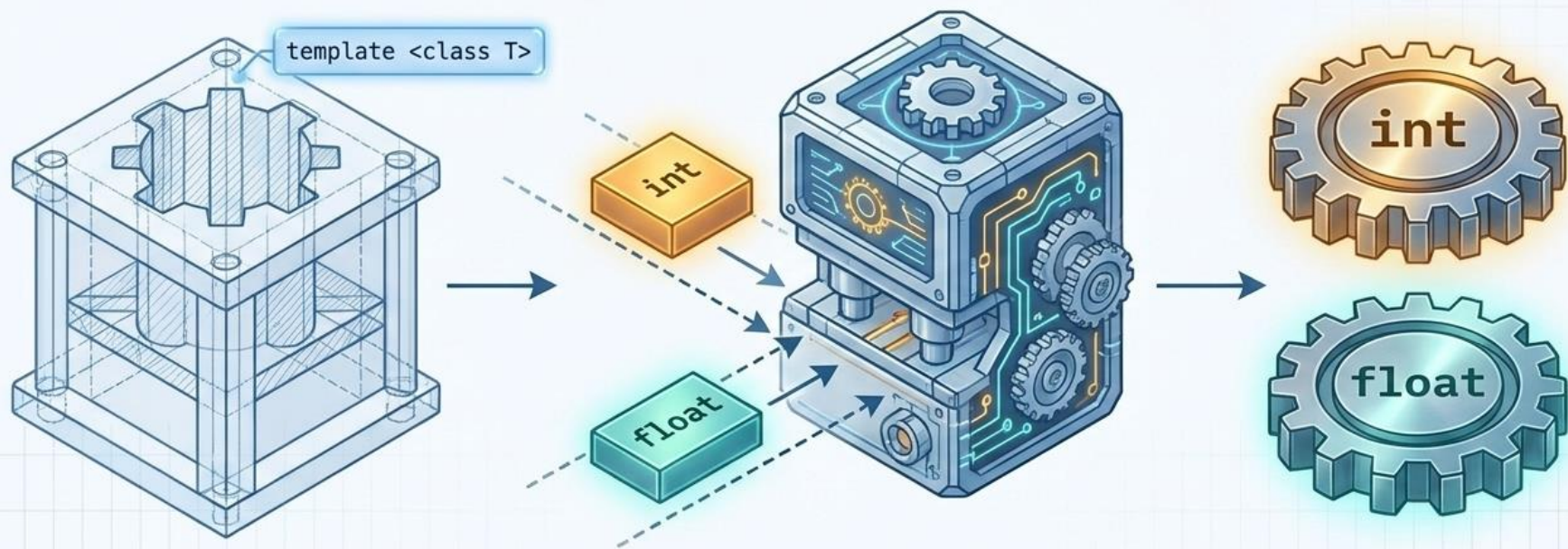
```
void print(string value) {  
    cout << value;  
}
```



Different Data Types = Duplicated Logic

Changing core logic requires updating multiple, hard-coded functions, leading to maintenance fragility.

Templates are Blueprints, Not Executable Code



A template is a set of instructions. The compiler uses this blueprint to automatically generate type-specific code exactly when, and only when, it is called.

Why Do We Need Templates?

Without templates, the same logic has to be retyped for every data type — more code, more bugs, more places to forget an update.

```
without_templates.cpp

int maxOf(int a, int b) {
    return (a > b) ? a : b;
}

double maxOf(double a, double b) {
    return (a > b) ? a : b;
}

char maxOf(char a, char b) {
    return (a > b) ? a : b;
}

// ...and again for every new type you invent.
// Same 1 line of LOGIC, copy-pasted 3+ times.
```



3x the code

Identical logic duplicated per type



3x the bugs

Fix a bug in one copy, forget the other two



Not extensible

A new class you write later needs its own copy again

What Is a Template?

A template is a blueprint (a formula) for creating a family of functions or classes. It is written once, using a placeholder for the data type, and the compiler generates the real, type-specific code automatically — only when and where it is actually used.

Two flavors of templates

Function templates

generate functions • e.g. `swap()`, `maxOf()`, `printArray()`

Class templates

generate classes • e.g. `Box<T>`, `Stack<T>`, `vector<T>`

Syntax anatomy

```
template < typename T >    // or: template<class T>
T functionName(T a, T b) { ... }
```

Key facts to remember

- ▶ T is a placeholder name — any identifier works, T / U / Value are conventions
- ▶ typename and class are 100% interchangeable here — no behavioral difference
- ▶ Nothing is compiled yet — a template alone produces zero machine code
- ▶ Instantiation: the compiler generates real code the first time you call it with a concrete type, e.g. `maxOf(3, 7)` triggers `T = int`

The Analogy: A Tailor's Paper Pattern

Imagine a tailor who designs ONE paper pattern for a shirt.

- The pattern defines the SHAPE: where the collar goes, where the sleeves attach, where the buttons sit — this shape never changes.
- The pattern does not, by itself, produce a wearable shirt. It is not fabric. Nothing exists yet.
- When the tailor lays the pattern on COTTON, out comes a cotton shirt. On SILK, a silk shirt. On DENIM, a denim shirt — same pattern, different real, physical output.
- Each shirt is cut only when a customer actually orders that fabric — the tailor does not pre-cut every possible fabric in advance.
- If a customer asks for a fabric the pattern's stitching cannot handle (say, chain-link metal, which can't be sewn with a needle), the tailor discovers the problem only while attempting that specific cut — not while designing the pattern.

Mapping the analogy → C++

Paper pattern

The template itself, e.g. `template<typename T>`

Fabric chosen

The concrete type: `int`, `double`, `string`, `Student`

Cutting a shirt

Instantiation — compiler generates real code

One shirt per order

One compiled function/class per distinct type used

Fabric that can't be sewn

Compile error when `T` lacks a needed operator

Anatomy of a Function Template

The Trigger Keyword:

Tells the compiler a blueprint definition follows.

```
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
```

The Injection Point:

'T' acts as a placeholder for the future, unknown data type.

Generic Parameters:

The function accepts references of the unknown type.

Internal Instantiation:

Utilizing the generic placeholder for local variables.

Function Templates

A function template is a single function definition that works for MANY data types. The compiler deduces the type T from the arguments you pass — you rarely have to say it yourself.

```
generic_swap.h

template < typename T >
void swapValues(T &a, T &b) {
    T temp = a;    // works for int, double, string...
    a = b;
    b = temp;
}
```

template<typename T>

The template header — declares T as a placeholder type for THIS function only

T &a, T &b

Both parameters use T — forces caller to pass two values of the SAME type

Type deduction

swapValues(x, y) → compiler reads x's type and sets T automatically; no <int> needed

Explicit instantiation

swapValues<double>(x, y) forces T — rarely needed, but legal and sometimes clearer

Why not just use a macro?

- Macros (#define) do dumb, blind text substitution before compilation — no type checking at all.
- Templates are type-safe: the compiler checks every operation against the real type and reports real, readable errors.
- Templates respect scope and namespaces; macros do not — macros can silently break unrelated code.
- Templates can be overloaded, specialized, and debugged with a real debugger; macros cannot.

Program: Generic swap() Across Three Types

```
swap_demo.cpp

#include <iostream>
#include <string>
using namespace std;

template < typename T >
void swapValues(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    double p = 1.1, q = 2.2;
    string s1 = "cat", s2 = "dog";

    swapValues(x, y);           // T deduced as int
    swapValues(p, q);           // T deduced as double
    swapValues(s1, s2);         // T deduced as string

    cout << x << " " << y << "\n";
    cout << p << " " << q << "\n";
    cout << s1 << " " << s2 << "\n";
}
```

Program output

```
20 10
2.2 1.1
dog cat
```

Tracing it, line by line

- ✓ One template is written. Three DIFFERENT functions get compiled behind the scenes: `swapValues<int>`, `swapValues<double>`, `swapValues<string>`.
- ✓ The compiler chose T by looking at the arguments — nothing was typed in angle brackets.
- ✓ `string` works because `std::string` supports copy assignment (`=`) — the exact operation the template body needs. Any type with a working `=` would compile here too.
- ✓ Zero repeated code was written by us for three different types.

Ambiguity Resolution: Exact Match vs. Template Match

Function Definitions

```
void func(int a) {  
    cout << "Exact Match";  
}
```

```
template <class T>  
void func(T a) {  
    cout << "Template Match";  
}
```

Execution Terminal

func(4);

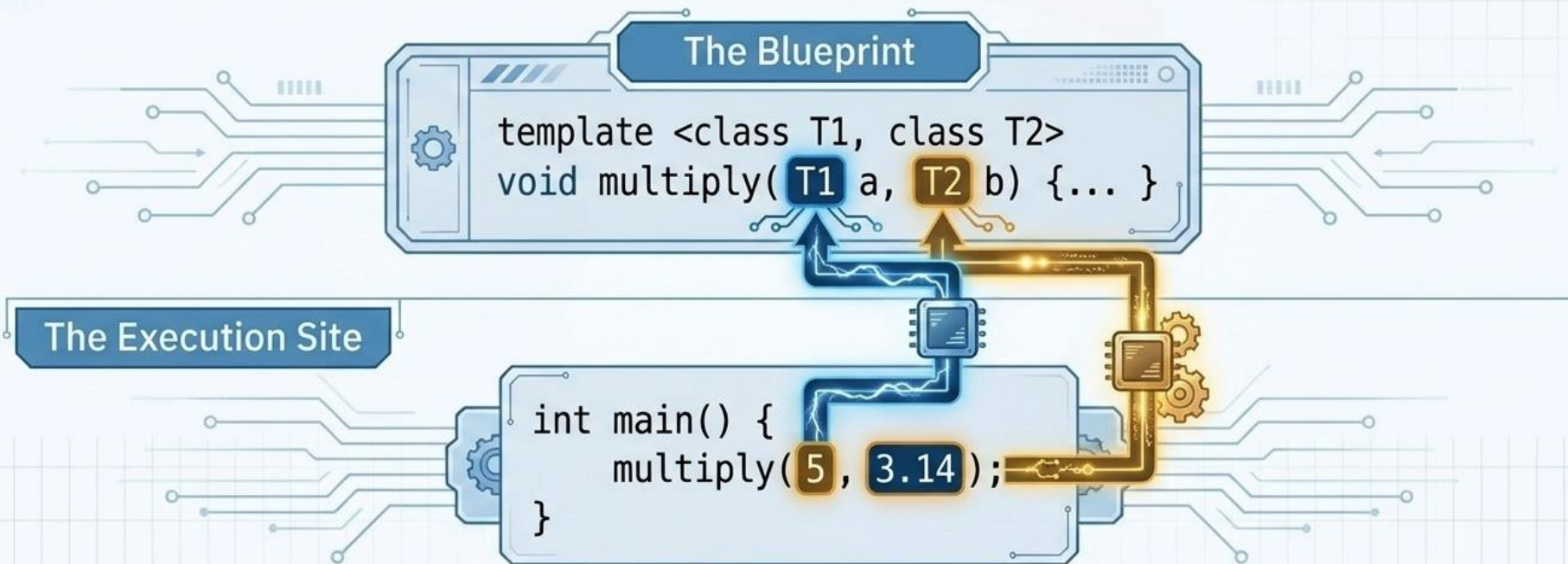
Type int perfectly matches

func(4.5);

No exact match, fallback to
template generation

Resolution Hierarchy: The compiler always prioritizes an exact type match before falling back to generating a template match.

Multi-Parameter Injection



The compiler deduces multiple independent types automatically based on the arguments provided at the call site.

Program: A Template With TWO Type Parameters

A template can declare more than one placeholder type. This lets a function combine values of DIFFERENT types — something a single-T template cannot do.

```
pair_printer.cpp

#include <iostream>
using namespace std;

template < typename T1, typename T2 >
void describe(T1 label, T2 value) {
    cout << label << ": " << value << "\n";
}

template < typename T >
T maxOf(T a, T b) {    // single-parameter version
    return (a > b) ? a : b;
}

int main() {
    describe("Age", 21);           // T1=const char*, T2=int
    describe("Price", 19.99);      // T1=const char*, T2=double
    describe("Passed", true);      // T1=const char*, T2=bool

    cout << "Max: " << maxOf(15, 42) << "\n";
}
```

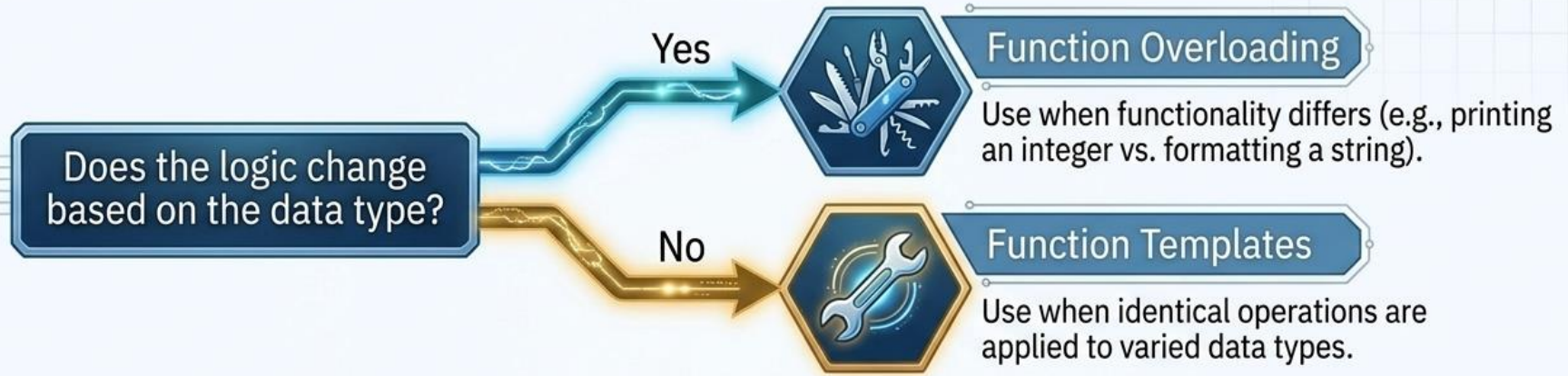
Program output

```
Age: 21
Price: 19.99
Passed: 1
Max: 42
```

What to notice

- T1 and T2 can deduce to DIFFERENT concrete types in the same call — unlike swapValues, which forced both parameters to match.
- true prints as 1: bool has no special-cased template behavior, it is just an integral type here.
- maxOf still needs a > b to be valid for T — this is the same rule from Program 1, applied again.
- A template can have as many type parameters as you like: template<typename A, typename B, typename C> is legal.

Architectural Choice: Overloading vs. Templates



	Function Overloading	Templates
Code Duplication	High	None
Maintenance	Difficult (update all overloads)	Streamlined (single source)
Flexibility	Rigid (limited to defined types)	Infinite (any compatible type)

Function Template Overloading

Just like ordinary functions, templates can be overloaded: several templates (or a template plus a normal function) can share ONE name, as long as their parameter lists differ. The compiler picks the single best match at each call site.

The compiler's pecking order at a call site

1

Exact non-template match

A normal, non-template function that fits perfectly is preferred over ANY template.

2

Best-matching template

Among templates, the one requiring the least type adjustment / most specific match wins.

3

Template + implicit conversion

Used only if nothing above fits without converting the argument's type.

Two ways to overload a template



example.cpp

```
template<typename T> T maxOf(T a, T b);  
template<typename T> T maxOf(T a, T b, T c);
```

A. Same name, different parameter COUNT — both are templates, just distinguished like normal overloads.



example.cpp

```
template<typename T> T maxOf(T a, T b);  
int maxOf(int a, int b);    // non-template
```

B. A plain non-template function alongside the template — handy for a hand-optimized special case for one specific type.

Program: Overloading maxOf() Three Ways

```
overload_demo.cpp

#include <iostream>
using namespace std;

// (1) Generic template - works for ANY comparable type
template<typename T>
T maxOf(T a, T b) {
    cout << "[template 2-arg] ";
    return (a > b) ? a : b;
}

// (2) Template overload - THREE arguments
template<typename T>
T maxOf(T a, T b, T c) {
    cout << "[template 3-arg] ";
    return maxOf(maxOf(a, b), c);
}

// (3) Non-template overload - hand-tuned for int
int maxOf(int a, int b) {
    cout << "[non-template int] ";
    return (a > b) ? a : b;
}

int main() {
    cout << maxOf(3, 9) << "\n";
    cout << maxOf(3.5, 2.1) << "\n";
    cout << maxOf(4, 9, 7) << "\n";
}
```

Program output

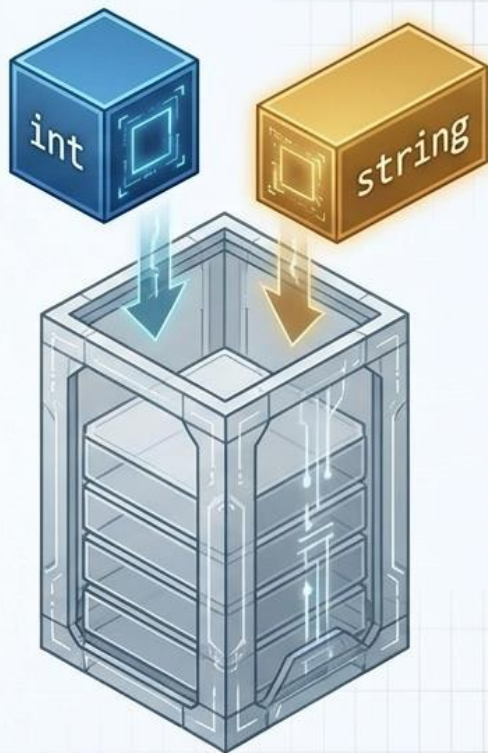
```
[non-template int] 9
[template 2-arg] 3.5
[template 3-arg] [non-template int]
[non-template int] 9
```

Why the compiler chose each one

- ▶ `maxOf(3, 9)`: two ints exactly match the NON-template overload — rule #1 wins, no template used at all.
- ▶ `maxOf(3.5, 2.1)`: no non-template version accepts double, so the 2-arg TEMPLATE is instantiated with `T = double`.
- ▶ `maxOf(4, 9, 7)`: only the 3-arg template matches the argument COUNT, so it is chosen (`T = int`). Inside it, `maxOf(a,b)` and then `maxOf(that, c)` each run on plain ints, so BOTH inner calls resolve to the non-template int overload again — rule #1 fires twice more, nested inside the outer template call.

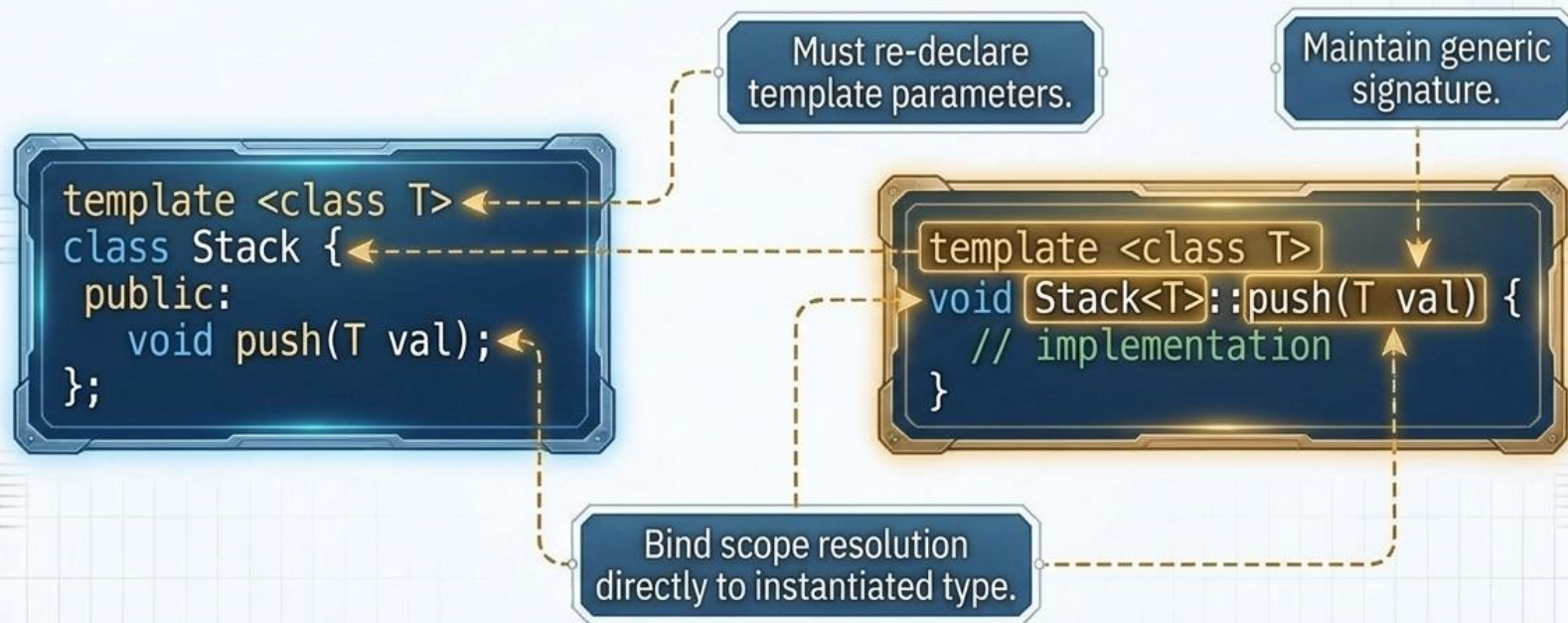
Scaling Up: Class Templates

```
template <typename T>
class Stack {
private:
    std::vector<T> elements;
public:
    void push(const T& element) {
        elements.push_back(element);
    }
    void pop() {
        if (!elements.empty()) {
            elements.pop_back();
        }
    }
    T top() const {
        if (!elements.empty()) {
            return elements.back();
        }
        throw std::out_of_range("Stack is empty");
    }
    bool empty() const {
        return elements.empty();
    }
};
```



Class templates extend generic programming to state and behavior. A single class blueprint can instantiate an `int` stack, a `string` stack, or a custom object stack without rewriting the underlying data structure.

Scope Resolution in Template Classes



When extracting a member function from the blueprint, you must **explicitly link** it back to the template architecture.

Non-Type and Default Parameters

Default Parameters

```
template <class T1, class T2 = float>
```

Provides a fallback type if the caller omits one, reducing verbosity at instantiation.



Non-Type Parameters

```
template <class T, int size = 10>  
class Array {  
    T arr[size];  
};
```



Non-type parameters must be compile-time constants. They inject fixed structural values (like array bounds) directly into the generated class blueprint, rather than generic data types.

Class Templates

A class template is a blueprint for a whole FAMILY of classes — same members, same methods, same behavior — but the data type(s) they operate on are supplied later, when the class is used. Every container in the STL (vector, list, map...) is a class template.

 syntax

```
template < typename T >
class Box {
private:
    T item;
public:
    Box(T val) : item(val) {}
    T get() { return item; }
};
```

Using it — T is supplied at the call site

 example.cpp

```
Box<int> b1(42);
Box<string> b2("hi");
```

- ▶ `Box<T>` alone is only a template — no memory, no class, until instantiated.
- ▶ `Box<int>` and `Box<string>` are TWO fully separate, unrelated compiled classes that both happen to share one written definition.
- ▶ Unlike function templates, the type is (almost) always given explicitly in angle brackets — the compiler cannot deduce `T` just from a constructor call in most cases before C++17.

Program: A Generic Box<T> Container

box_demo.cpp

```
#include <iostream>
#include <string>
using namespace std;

template<typename T>
class Box {
private:
    T item;
public:
    Box(T val) : item(val) {}           // constructor

    T get() const { return item; }
    void set(T val) { item = val; }
};

int main() {
    Box<int> intBox(10);
    Box<string> strBox("hello");

    cout << intBox.get() << "\n";
    intBox.set(intBox.get() + 5);
    cout << intBox.get() << "\n";

    cout << strBox.get() << "\n";
}
```

Program output

```
10
15
hello
```

What is really happening

- ✓ Box<int> and Box<string> are two independent classes generated from one definition — intBox and strBox share zero compiled code with each other.
- ✓ T behaves exactly like a real type everywhere inside the class: as a member's type, a parameter's type, and a return type.
- ✓ get() is a const member function — a normal OOP feature, completely unaffected by T being generic. Templates and ordinary class features combine freely.
- ✓ If you called Box<int> box2; (no argument) it would fail to compile — there is no default constructor, exactly as it would for a non-template class missing one.

Program: A Class Template With TWO Type Parameters

pair_demo.cpp

```
#include <iostream>
#include <string>
using namespace std;

template<typename T1, typename T2>
class KeyValue {
private:
    T1 key;
    T2 value;
public:
    KeyValue(T1 k, T2 v) : key(k), value(v) {}

    void show() {
        cout << key << " -> " << value << "\n";
    }
};

int main() {
    KeyValue<string, int> age("Rita", 22);
    KeyValue<int, double> price(101, 49.5);

    age.show();
    price.show();
}
```

Program output

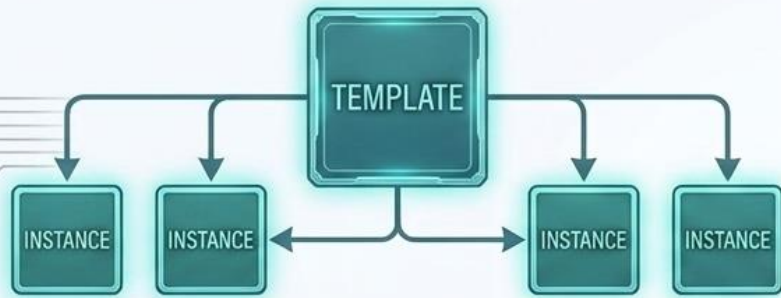
```
Rita -> 22
101 -> 49.5
```

Sound familiar?

- ▶ This is, on purpose, a hand-rolled version of `std::pair<T1,T2>` from the STL — the real one adds operators like `==`, `<`, and structured-binding support, but the core idea is identical.
- ▶ `KeyValue<string,int>` and `KeyValue<int,double>` are two totally distinct classes — again, sharing one source definition and zero runtime relationship.
- ▶ This is exactly the same jump we made earlier from single-T `swapValues()` to two-T `describe()` — the same principle just applied to a class instead of a function.

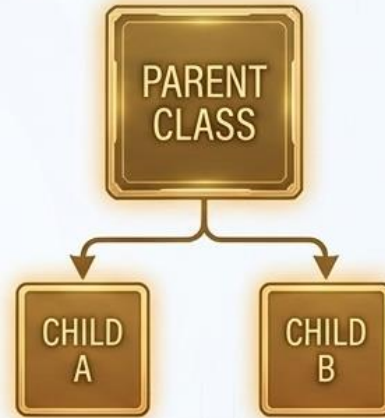
Architectural Paradigms: Templates vs. Inheritance

Generic Programming



- Relationship: Independent, sibling-like instances.
- Binding Time: Compile-time (Static).
- Purpose: Code reusability for different data types executing the exact same logic.

Object-Oriented Programming



- Relationship: "Is-A" hierarchical relationships (Parent/Child).
- Binding Time: Run-time (Dynamic / Virtual).
- Purpose: Code sharing and polymorphism based on varied object behaviors.

Templates vs. Inheritance: Two Different Kinds of Reuse

Both let you avoid rewriting code — but they solve DIFFERENT problems. Confusing them is one of the most common intermediate-level mistakes in C++ design.

T

Templates: same LOGIC, different TYPES

- Answers: 'I want to run the IDENTICAL algorithm on int, double, string, MyClass...'
- Relationship between the types used: NONE required. int and string share no relationship, yet both work with `maxOf<T>`.
- Resolved at COMPILE time. The compiler generates separate code per type; nothing is decided while the program runs.
- No shared base class, no virtual functions, no runtime overhead for the genericity itself.
- Example: `vector<int>` and `vector<string>` are unrelated types that happen to reuse one written definition.

I

Inheritance: same FAMILY, different BEHAVIOR

- Answers: 'I want several RELATED classes to share common fields/methods, while each can override some behavior.'
- Relationship between the types: an IS-A relationship is required — Dog IS-A Animal, Circle IS-A Shape.
- Can be resolved at RUN time via virtual functions (polymorphism) — the actual method invoked can depend on the object's real type, decided while the program runs.
- Introduces a base class, a vtable, and (typically) a small runtime dispatch overhead per virtual call.
- Example: Shape → Circle, Square, Triangle all share the Shape interface but compute `area()` differently.

Templates vs. Inheritance — At a Glance

Aspect	Templates (Generic Programming)	Inheritance (OOP)
Purpose	Same algorithm/structure for many, unrelated types	Shared interface/behavior across related types
Relationship needed	None — types can be completely unrelated	IS-A relationship between base and derived
When resolved	Compile time (static)	Run time, when using virtual functions (dynamic)
Runtime cost	None — fully resolved before the program runs	Small — vtable lookup per virtual call
Typical use	Containers, algorithms: <code>vector<T></code> , <code>sort<T></code>	Polymorphic hierarchies: <code>Shape</code> , <code>Employee</code> , <code>Animal</code>
C++ example	<code>template<typename T> class Stack {...}</code>	<code>class Circle : public Shape {...}</code>

Rule of thumb: reach for a template when the TYPE varies but the LOGIC does not. Reach for inheritance when the BEHAVIOR must vary across a family of related objects.

The Culmination: The Standard Template Library (STL)

The STL

Containers

Pre-built generic data structures (e.g., `std::vector<T>`).



Algorithms

Generic logic that operates on data (e.g., `std::sort`).



Iterators

The standardized pointers linking algorithms to containers.



C++ Templates

The STL is the ultimate validation of generic programming—a unified library of highly optimized, type-safe structures built entirely on template architecture.

STL in Action: Type Deduction at Scale

Class Template
instantiated as a
highly optimized
Integer Array.

```
std::vector<int> numbers = {5, 2, 8, 1};  
std::sort(numbers.begin(), numbers.end());
```

Function Template
deducing the integer type
automatically via the
iterators provided.

Notice what is missing:
You did not write a sorting algorithm.
You did not write a dynamic array class.
The compiler generated assembly for both,
perfectly matched to your type.

The Standard Template Library (STL)

The STL is a collection of ready-made, thoroughly tested, highly optimized class templates and function templates that ship with every standard C++ compiler — so you rarely need to hand-write a `Box<T>`, a `Stack<T>`, or a `maxOf<T>` again.

Containers

Class templates

Store collections of objects

`vector`, `list`, `map`, `set`, `stack`

Iterators

Generalized pointers

Walk through a container's elements uniformly

`begin()`, `end()`, `it++`

Algorithms

Function templates

Operate on ranges via iterators

`sort()`, `find()`, `accumulate()`

Functors / Lambdas

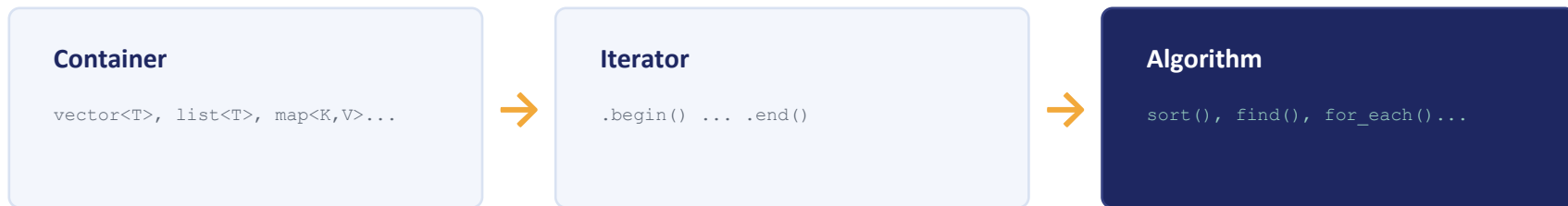
Callable objects

Customize how an algorithm behaves

`less<T>`, custom comparators

Containers, Iterators, and Algorithms Work Together

The power of the STL comes from ONE convention shared by every container: it exposes `begin()` and `end()` iterators. Any algorithm written against that convention works on ANY container — without the algorithm ever needing to know which container it received.



Why this design is powerful (N containers × M algorithms, not N×M code)

- ▶ Without this design: to sort a vector you'd need `vectorSort()`, and to sort a list, `listSort()` — N containers times M algorithms means N×M hand-written functions.
- ▶ With iterators as the 'universal adapter': one `sort()` function template, written once, works across every container that provides compatible iterators — N + M effort, not N×M.
- ▶ This is exactly the function-template idea (Program 1's `swapValues<T>`) applied one level higher: instead of parameterizing over a data TYPE, the STL parameterizes algorithms over an ITERATOR type.

STL Program: vector<T> + Iterators + sort()

```
vector_demo.cpp

#include <iostream>
#include <vector>
#include <algorithm> // sort() lives here
using namespace std;

int main() {
    vector<int> nums = {40, 10, 30, 20}; // container

    nums.push_back(50); // grows dynamically

    sort(nums.begin(), nums.end()); // algorithm + iterators

    for (vector<int>::iterator it = nums.begin();
         it != nums.end(); ++it) {
        cout << *it << " ";
    }
    cout << "\n";

    // range-based for - same iterators, cleaner syntax
    for (int n : nums) cout << n << " ";
}
```

Program output

```
10 20 30 40 50
10 20 30 40 50
```

Mapping back to the 3-box diagram

- ✓ vector<int> is a CLASS TEMPLATE — exactly Box<T>'s cousin, but pre-built and resizable.
- ✓ nums.begin() / nums.end() are ITERATORS — pointer-like objects marking the range to work on.
- ✓ sort(...) is a FUNCTION TEMPLATE — it never mentions vector or int in its own definition; it only requires two iterators, so it works on arrays, lists, and more.
- ✓ The range-based for loop is syntactic sugar that uses these same iterators under the hood.

STL Program: map<K,V> + Iterators + find() / accumulate()

```
map_demo.cpp

#include <iostream>
#include <map>
#include <numeric> // accumulate()
using namespace std;

int main() {
    map<string, int> marks; // class template, K != V

    marks["Asha"] = 88;
    marks["Bimal"] = 76;
    marks["Chandra"] = 95;

    // keys are stored in SORTED order automatically
    for (auto& p : marks)
        cout << p.first << ": " << p.second << "\n";

    auto it = marks.find("Bimal");
    if (it != marks.end())
        cout << "Found: " << it->second << "\n";
}
```

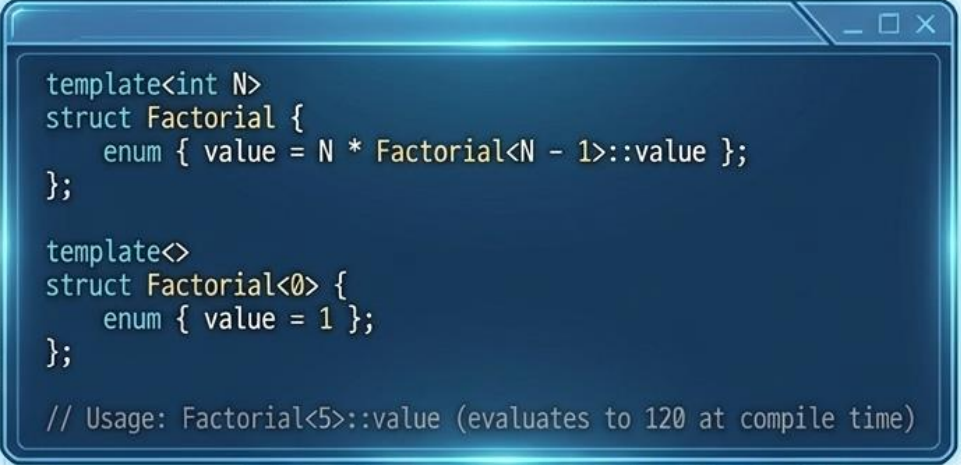
Program output

```
Asha: 88
Bimal: 76
Chandra: 95
Found: 76
```

New ideas this program adds

- ▶ map<K,V> is a class template with TWO type parameters — exactly like the KeyVal<T1,T2> built earlier — but self-balancing and always kept sorted by key.
- ▶ p.first / p.second: each element an iterator yields is a std::pair<const K, V> — the very class template built by hand a few slides ago.
- ▶ find() is another FUNCTION TEMPLATE from <algorithm>-style headers — same family as sort(), just a different job: locate an element and return its iterator, or end() if absent.
- ▶ auto lets the compiler deduce the (verbose) iterator type for us — purely a convenience, not a new concept.

Beyond Blueprints: Template Metaproprogramming



```
template<int N>
struct Factorial {
    enum { value = N * Factorial<N - 1>::value };
};

template<>
struct Factorial<0> {
    enum { value = 1 };
};

// Usage: Factorial<5>::value (evaluates to 120 at compile time)
```



- Because templates are evaluated before the program ever runs, they can be used to perform **complex calculations and logic checks** during compilation.
- This unlocks **metaprogramming**: writing code that generates code, resulting in **zero runtime overhead**.

The Generic Programming Paradigm



Write Once, Compile Anywhere

Templates replace manual redundancy with compiler-driven automation.

Zero Overhead

Generic code is just as fast as hand-written, type-specific code because generation happens entirely at compile-time.

Type Safety Guarantee

Unlike raw pointers or macros, templates enforce strict type checking, combining infinite flexibility with uncompromised safety.

Templates transform C++ from a language of strict types into a flexible engine of generic design.

C++ Namespaces

Giving every name in your program its own address, so nothing ever collides.

Definition & Syntax

`namespace { ... } and ::`

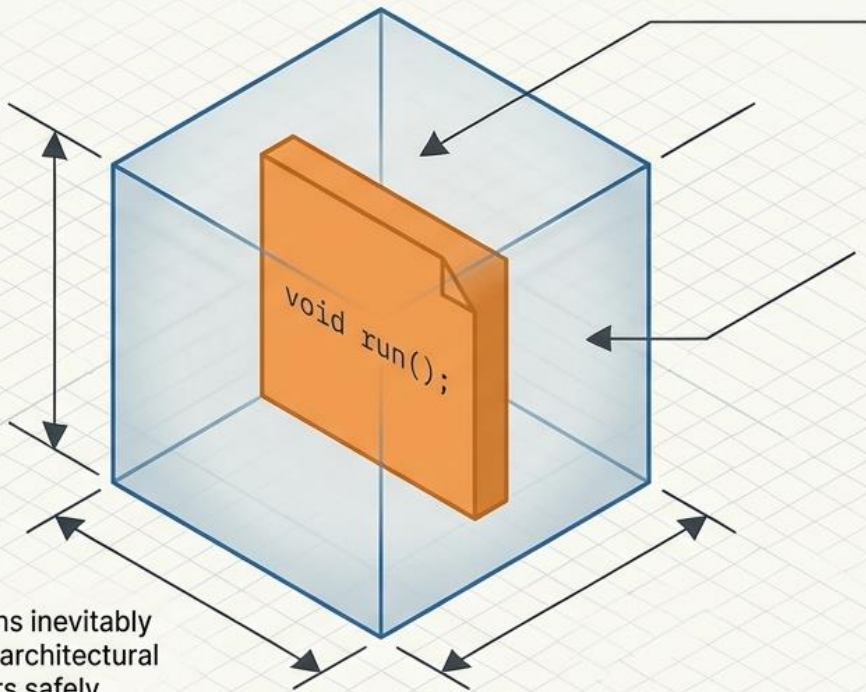
Creating & Using

3 full programs, traced output

Best Practices

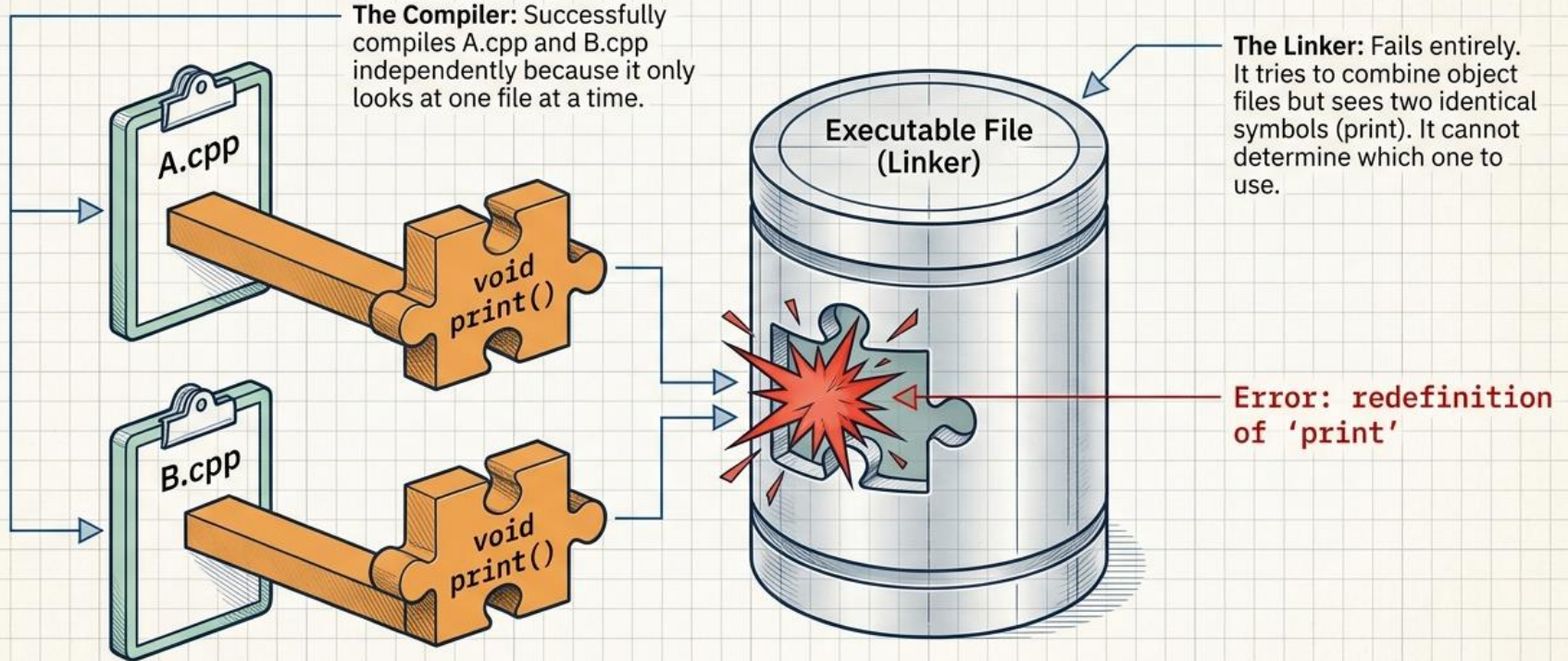
using vs. fully-qualified names

Mastering C++ Namespaces



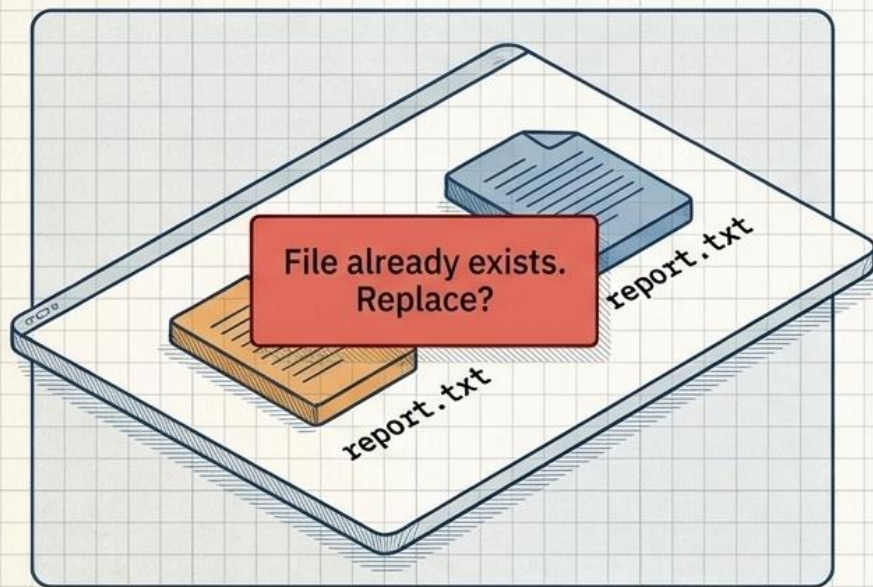
As C++ programs grow, naming collisions inevitably crash the system. Namespaces are the architectural boundaries that keep identical identifiers safely separated. This guide maps the mechanics of scopes, standard libraries, and collision prevention.

Linker errors occur when identical names collide in the global scope

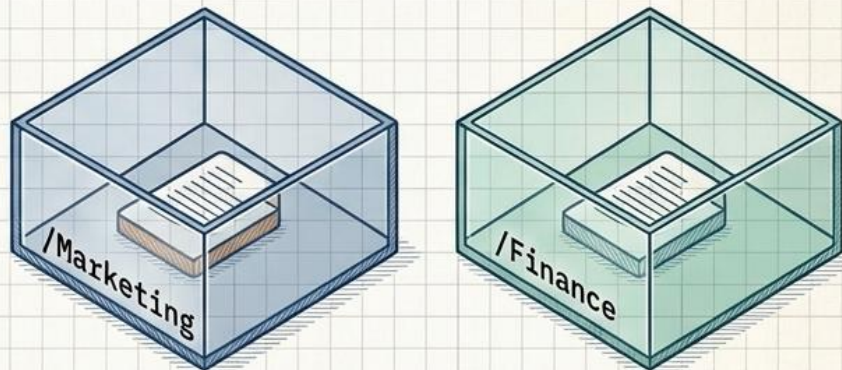


Namespaces act as dedicated physical folders for your code

The Global Scope (Desktop)



Namespaces (Folder System)



`Marketing/report.txt` → `Marketing::report`

The **Scope Resolution Operator** (`::`) is simply C++'s version of a file path slash (`/`). It tells the compiler exactly which container to look inside.

Why Do We Need Namespaces?

As programs grow — and especially once you combine code from multiple libraries — it becomes likely that two different pieces of code will use the SAME name for two DIFFERENT things.

```
name_collision.cpp

// From Library A
void display() { /* prints a report */ }

// From Library B — SAME name, different job
void display() { /* prints a UI widget */ }

int main() {
    display();    // Which one?? — will not even compile
}
```

```
error: redefinition of 'void display()'
```

Definition

A namespace is a declarative region that gives its own NAME to a group of identifiers — functions, classes, variables — so they can be told apart from identically-named identifiers declared elsewhere. It doesn't change WHAT the code does; it changes how the compiler locates a name.

- ▶ Solves exactly one problem: naming collisions across large or multi-library codebases.
- ▶ The C++ Standard Library itself lives inside one giant namespace: `std` (`std::cout`, `std::vector...`).
- ▶ A namespace can be reopened — the same namespace name can appear in multiple places/files and all contributions merge together.

The Analogy: A Village Full of Johns

Imagine a small village where **SIX** different families each happen to have a son named John.

- If you shout 'John!' in the village square, six heads turn — the name alone is **AMBIGUOUS**. This is exactly the compiler's 'redefinition' or 'ambiguous name' error.
- The village solves this with **FAMILY SURNAMES**: John Smith, John Carter, John Lee... Now 'John Smith' unambiguously refers to exactly one person.
- You only need the surname when there's a chance of confusion. Standing inside the Smith house, everyone already knows which John you mean, so 'John' alone is fine there.
- Two families can each organize their **OWN** household however they like — the Smiths can have a John and a Mary; the Carters can also have a John and a Mary. No collision, because each name is really 'scoped' to its family.
- A visitor from out of town who wants to be unambiguous everywhere just always says the full name: 'Smith's John' — slower to say, but never wrong.

Mapping the analogy → C++

The village square (no surname)

The global namespace — where an unqualified name is looked up first

A family surname

A namespace name, e.g. Physics, Company, std

"Smith::John" style full name

Scope resolution: `Smith::John` $\equiv$ `Physics::calculateArea`

Talking freely inside the Smith house

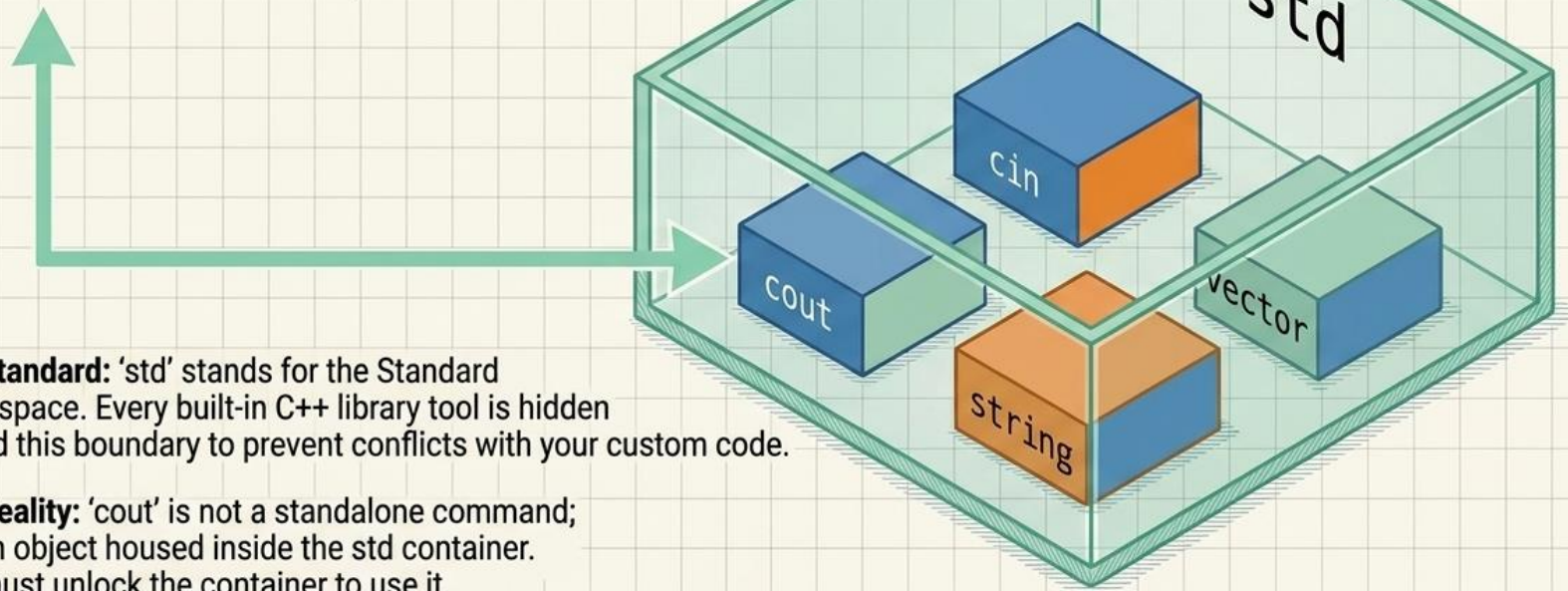
using namespace Smith; — brings ALL of Smith's names into scope, unqualified

Introducing just one Smith by name at a party

using Smith::John; — a using-DECLARATION, brings only ONE name into scope

The standard library lives exclusively inside the 'std' container

```
#include <iostream>  
std::cout << "Hello";
```



 **The Standard:** 'std' stands for the Standard Namespace. Every built-in C++ library tool is hidden behind this boundary to prevent conflicts with your custom code.

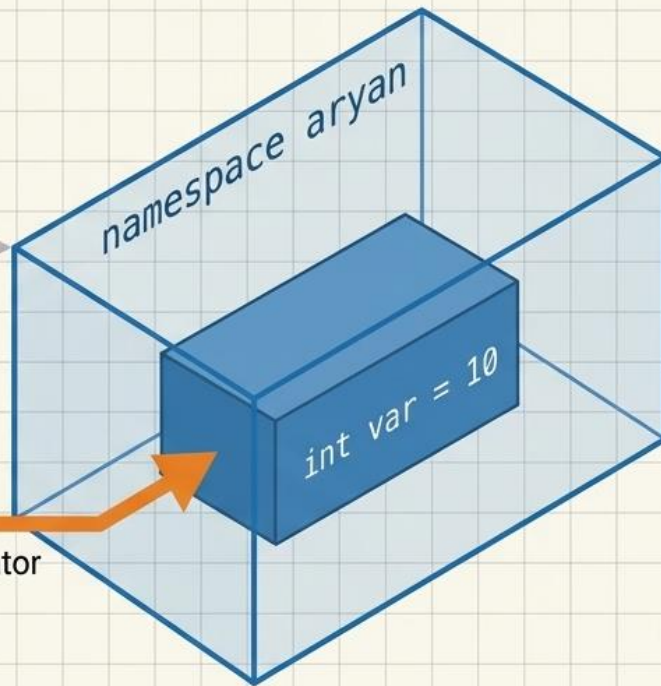
 **The Reality:** 'cout' is not a standalone command; it is an object housed inside the std container. You must unlock the container to use it.

Defining and accessing custom namespace boundaries

```
namespace aryan {  
    int var = 10;  
}
```

```
int main() {  
    std::cout << aryan::var;  
}
```

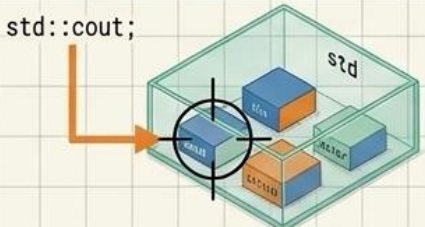
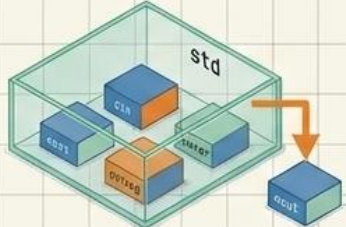
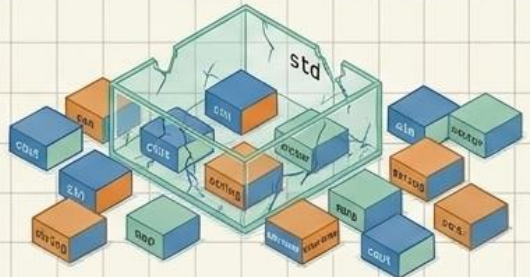
Scope Resolution Operator



 **Declaration:** The namespace keyword defines a declarative region that provides a unique scope to identifiers inside it.

 **Access:** To retrieve a variable, use the namespace name followed by the scope resolution operator (::).

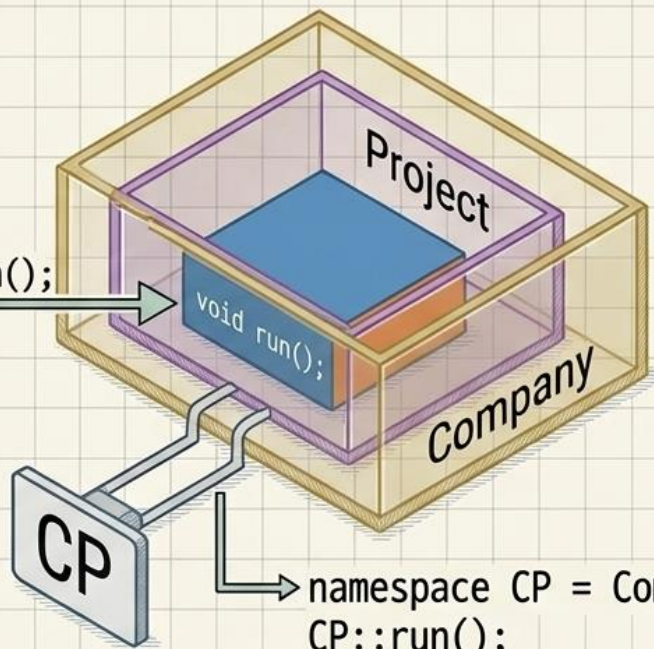
Evaluating the three primary namespace access strategies

Strategy	Visual Metaphor	Risk Level	Verdict
Fully Qualified Names (e.g., <code>std::cout</code>)		 Zero Risk	Industry Standard. Explicitly communicates which namespace an object belongs to.
Specific Using Declarations (e.g., <code>using std::cout;</code>)		 Low Risk	Clean & Practical. Reduces typing for repetitive local use without polluting the entire file.
Global Using Directives (e.g., <code>using namespace std;</code>)		 Extreme Risk	Highly Discouraged. Undoes the protection of namespaces, dramatically increasing the chance of naming conflicts as your project scales.

Organizing enterprise systems with nested namespaces and aliases

```
namespace Company {  
  namespace Project {  
    void run();  
  }  
}
```

`Company::Project::run();`



✚ **Nesting:** Namespaces can be layered infinitely to organize massive codebases (e.g., separating physics code from rendering code).

✚ **Nesting:** Namespaces can be layered infinitely to organize massive codebases (e.g., separating physics code from rendering code).

✚ **Aliasing:** Deeply nested paths become tedious to type. Namespace aliases provide a custom, shorthand identifier for a deeply nested scope without polluting the global namespace.

Nested Namespaces & the Two “using” Forms

```
nested_using.cpp

#include <iostream>
using namespace std;

namespace Company {
    namespace HR {          // nested inside Company
        void greet() { cout << "Hello from HR\n"; }
    }
    namespace Finance {
        void greet() { cout << "Hello from Finance\n"; }
    }
}

int main() {
    using Company::HR::greet;    // using-DECLARATION: one name
    greet();                    // resolves to HR::greet, unqualified

    // still fully-qualified — no using needed for this one
    Company::Finance::greet();

    using namespace Company::HR; // using-DIRECTIVE: ALL names
}
```

Program output

```
Hello from HR
Hello from Finance
```

using-DECLARATION vs. using-DIRECTIVE

- ▶ using Company::HR::greet; brings only ONE specific name into the current scope — precise, low risk of new collisions.
- ▶ using namespace Company::HR; brings EVERY name HR has (now or added later) into scope at once — convenient, but higher risk of an unexpected future collision.
- ▶ Company::Finance::greet() needs no using at all — fully-qualified access always works, everywhere, forever.

Creating a Namespace & the :: Operator

```
namespace_basics.cpp

#include <iostream>
using namespace std;

namespace Physics {           // declare a namespace
    double calculateArea(double side) {
        return side * side;    // area of a square
    }
}

namespace Geometry {          // a SECOND namespace
    double calculateArea(double radius) {
        return 3.14159 * radius * radius; // area of a circle
    }
}

int main() {
    // SAME function name, no collision — disambiguated by ::
    cout << Physics::calculateArea(4) << "\n";
    cout << Geometry::calculateArea(4) << "\n";
}
```

Program output

```
16
50.2654
```

Anatomy of what's on screen

- ✓ namespace `Physics { ... }` opens a declarative region — everything inside it now has the qualified name `Physics::whatever`.
- ✓ Two functions named `calculateArea` coexist peacefully in the SAME file because they live in two different namespaces — exactly like two families each having a John.
- ✓ `::` is the scope resolution operator — read `Physics::calculateArea` as 'the `calculateArea` that belongs to `Physics`.'
- ✓ Nothing here is a class or an object — a namespace has no constructor, no instances, and cannot be 'called'; it is purely an addressing/organizational tool.

Namespace Aliases & the Anonymous Namespace

```
alias_anonymous.cpp

#include <iostream>
using namespace std;

namespace VeryLongCompanyName {
    void report() { cout << "Report generated\n"; }
}

// ALIAS — a short, local nickname for a long namespace name
namespace VLC = VeryLongCompanyName;

// ANONYMOUS namespace — no name at all
namespace {
    int secretCounter = 0;    // only visible in THIS file
}

int main() {
    VLC::report();            // shorthand, via the alias

    secretCounter++;
    cout << "Counter: " << secretCounter << "\n";
}
```

Program output

```
Report generated
Counter: 1
```

Two more tools worth knowing

- ▶ namespace VLC = VeryLongCompanyName; creates an ALIAS — a short nickname for a long or deeply-nested namespace name, purely for readability. (The real STL uses this pattern too, e.g. namespace fs = std::filesystem;)
- ▶ An anonymous namespace — namespace { ... } with no name — makes everything inside it usable ONLY within this one file, invisible to every other file in the project. It is the modern, safer alternative to the old C-style static global variable.

Templates & Namespaces

- Function Template: Same algorithm, any data type
- Class Template: Generic container/data structure
- STL uses templates: `vector<T>`, `stack<T>`, `map<K,V>`
- Namespace: Prevents name collisions between libraries
- `using namespace X;` — import all names from X
- `using X::name;` — import only one specific name
- Nested namespaces allowed in C++17 (`A::B::C`)

```
// Function template
template <typename T>
T findMax(T a, T b) {
    return (a > b) ? a : b;
}

// Class template
template <typename T>
class Stack {
    T data[100]; int top=-1;
public:
    void push(T v) { data[++top]=v; }
    T pop() { return data[top--]; }
};
```


Exception Handling in C++

Try. Throw. Catch.

A complete, code-first walkthrough of how C++ detects, signals, and recovers from runtime errors — without crashing the program.

```
main.cpp

try {
    riskyOperation();
}
catch (const exception& e) {
    handle(e);
}

// the program survives.
```

Unpredictable runtime errors require active safety mechanisms

Syntax Errors

Caught by the compiler before the machine even turns on.

E.g., Missing semicolons.

Logic Errors

The machine runs, but produces the wrong output.

E.g., Incorrect math formulas.

Runtime Errors

The machine crashes during operation due to unpredictable conditions.

E.g., Division by zero, missing files, out-of-memory.

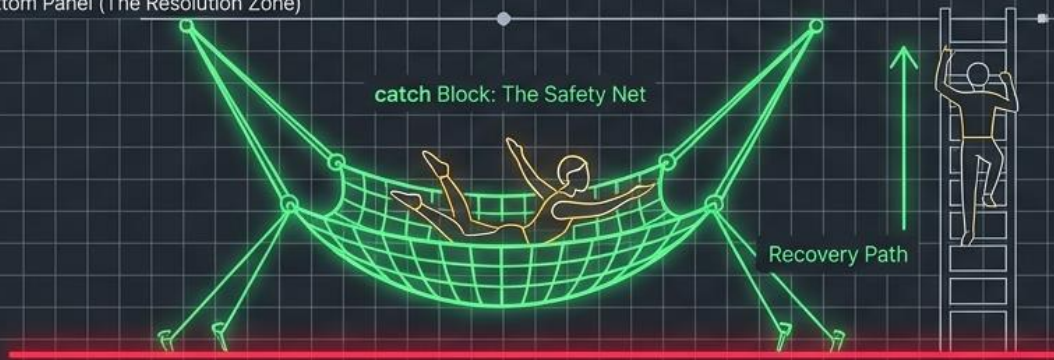
C++ Exception Handling exists exclusively to control the unpredictable: it intercepts Runtime Errors, prevents catastrophic program termination, and safely redirects execution.

The Exception Pipeline isolates hazards and catches failures

Top Panel (The Hazard Zone)



Bottom Panel (The Resolution Zone)



1. Try: Encloses code that might fail. ⓘ

2. Throw: Explicitly signals an exceptional condition has occurred. ⚡

3. Catch: Matches the exception type and executes recovery logic. 🛡️

What Is an Exception?



Definition

An exception is an unexpected condition or error that arises during program execution and disrupts the normal flow of instructions — for example, dividing by zero, an invalid array index, or a failed memory allocation.

Why Exception Handling Matters

1

Separates concerns

Error-recovery code stays out of the normal business logic.

2

Prevents crashes

A handled exception lets the program recover instead of terminating abruptly.

3

Automatic propagation

An unhandled exception travels up the call stack until a handler is found.

4

Structured recovery

Different problems can be matched to different, specific responses.



Detailed Analogy

The Building Fire-Alarm System

Normal operations → try block

Employees work, elevators run, meetings happen — the building runs its regular routine, just like code executing inside a try block.

Fire detected → throw

A sensor detects smoke and triggers the alarm. It does not fix the fire itself — it only signals “something is wrong,” exactly like a throw statement raising an exception object.

Response team → catch

Different alarms summon different responders — the fire department for smoke, medics for a health emergency, security for a break-in. Each catch block matches one specific type of problem.

No one responds → uncaught exception

If no team answers the alarm, the building must be evacuated entirely. Likewise, an uncaught exception halts the whole program via `std::terminate()`.

Exception Handling

Unlike its predecessor, C, C++ boasts a robust Exception Handling mechanism, introducing specialized keywords for effective error management. Exceptions in C++ denote irregularities or abnormal conditions during program execution, categorized into Synchronous and Asynchronous types.

Key Exception Handling Keywords

1

Try :

Marks a code block under scrutiny for potential exceptions.

2

Catch:

Represents a block executed when a specific exception is thrown, serving as a handler.

3

Throw:

Explicitly used to throw an exception, also specifying exceptions a function may throw without internal handling.

Exception Handling in C++

Exception handling in C++ is a vital mechanism that allows programs to gracefully manage runtime errors and exceptional situations. These exceptions, such as division by zero or file I/O errors, are addressed through a structured approach in C++. The fundamental idea behind exception handling is to separate normal program flow from error-handling code. Rather than abruptly terminating the program on encountering an error, C++ employs the throw statement to indicate an exception, representing the error or exceptional condition. The thrown exception is then caught by specific catch blocks, where the program can handle the error in a controlled manner.

Basic Exception Handling Flow

1

Throwing an Exception :

When a critical error occurs, the throw statement is used to raise an exception. An object typically accompanies this statement, serving as the representation of the error.

2

Catching an Exception:

The try block encompasses code that might raise an exception. If an exception is thrown within the try block, control jumps to the corresponding catch block.

3

Handling the Exception:

The catch block handles the caught exception, specifying the type of exception it can handle and providing code to address the exceptional condition.

Exception Handling: try-throw-catch

```
#include <iostream>
#include <stdexcept>
using namespace std;

float divide(float a, float b) {
    if (b == 0)
        throw runtime_error("Div by zero!");
    return a / b;
}

int main() {
    try {
        cout << divide(10, 2); // OK
        cout << divide(5, 0);  // Throws
    }
    catch (const runtime_error &e) {
        cout << "Error: " << e.what();
    }
    catch (...) { // Catch-all
        cout << "Unknown error";
    }
}
```



Exceptions separate error handling from normal logic. Never use exceptions for normal flow control.

Exceptions decouple error management from core logic

Code Readability

Traditional Error Codes

Cluttered. Logic is buried under layers of `if (error == true)` checks.

Exception Handling

Clean. Primary logic is separated from error management.

Control Flow

Linear and manual. Every function must manually check and pass errors up.

Automatic jumping. Execution immediately halts and jumps to the nearest handler.

Error Propagation

Silent failures if a developer forgets to check a return code.

Unignorable. Unhandled exceptions force the program to safely **terminate** rather than corrupting data.

The Try – Throw – Catch Model

TRY

Wrap code that might fail

Marks a region of “risky” statements. If nothing goes wrong, execution simply skips every catch block.

THROW

Signal the problem

Creates an exception object (any type) and immediately transfers control out of the current function.

CATCH

Handle the problem

Matches the thrown object's type against each catch in order and runs the first one that fits.

```
syntax.cpp

try {
    // code that MIGHT throw
    doSomethingRisky();
}
catch (ExceptionType1 e1) {
    // handles ExceptionType1
}
catch (ExceptionType2 e2) {
    // handles ExceptionType2
}
catch (...) {
    // catches ANYTHING else
}
```

Key Rules to Remember

- throw can raise a value of any type — an int, a string, or (best practice) an object derived from std::exception.
- Control leaves the try block the instant throw executes; any remaining try statements are skipped entirely.
- Stack unwinding: as control exits each function, that function's local objects are destroyed in reverse order until a matching catch is found.
- catch(...) matches any exception type and is normally placed LAST as a safety net.
- An exception with no matching catch anywhere calls std::terminate() and ends the program.

Intercepting a division by zero error in real time

```
int main() {  
    int numerator = 10, denominator = 0;  
    try {  
        if (denominator == 0) {  
            throw std::runtime_error("Division by zero!");  
        }  
        int result = numerator / denominator;  
        std::cout << result;  
    } catch (const std::exception& e) {  
        std::cout << "Error: " << e.what();  
    }  
    return 0;  
}
```

A diagram illustrating the flow of an exception. A blue line starts at the 'throw' statement, goes down the left side of the try block, and then turns right to point at the catch block. An orange line starts at the 'throw' statement, goes down the right side of the try block, and then turns left to point at the catch block. Both lines converge on the catch block, indicating that the exception is caught there.

Basic Try / Throw / Catch

```
program1.cpp

#include <iostream>
using namespace std;

double divide(double a, double b) {
    if (b == 0)
        throw runtime_error("Division by zero!");
    return a / b;
}

int main() {
    try {
        cout << divide(10, 2) << endl;
        cout << divide(5, 0) << endl;
    }
    catch (const runtime_error& e) {
        cout << "Error: " << e.what() << endl;
    }
    cout << "Program continues..." << endl;
}
```

```
OUTPUT 5
Error: Division by zero!
Program continues...
```

Walkthrough

1

divide() guards the risk

It checks the dangerous condition ($b == 0$) and throws a `runtime_error` object built from `<stdexcept>` before any harm is done.

2

main() wraps the risk

Both calls sit inside one try block. The first call ($10 / 2$) succeeds normally and prints 5.

3

throw jumps immediately

On the second call, `throw` fires. The rest of the try block — including its own `cout` line — is skipped instantly.

4

catch receives by reference

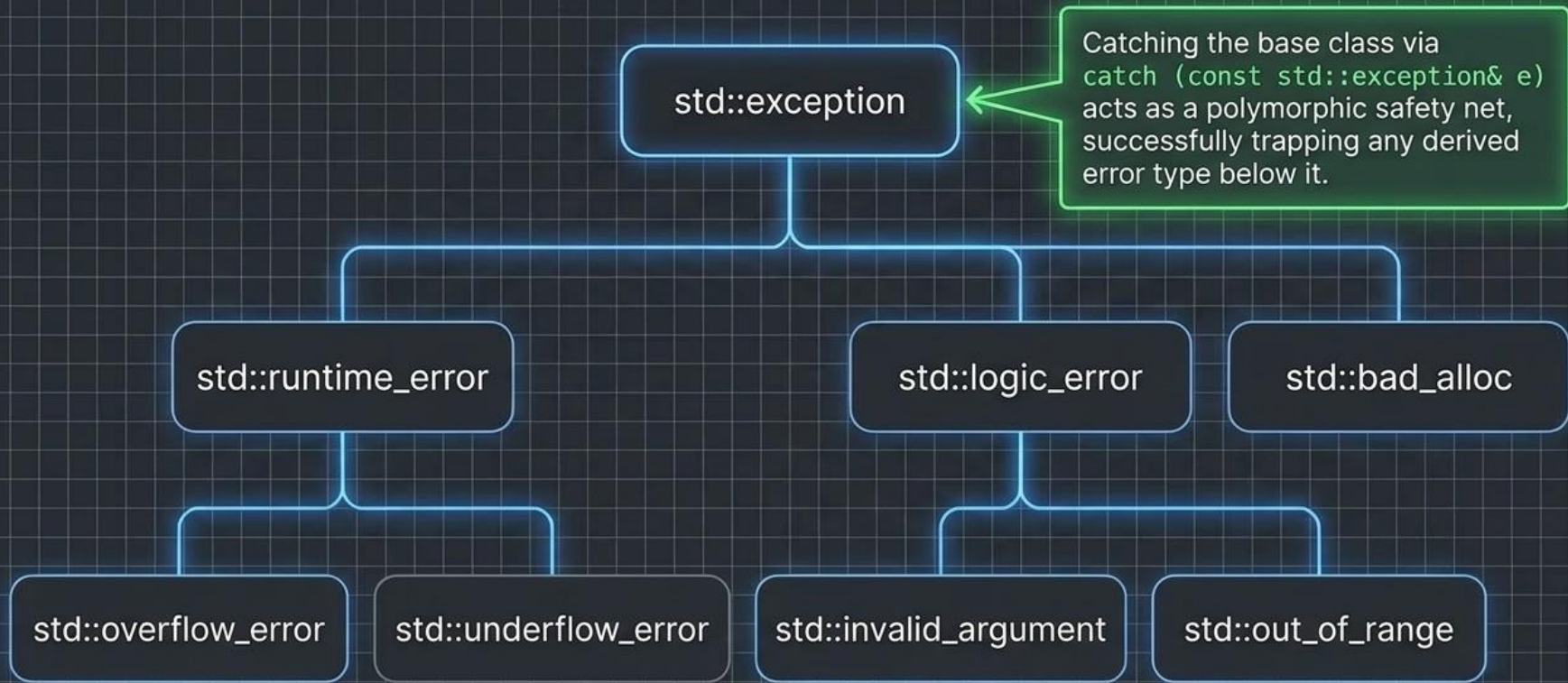
`catch (const runtime_error& e)` binds the thrown object; `e.what()` returns its stored message text.

5

Execution resumes after catch

The program does NOT crash. Control simply continues with the very next statement after the catch block.

The C++ Standard Exception Hierarchy



Multiple Catch Blocks & the Exception Hierarchy

program2.cpp

```
#include <iostream>
#include <stdexcept>
using namespace std;

int getElement(int arr[], int size, int idx) {
    if (idx < 0 || idx >= size)
        throw out_of_range("Index out of bounds");
    if (size == 0)
        throw invalid_argument("Empty array");
    return arr[idx];
}

int main() {
    int marks[5] = {90, 85, 78, 92, 60};
    int index = 7;    // deliberately out of range

    try {
        cout << getElement(marks, 5, index);
    }
    catch (const out_of_range& e) {
        cout << "Range Error: " << e.what();
    }
    catch (const invalid_argument& e) {
        cout << "Argument Error: " << e.what();
    }
    catch (const exception& e) {
        cout << "Unknown error: " << e.what();
    }
}
```

std::exception Hierarchy

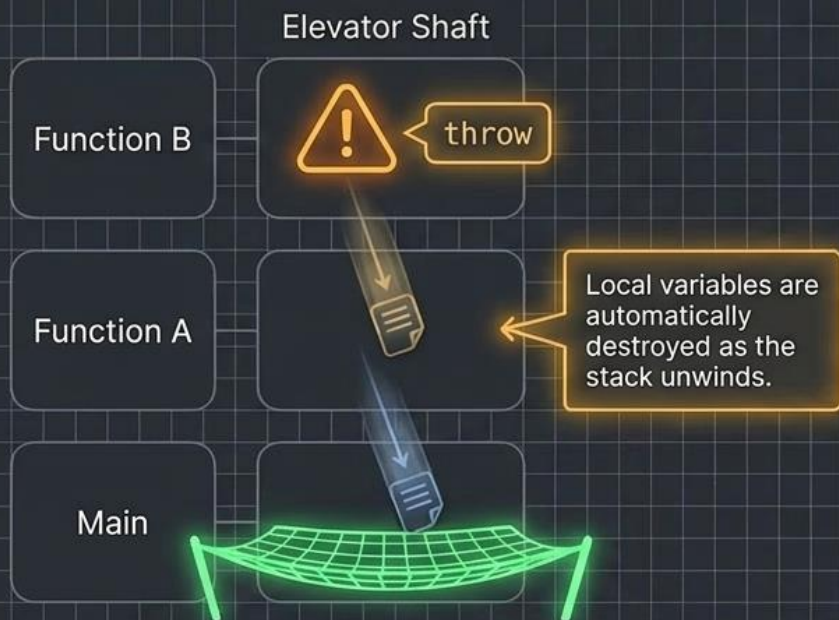
- std::exception
 - logic_error
 - out_of_range
 - invalid_argument
 - runtime_error

Defined in <stdexcept>. out_of_range and invalid_argument both derive from logic_error, which derives from std::exception.

Reading This Program

- One try can be followed by several catch blocks — each handles one exception type.
- Blocks are tested top-to-bottom; the FIRST type match wins.
- Put derived (specific) types before base (general) types — a base-class catch placed first would swallow everything below it.
- catch(const exception&) is a safe general fallback since every standard exception derives from it.

Stack Unwinding propagates errors up the call chain



```
void functionB() {  
    throw std::runtime_error("Hardware fault");  
}  
  
void functionA() {  
    functionB(); // No try/catch here!  
}  
  
int main() {  
    try {  
        functionA();  
    } catch (const std::exception& e) {  
        std::cout << e.what();  
    }  
}
```

Exceptions automatically 'bubble up' through function calls until they find a matching handler. You do not need to wrap every function in a try-block; only the function tasked with managing the error.

Custom Exceptions & Stack Unwinding

```

program3.cpp

#include <iostream>
#include <exception>
using namespace std;

class InsufficientFunds : public exception {
    double shortfall;
public:
    InsufficientFunds(double amt) : shortfall(amt) {}
    const char* what() const noexcept override {
        return "Insufficient funds for withdrawal";
    }
    double getShortfall() const { return shortfall; }
};

class Account {
    double balance;
public:
    Account(double b) : balance(b) {}
    void withdraw(double amt) {
        if (amt > balance)
            throw InsufficientFunds(amt - balance);
        balance -= amt;
        cout << "Withdrew " << amt << ". Balance: " << balance;
    }
};

int main() {
    Account acc(1000);
    try {
        acc.withdraw(400);
        acc.withdraw(800);
    }
    catch (const InsufficientFunds& e) {
        cout << e.what() << ". Short by " << e.getShortfall();
    }
}

```

Custom Exception Classes

- Inherit from `std::exception` (directly or via a subclass).
- Override `what()` as `const noexcept` to return a message.
- Can carry EXTRA data — here, `getShortfall()` reports exactly how much money was missing, not just a text string.

Stack Unwinding

When `withdraw(800)` throws, C++ unwinds the call stack: `withdraw()`'s local objects are destroyed in reverse order as control jumps back to the matching `catch` in `main()`.

C++ has NO finally keyword. Guaranteed cleanup (closing files, freeing locks) is achieved instead through RAII — destructors that run automatically during unwinding.

Custom exception classes provide absolute diagnostic control

```
class ValvePressureException : public
    std::exception {
public:
    const char* what() const noexcept override {
        return "Critical fault: Valve pressure
            exceeded safe limits!";
    }
};
```

The `what()` method is overridden to return a precise, context-specific payload without throwing exceptions itself.

```
// Inside operational code:
throw ValvePressureException();
```

The Exception Protocol

- **Throw By Value, Catch By Reference:** Always use `catch(const std::exception& e)` to prevent object slicing and unnecessary copying.
- **Target Exceptions:** Catch specific exceptions before generic ones. Order matters.
- **Use RAI:** Rely on stack unwinding to automatically close files and release memory via object destructors.
- **The Catch-All:** Use `catch(...)` only as a final, absolute last-resort net for unknown anomalies.

Analogy & PU Question



Real-World Analogy

Templates are like cookie cutters: one cutter works for any type of dough (data type). Namespaces are like area codes in phone numbers — (01) Kathmandu::Ram is different from (061) Pokhara::Ram. Exception handling is like a safety net in a circus: performers (code) do their act, and if they fall (error), the net (catch) prevents catastrophe.



Probable PU Exam Question

Q: What are templates in C++? Differentiate function templates and class templates. Write a C++ program demonstrating exception handling with multiple catch blocks. [PU Exam 2023]

File Handling

- Stream Class Hierarchy: ios → fstream
- Opening files: constructor and open() method
- File Modes: in, out, app, trunc, binary, ate
- File Pointers: seekg, seekp, tellg, tellp
- Error Handling: good(), bad(), fail(), eof()

File Handling in C++

Streams, Modes, Random Access & Object Persistence — from fstream basics to reading and writing binary class objects.



```
#include <fstream>

ofstream fout;
fout.open("data.txt");
fout << "Hello, File!";

fout.close();
```

BUILDING PERSISTENT DATA VAULTS IN C++

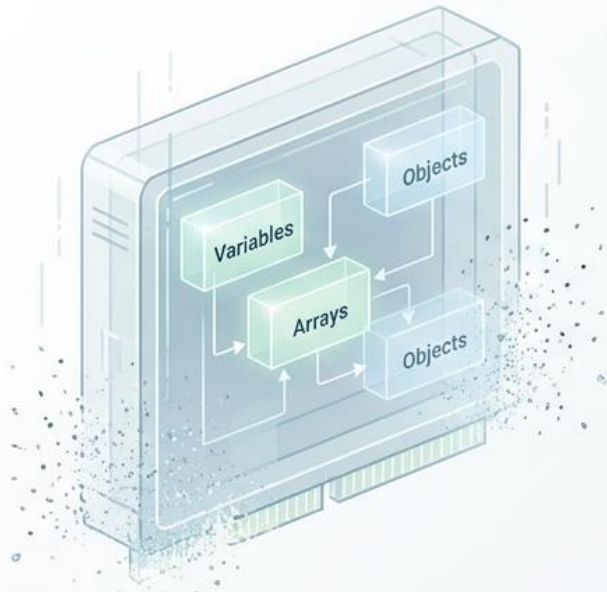
• A comprehensive, visual reference manual for C++ File Handling.

• Master stream architecture, sequential character processing, byte-level random access, and binary object serialization.

- A comprehensive, visual reference manual for C++ File Handling.
- Master stream architecture, sequential character processing, byte-level random access, and binary object serialization.

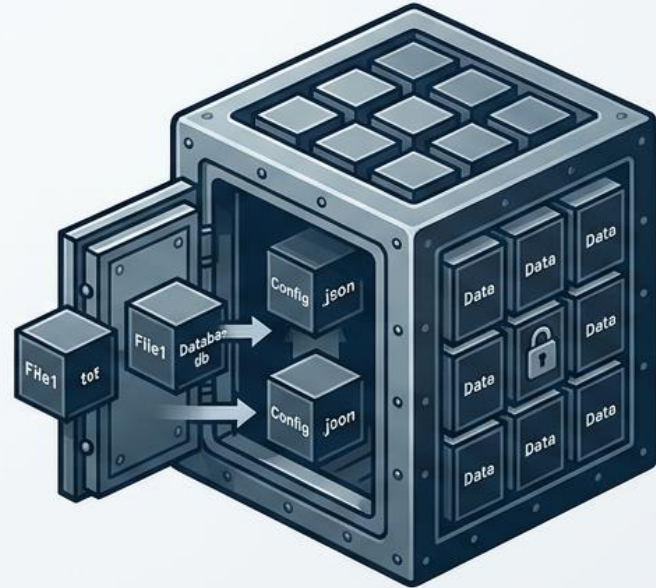
Ephemeral Execution vs. Persistent Storage

RAM (Volatile)



**Power Off / Program Exit
= Data Lost forever.**

Disk (Persistent)



**Files survive termination.
Data is etched permanently.**

File handling bridges the gap between ephemeral memory and permanent storage, allowing programs to store data permanently, retrieve it across sessions, and handle volumes too large for system RAM.

What Is File Handling?

File handling is the mechanism by which a C++ program creates, opens, reads, writes, and closes files stored on secondary storage (disk), using stream classes from the `<fstream>` header so that data can persist beyond the program's lifetime.

- RAM (variables, arrays, objects) is volatile — its contents disappear the instant the program terminates.
- A disk file is persistent — it survives program exit, system reboot, and can be shared between programs.
- C++ treats every source of input and every destination of output — keyboard, screen, or file — uniformly as a stream of bytes.
- The header `<fstream>` supplies three classes: `ifstream` (input/read), `ofstream` (output/write), and `fstream` (both).

RAM vs. DISK

RAM (Program Memory)

int, float, arrays, class objects

⚡ Fast ✗ Erased when program ends

`fstream` / `ifstream` / `ofstream`

Disk (Secondary Storage)

data.txt, students.dat, ...

📁 Slower ✓ Persists after program ends

The Library System Metaphor

Programming syntax mirrors the mechanics of a physical library.

The Library Translation Matrix		
Physical Action	C++ Concept	C++ Syntax
 A specific book	A file on disk	"data.txt"
 Reading/Writing process	Data flow stream	fstream
 Taking book from shelf	Establishing connection	file.open()
 Returning book	Disconnecting	file.close()
 Turning to specific page	Moving the read position	file.seekg()
 Noting current page	Getting current position	file.tellg()

The Library Analogy

Keep this picture in your head for the entire lecture — every function we meet maps onto it.



The Program

You want to read or record information, but you never touch the shelves yourself.



The Librarian

The stream object (ifstream / ofstream / fstream) — the only one allowed to fetch, place or return books.



The Book

The disk file itself — pages of data sitting on a shelf (the disk) until someone requests it.



The Bookmark

The file pointer — marks exactly which page/word you'll read or write next.

Opening a file

The librarian walks to the shelf, retrieves the book, and lays it open on your desk — you may now use it.

Closing a file

The librarian saves any notes you made, then returns the book to the shelf. Skip this and notes can be lost.

seekg() / seekp()

You tell the librarian: “flip directly to page 50” — jump the bookmark anywhere instead of reading in order.

tellg() / tellp()

You ask: “what page is my bookmark on right now?” — the librarian reports the exact position.

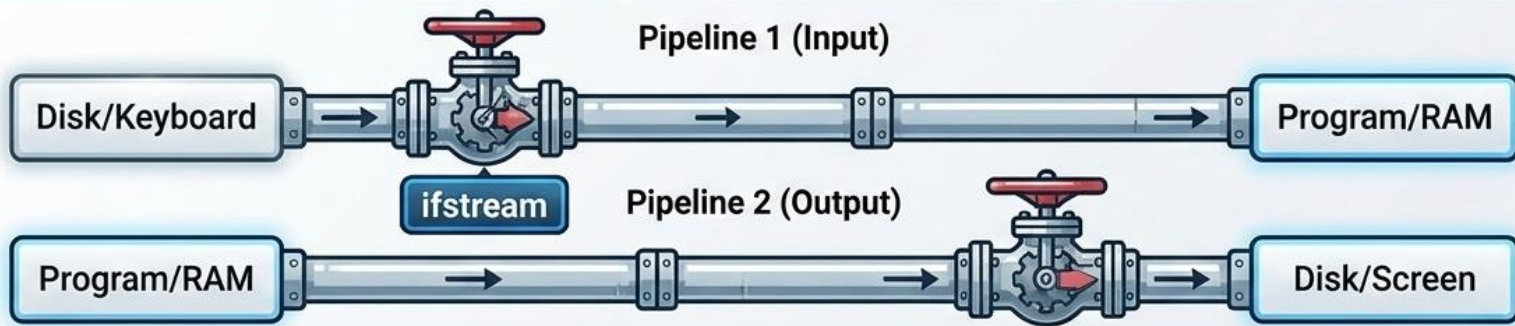
get() / put()

Reading or writing one word at a time, in natural order, exactly where the bookmark currently sits.

read() / write() of an object

Photocopying an entire filled-in form in one motion — every field, byte-for-byte — instead of retyping it.

The Stream Genealogy and Data Pipelines



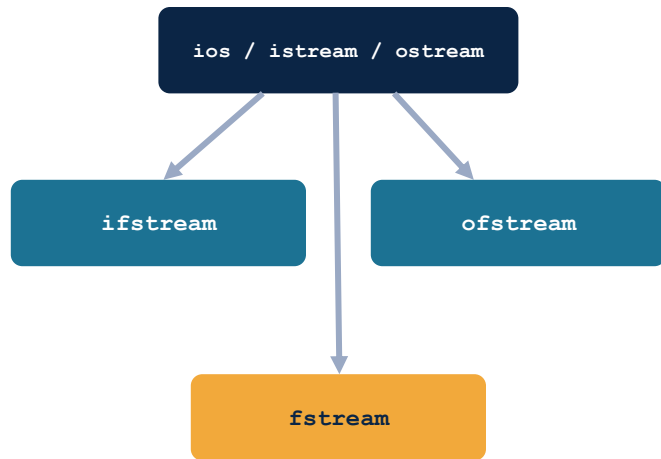
A stream is a continuous sequence of bytes flowing between a source and a destination using specialized `<fstream>` classes for type-safe data transfer.

The fstream Family of Classes

```
#include <fstream>
```

- `ifstream` — “input file stream”. Derived for reading data FROM a file. Default mode: `ios::in`.
- `ofstream` — “output file stream”. Derived for writing data TO a file. Default mode: `ios::out` (truncates existing content).
- `fstream` — combines both; can read AND write the same file, useful for random-access/update operations.
- All three inherit stream-formatting behaviour from `ios`, so the familiar `<<` (insertion) and `>>` (extraction) operators work exactly as they do with `cout` and `cin`.

SIMPLIFIED CLASS HIERARCHY



read-only stream

write-only stream

read + write stream

Establishing and Terminating Connections

Constructor Method

```
std::ofstream file("data.txt");
```

Implicitly opens upon object creation. Creates the file if it doesn't exist.

Open() Function Method

```
std::ofstream file;  
file.open("data.txt");
```

Explicit opening. Useful when reusing a single stream object for multiple different files.



```
file.close();
```

Critically **important**. Flushes any remaining buffer data to the disk and releases operating system locks.

Opening and Closing a File — The Big Picture

- “Opening” a file means associating a stream object (ifstream/ofstream/fstream) with a physical file on disk, so that operations on the object act on that file.
- C++ gives two ways to open a file: via the constructor (at the moment the object is created) or via the open() member function (after the object already exists).
- A file **MUST** be closed with close() when you're done. Closing flushes any buffered data to disk and releases the operating-system file handle.
- Forgetting to close a file can lose buffered (not-yet-written) data, or leave the file locked/inaccessible to other programs.
- Always verify the file actually opened — test the stream object in a boolean context: `if (!file) { /* failed */ }`.

FILE LIFECYCLE

1

Declare stream object

2

Open (constructor or open())

3

Check it opened successfully

4

Read and/or write data

5

Close the file (close())

Opening a File Using the Constructor

- Pass the filename (and optionally, the mode) directly to the stream object's constructor at the moment it is declared.
- **Syntax:**
 - `ClassName streamObj("filename", mode);`
- Most concise option — ideal when you know the exact file you need right away and won't need to reuse the object for another file.
- If the mode is omitted, ifstream defaults to ios::in and ofstream defaults to ios::out.
- Limitation: you cannot re-open a different file with the same object this way — for that, use open() instead (next slide).

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Constructor opens the file directly -
    // no separate open() call is needed
    ofstream outFile("data.txt");    // mode: ios::out
    ifstream inFile("data.txt");    // mode: ios::in

    outFile << "Created using constructor!";
    outFile.close();

    // Always verify the constructor actually
    // succeeded in opening the file
    if (!inFile) {
        cout << "File could not be opened!";
    }

    inFile.close();
    return 0;
}
```

Opening a File Using open()

- First declare the stream object with no file (default constructor), then bind it to a file later using open().
- **Syntax:**
 - `streamObj.open("filename", mode);`
- Lets you decide the filename at runtime (e.g., from user input) rather than hard-coding it at declaration.
- The SAME object can be reused: close() one file, then open() another — useful inside loops that process many files.
- The second argument (mode) can combine multiple flags with the bitwise-OR operator |, e.g. `ios::in | ios::out | ios::app`.

```
#include <fstream>
using namespace std;

int main() {
    fstream file;                // no file yet

    // Bind it to report.txt, allow reading,
    // writing, and always append new writes
    file.open("report.txt",
              ios::in | ios::out | ios::app);

    if (!file) { return 1; }    // safety check

    file << "New entry logged." << endl;
    file.close();

    // Reuse the SAME object for another file
    file.open("log.txt", ios::out | ios::trunc);
    file << "Fresh log started.";
    file.close();

    return 0;
}
```

Configuring the Pipeline: File Open Modes

Flag	Mode	Behavior
<code>ios::in</code>	Read 	Open for reading (Default for <code>ifstream</code>).
<code>ios::out</code>	Write 	Open for writing. Deletes existing content upon opening (Default for <code>ofstream</code>).
<code>ios::app</code>	Append 	All output is safely appended to the absolute end of the file. Preserves previous data.
<code>ios::trunc</code>	Truncate 	Deletes all contents if the file already exists upon opening.
<code>ios::binary</code>	Binary 	Opens in raw machine byte mode instead of formatted text mode.



Modes can be combined using the bitwise OR operator (`|`). Example:
`ios::out | ios::app` safely opens a file to add new data without erasing previous contents.

File Open Modes (`ios::`)

- Modes are flags defined in the `ios` class, combined with the bitwise-OR operator `|` when more than one behaviour is needed.

MODE FLAG	MEANING
<code>ios::in</code>	Open for reading (input). Fails if the file does not exist (for <code>ifstream</code>).
<code>ios::out</code>	Open for writing (output). Creates the file if absent; truncates (empties) it if present.
<code>ios::app</code>	Append. Every write goes to the end of the file, no matter where the pointer is.
<code>ios::ate</code>	“At End”. Pointer starts at end-of-file on open, but writes may still occur anywhere.
<code>ios::trunc</code>	Truncate. If the file exists, discard all its previous contents immediately.
<code>ios::binary</code>	Open in binary mode — bytes are transferred exactly as stored, no text translation.

Combine with `|` — e.g. `ios::out | ios::binary | ios::trunc` opens for writing, in binary mode, erasing any old content first.

PROGRAM 1

Sequential Write, Then Read (Text File)

Using `open()` to write student data, close it, then reopen the same file to read it back line by line.

● ● ● student.txt - write

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    // Step 1: open file in write mode
    ofstream outFile;
    outFile.open("student.txt", ios::out);
    if (!outFile) {
        cout << "Error opening file!";
        return 1;
    }

    // Step 2: write data (text I/O)
    outFile << "Roll No: 101" << endl;
    outFile << "Name: Aarav Sharma" << endl;
    outFile << "Marks: 87.5" << endl;

    outFile.close();           // Step 3
```

● ● ● student.txt - read

```
// Step 4: reopen the SAME file,
// this time for reading
ifstream inFile;
inFile.open("student.txt", ios::in);
if (!inFile) {
    cout << "Error reading file!";
    return 1;
}

// Step 5: read and display,
// line by line
string line;
cout << "----- File Contents -----"
    << endl;
while (getline(inFile, line)) {
    cout << line << endl;
}

inFile.close();               // Step 6
return 0;
}
```

What Actually Happens, Step by Step

1

Write phase

`outFile.open("student.txt", ios::out)` creates the file if it doesn't exist. Because mode is `ios::out`, any old content is discarded first.

2

Formatted output

`outFile << "Roll No: 101" << endl;` sends characters through the stream's insertion operator — identical syntax to `cout`, different destination.

3

`close()` the writer

Flushes the buffer to disk and releases the handle. Skipping this risks losing the last unflushed bytes and can block reopening.

4

Reopen to read

A NEW `ifstream` object opens the same filename with `ios::in`. Two different objects — one only ever writes, one only ever reads.

5

`getline()` loop

Reads up to the next `'\n'` each call, returns the stream so the while condition becomes false automatically at end-of-file.

CONSOLE OUTPUT

----- File Contents -----

Roll No: 101

Name: Aarav Sharma

Marks: 87.5

*student.txt on disk
now permanently contains
those 3 lines — they will
still be there after a
reboot, until deleted.*

seekg() and seekp()

- Every open file keeps two internal position markers: a GET pointer (for reading) and a PUT pointer (for writing).
- `seekg(offset, direction)` — moves the GET pointer. Think: “seek for get.”
- `seekp(offset, direction)` — moves the PUT pointer. Think: “seek for put.”
- **direction is one of three anchors:**
 - `ios::beg` — offset counted from the beginning of the file
 - `ios::cur` — offset counted from the CURRENT position
 - `ios::end` — offset counted backwards from the end of file
- offset can be negative when combined with `ios::cur` or `ios::end`, to move backwards.

```
fstream file;
file.open("alphabet.txt", ios::in | ios::out);

file.seekg(4, ios::beg);
// -> jump the READ pointer to byte index 4
//     (the 5th character, 0-indexed)

file.seekp(0, ios::beg);
// -> jump the WRITE pointer back to the start

file.seekg(-1, ios::cur);
// -> step the READ pointer 1 byte backwards
//     from wherever it currently is

file.seekg(-2, ios::end);
// -> position READ pointer 2 bytes before EOF
```

tellg() and tellp()

- tellg() returns the CURRENT position of the GET (read) pointer, as an integer byte offset from the start of the file.
- tellp() returns the CURRENT position of the PUT (write) pointer, in the same way.
- Both take no arguments and return a value of type streampos (safely convertible to a plain integer for display or arithmetic).
- Typical uses:
 - measuring the total size of a file — seekg(0, ios::end) then tellg()
 - remembering a position to seekg() back to later
 - confirming how many bytes were just written, via tellp()

```
fstream file;
file.open("alphabet.txt", ios::in | ios::out);

// A classic idiom: find total file size
file.seekg(0, ios::end); // jump to end
streampos size = file.tellg();
cout << "File size: " << size << " bytes";

// Confirm the write pointer's position
// right after writing 26 characters
cout << "Bytes written: " << file.tellp();

// Save a position, do other work, return to it
streampos mark = file.tellg();
// ... read elsewhere ...
file.seekg(mark); // jump straight back
```

Sequential Deep Dive: Characters and Lines

Formatted operators (<< and >>) skip whitespaces. For precise parsing, use **unformatted character** and line-level functions.

Track 1: **put()**



```
file.put('A');
```

Writes a single exact character to the stream.

Track 2: **get()**

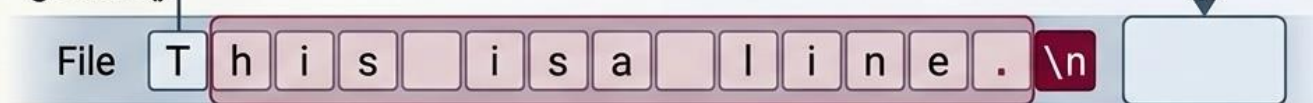


```
char c; file.get(c);
```

char c

Reads a single character, strictly including spaces and newlines.

getline()



```
getline(file, stringVar);
```

stringVar

Reads all characters continuously until it hits the newline delimiter.

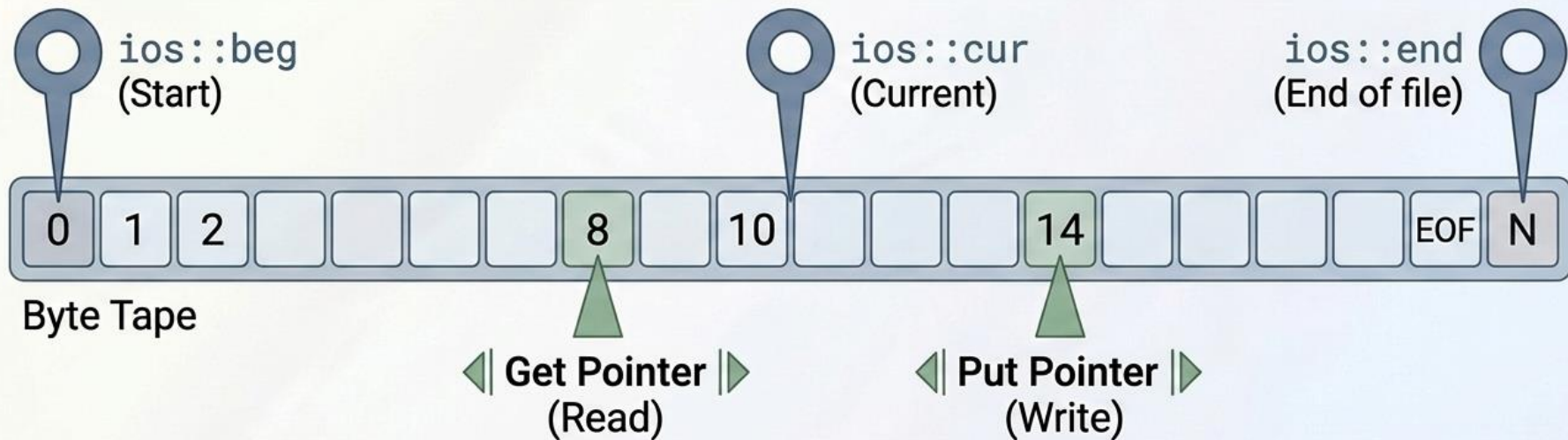
Get Pointer vs. Put Pointer, In One Picture



- Positions are 0-indexed — the first byte of the file is at offset 0, not 1.
- The GET and PUT pointers are independent on an fstream — you can read from the middle while a separate write happens at the end.
- `seekg()/seekp()` move a pointer; `tellg()/tellp()` report one; neither changes the file's actual content by itself.
- After `seekg()/seekp()`, the very next `get()/read()` or `put()/write()` happens starting exactly at that new position.

The Anatomy of Random Access

Sequential reading is like a VHS tape. Random Access is like a DVD:
jump instantly to any **exact byte offset** in the file.



C++ maintains two independent position pointers. You can command these pointers to jump to any specific byte address in the entire file.

PROGRAM 2

Random Access with seekg / seekp / tellg / tellp

Writes A-Z to a file, then jumps around it non-sequentially, reading and overwriting specific bytes.

● ● ● write A-Z, read index 4

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file;
    file.open("alphabet.txt",
              ios::out | ios::in | ios::trunc);

    for (char ch = 'A'; ch <= 'Z'; ch++)
        file.put(ch); // write A..Z
    cout << "Bytes written: "
         << (int)file.tellp() << endl;

    file.seekg(4, ios::beg); // -> index 4
    char letter;
    file.get(letter);
    cout << "5th character: " << letter << endl;
```

● ● ● overwrite + relative seeks

```
file.seekp(0, ios::beg); // -> index 0
file.put('#'); // overwrite 'A'

file.seekg(-1, ios::cur); // step back 1
file.get(letter);
cout << "After stepping back: " << letter;

file.seekg(-2, ios::end); // 2 before EOF
file.get(letter);
cout << "\n2nd-last character: " << letter;

file.close();
return 0;
}
```


The Random Access Toolkit

seekg() Seek-Get	seekp() Seek-Put
 Moves the read pointer. Parameters: offset (bytes), direction.	 Moves the write pointer. Parameters: offset (bytes), direction.
tellg() Tell-Get	tellp() Tell-Put
 Returns the current read pointer location in total bytes from the start.	 Returns the current write pointer location in total bytes from the start.

Rule of Thumb: Functions ending in 'g' manipulate the get (read) pointer. Functions ending in 'p' manipulate the put (write) pointer. Seek commands movement; Tell reports the address.

Tracing the Pointer, Call by Call

CALL	GET →	PUT →	RESULT
<code>put() loop</code>	GET: —	PUT: 26 (end)	File now holds A B C ... Z
<code>seekg(4, beg)</code>	GET: 4	PUT: 26	<code>get()</code> next reads 'E'
<code>seekp(0, beg) + put('#')</code>	GET: 5	PUT: 1	Index 0 overwritten: A → #
<code>seekg(-1, cur)</code>	GET: 4	PUT: 1	<code>get()</code> re-reads 'E'
<code>seekg(-2, end)</code>	GET: 24	PUT: 1	<code>get()</code> reads 'Y' (2 before 'Z')

Key insight: GET and PUT pointers are tracked completely independently. Reading at index 4 never affects the write position, and overwriting index 0 never affects the read position — that's what makes true random-access edit-in-place possible.

Sequential Access: `put()` and `get()`

- Sequential I/O means reading or writing data IN ORDER, one character at a time, from wherever the pointer currently sits — the natural, page-by-page way of using a file.
- `get(char &c)` — reads a SINGLE character from the file into `c`, then automatically advances the `get`-pointer by one.
- `put(char c)` — writes a SINGLE character `c` to the file, then automatically advances the `put`-pointer by one.
- Both are member functions of the stream object — `ifstream/fstream` have `get()`, `ofstream/fstream` have `put()`.
- Used in a loop, `get()` naturally stops returning valid data at end-of-file — the stream itself becomes `false` in a boolean context, which is what powers the classic `while (source.get(ch))` pattern.
- Contrast with `<<` and `>>`: those skip whitespace and interpret formatted data (numbers, words); `get()/put()` move raw individual characters, whitespace included — essential when the exact bytes matter.

Copying a File, Character by Character

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream source("source.txt");
    ofstream destination("backup.txt");
    char ch;

    // Read one character at a time until EOF,
    // then immediately write that same character
    while (source.get(ch)) {
        destination.put(ch);
    }

    cout << "File copied successfully "
         << "using get() and put().";

    source.close();
    destination.close();
    return 0;
}
```

HOW THE LOOP ENDS

- `source.get(ch)` reads one character and returns the stream object itself.
- In a while condition, the stream converts to a bool: true while extraction succeeded, false the instant it fails (i.e. end-of-file reached).
- So the loop needs no separate EOF check or counter — it stops itself naturally.
- Every character — letters, digits, spaces, blank lines — is copied exactly, because `get()/put()` never skip or interpret anything.

write() and read(): Moving Raw Bytes

- write() and read() transfer an entire BLOCK of bytes in one call, instead of one character at a time — far more efficient for large or structured data.
- **Syntax:**
 - `outFile.write(reinterpret_cast<char*>(&obj), sizeof(obj));`
 - `inFile.read(reinterpret_cast<char*>(&obj), sizeof(obj));`
- Both take: a char* pointer to where the bytes start, and a byte count (usually sizeof(...) of the data being moved).
- Why reinterpret_cast<char*>? write()/read() only understand raw bytes (char*) — this cast tells the compiler “treat this object's memory as a plain sequence of bytes”, without converting or reformatting anything.
- Must be paired with ios::binary. In text mode, some bytes could be silently translated, corrupting non-text data such as a float or an int.
- This is exactly the tool needed to save/load whole variables, arrays, or class objects in a single, fast operation.

Reading and Writing Class Objects

- A class object in RAM is just a contiguous block of memory holding its data members, one after another (roughly — padding/alignment aside).
- `write(reinterpret_cast<char*>(&obj), sizeof(obj))` copies that ENTIRE block — every data member, in one call — to the file exactly as it sits in memory.
- `read(reinterpret_cast<char*>(&obj), sizeof(obj))` copies bytes from the file back into an object's memory, reconstructing every member at once.
- **Requirements for this to work correctly:**
 - the file must be opened in `ios::binary` mode on both the writing and reading side
 - the reading program must use the exact same class definition (same members, same order, same types) as the writer
 - member functions themselves are never written to the file — only the DATA members are part of an object's memory footprint
- Multiple objects can be written back-to-back (e.g., in a loop) to build a simple binary “records” file — and later `seekg(n * sizeof(obj))` can jump straight to the n-th record.

Student OBJECT IN RAM

`rollNo` `int`

`name[30]` `char[]`

`marks` `float`

`write(&obj, sizeof(obj))`

students.dat (binary)

```
[ 65 00 00 00 | 41 61 72 61 76 00 ... | 00 00  
AE 42 ]
```

The exact byte layout of the object, unchanged — not human-readable, but perfectly restorable with `read()`.

The Paradigm Shift: Text vs. Binary

Complex class objects cannot be safely saved as plain text.

Text Files (Human Readable)

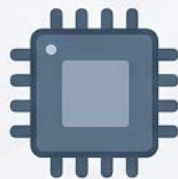


[9]	[9]	[9]	[9]	[9]
-----	-----	-----	-----	-----

Consumes 5 bytes of disk space (one for each character).

- Easily editable in Notepad.
- Inefficient for storage.
- Requires parsing (string-to-int conversion).
- Syntax: `<<` and `>>`

Binary Files (Machine Readable)

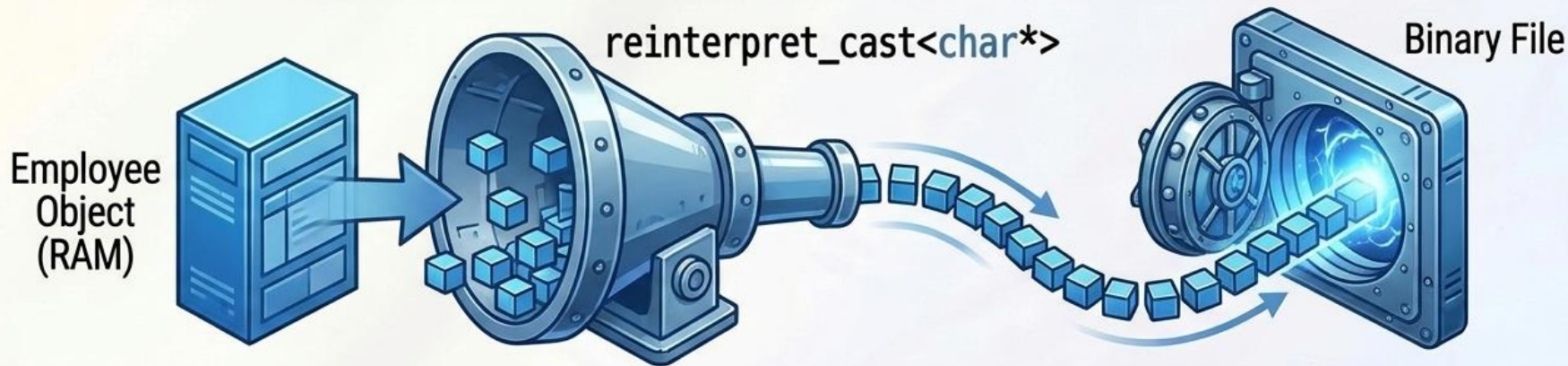


[0x01]	[0x86]	[0x9F]	[0x00]
--------	--------	--------	--------

Consumes exactly 4 bytes (the raw memory size of an int).

- Lightning fast.
- Exact mirror of system RAM.
- Mandatory for saving complex Class Objects.
- Syntax: `write()` and `read()`

Object Serialization Rules



The 3 Golden Rules

1. **The Casting Syntax:** `file.write((char*)&obj, sizeof(obj));` You must cast the object's memory address to a character pointer so the stream can process it byte-by-byte.
2. **No Pointers Allowed:** Never serialize an object containing pointers (like `std::string`). Pointers point to RAM addresses that will be completely invalid the next time the program runs. Use fixed-size arrays (e.g., `char name[50]`).
3. **Exact Sizing:** Always use `sizeof(Class)` to guarantee you are reading/writing the exact byte footprint, mathematically accounting for compiler padding.

Program 3: Serializing Class Objects

```
1 class Employee {  
2     int id;  
3     char name[50];  
4 };  
  
5  
6 Employee emp = {1, "Ada"};  
7 ofstream outFile("emp.dat", ios::binary);  
8 outFile.write(reinterpret_cast<char*>(&emp), sizeof(Employee));  
9 outFile.close();  
10  
11 Employee tempEmp;  
12 ifstream inFile("emp.dat", ios::binary);  
13 inFile.read(reinterpret_cast<char*>(&tempEmp), sizeof(Employee));
```

Use orthogonal line:

Uses a **fixed-size** char array instead of dynamic `std::string` for absolute binary safety.

Mandatory `ios::binary` flag

Mandatory `ios::binary` flag prevents any alteration of the raw memory bytes.

Shrink-wraps the 'emp' object into a raw byte stream and writes its exact physical size directly to the disk.

Reads a chunk of bytes exactly the size of an Employee, pasting it directly into the RAM footprint of 'tempEmp'.

PROGRAM 3

Writing and Reading a Class Object (Binary)

A complete Student class persisted to disk with write(), then fully reconstructed with read() into a second object.

the Student class

```
#include <iostream>
#include <fstream>
using namespace std;

class Student {
    int rollNo;
    char name[30];
    float marks;
public:
    void getData() {
        cout << "Roll, Name, Marks: ";
        cin >> rollNo >> name >> marks;
    }
    void showData() {
        cout << rollNo << "\t" << name
             << "\t" << marks << endl;
    }
};
```

main() - write, then read

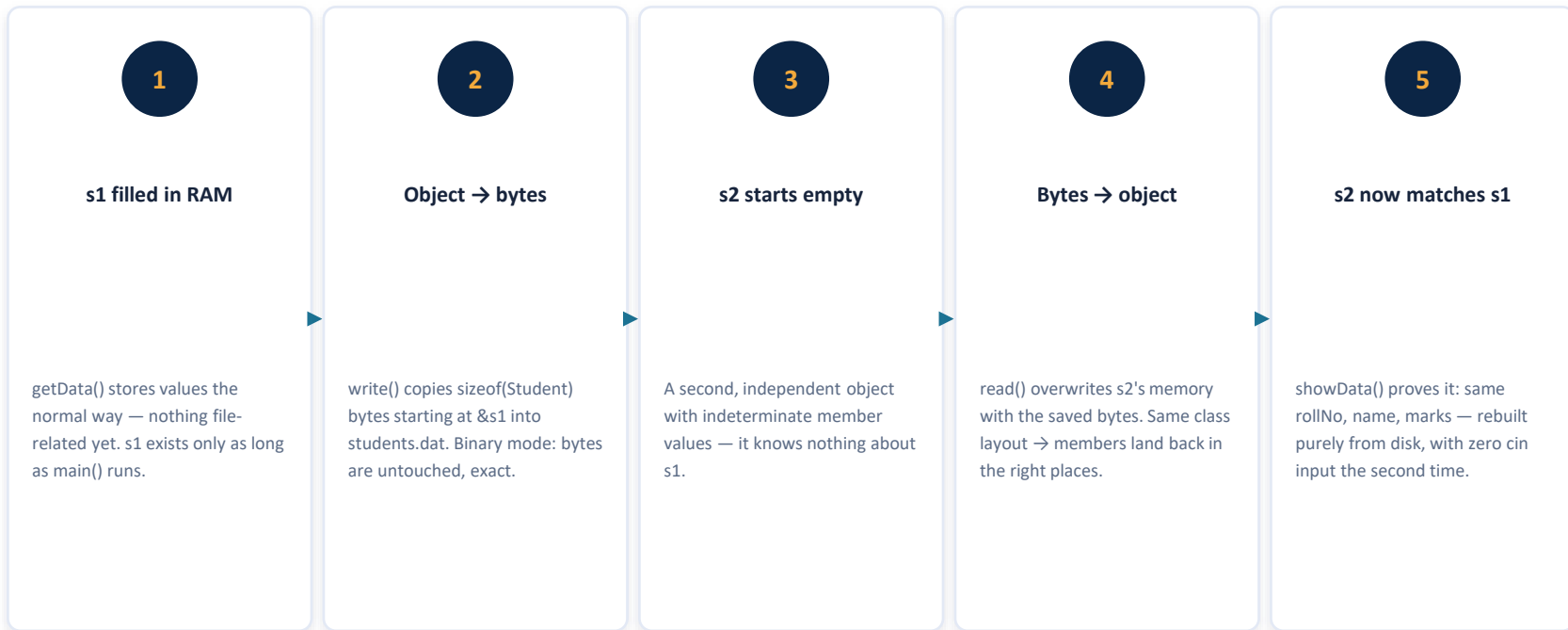
```
int main() {
    Student s1;
    s1.getData();

    ofstream outFile("students.dat",
                    ios::out | ios::binary);
    outFile.write(reinterpret_cast<char*>(&s1),
                  sizeof(Student));
    outFile.close();

    Student s2; // empty, untouched object
    ifstream inFile("students.dat",
                    ios::in | ios::binary);
    inFile.read(reinterpret_cast<char*>(&s2),
                sizeof(Student));
    inFile.close();

    cout << "\nRead back from file:\n";
    s2.showData();
    return 0;
}
```

Why This Actually Rebuilds the Object



This pattern — write()/read() with sizeof(class) — is how simple flat-file record stores are built before a real database is introduced.

Everything We Covered, At a Glance

Opening & Closing

Constructor opens inline; `open()` opens after declaration and allows reuse. Always `close()`, always check success.

Modes

`ios::in / out / app / ate / trunc / binary` — combine with `|` to control exactly how a file behaves on open.

Positioning

`seekg()/seekp()` move the read/write pointer to any byte; `tellg()/tellp()` report where it currently sits.

Sequential I/O

`get()/put()` move one character at a time, in order, including whitespace — the basis of a byte-exact file copy.

Binary Block I/O

`write()/read()` move a whole block of raw bytes (via `sizeof`) in one call — fast, and required for non-text data.

Class Objects

A full object's memory can be saved and restored intact with `write()/read()`, as long as both sides share the same class layout and binary mode.

File Handling — Complete Overview

- ifstream — file input (reading)
- ofstream — file output (writing)
- fstream — both reading and writing
- File Modes:
 - ios::in = read | ios::out = write | ios::app = append
 - ios::trunc = overwrite | ios::binary = binary mode
- File Pointers:
 - seekg(n, ios::beg/cur/end) — move read pointer
 - seekp(n, ios::beg/cur/end) — move write pointer
 - tellg() / tellp() — get current position

```
#include <fstream>
using namespace std;

int main() {
    ofstream out("data.txt");
    out << "Hello File!" << endl;
    out.close();

    ifstream in("data.txt");
    if (!in) { cout << "Error!"; }
    if (in.good()) cout << "OK";
    if (in.eof()) cout << "EOF";
    in.close();
}
```

Ch.10 Analogy & PU Question



Real-World Analogy

File pointers are like bookmarks in a physical book. `seekg()` moves your reading bookmark, `seekp()` moves your writing bookmark. `tellg()` tells you which page you're on. `ios::app` is like opening a diary and writing at the last used page — you never overwrite old entries.



Probable PU Exam Question

Q: Write a C++ program to read student records from a file, search by roll number, and display the result. Explain file pointer manipulation functions with examples. [PU Exam 2022]

Thank You!

OOP with C++ — Complete Course Module